

프로젝트 개요

3D 멀티플레이 격투 액션 게임

프로젝트 타입 : 3D 멀티플레이 격투 액션 게임(플랫폼 미정)

플레이 타입 : PvP 멀티플레이

제작사 : 알서포트 주식회사

제작인원 : 8명(2025.10월 기준)

- 🎮 유니티 클라이언트 개발자 4명
- 🎨 3D 디자인 및 UI디자인 3명
- 🌐 백엔드 1명

참여 역할 : 유니티 클라이언트 개발

프로젝트 참여 기간 : 2025.01 - 2025.10

플랫폼 : 플랫폼 미정(프로토타입은 WebGL)

엔진 및 도구 : Unity3D, C#, Git

네트워킹 라이브러리 : [Fusion2-Shared](#) ↗

기여 요약

클라이언트 개발 항목

- 1 [HeroNetworkSystem](#) : 영웅 동기화 시스템
- 2 [HeroControlSystem](#) : 영웅 조작 시스템
- 3 HeroAbilitySystem : 영웅 능력 시스템
- 4 HeroSkillSystem : 영웅 스킬 시스템
- 5 HeroVisualSystem : 영웅 외관시스템
- 6 HeroVFXSystem : 영웅 특수효과 시스템

영웅 동기화 시스템 (Hero Network System) 소개

Photon Fusion 기반 영웅 네트워크 동기화 시스템

1. 기능 개요

- **HeroNetworkSystem**은 RFist 게임의 **영웅 캐릭터 네트워크 동기화 시스템**으로, Photon Fusion2-Shared 기반의 네트워크 환경에서 영웅 오브젝트의 수명주기, 상태, 입력을 관리하고 동기화하는 시스템입니다.

개발 배경 및 요구사항

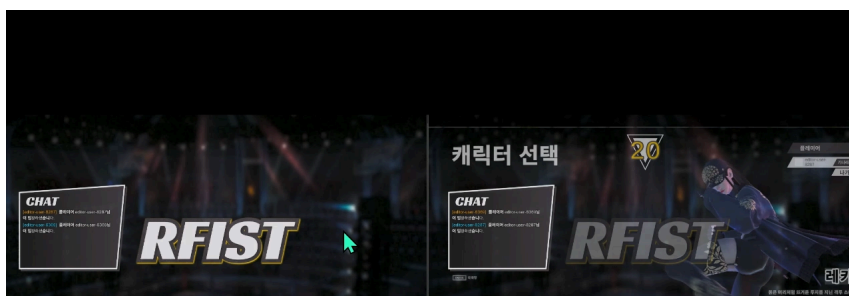
- 네트워크 환경에서 영웅 오브젝트의 일관된 상태 동기화가 필요
- 로컬/원격 플레이어 간 입력 지연 최소화 및 부드러운 동기화
- HP, SP, 경직 포인트 등 전투 상태의 실시간 네트워크 동기화
- 영웅 능력(Ability) 및 프로필 정보의 네트워크 동기화
- 이벤트 기반 아키텍처를 통한 느슨한 결합 및 확장성 확보

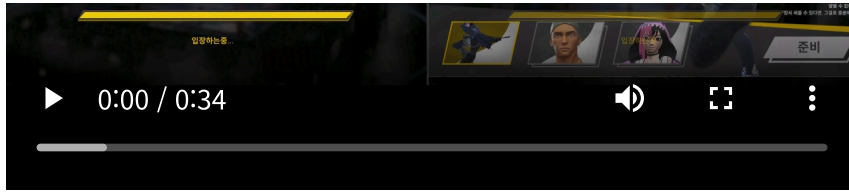
주요 기능

기능	설명
영웅 오브젝트 수명주기 관리	스폰/디스폰, 능력 변경, 컴포넌트 초기화 조율
네트워크 상태 동기화	HP, SP, 경직 포인트, 자세(Stance) 등 상태 네트워크 동기화
네트워크 입력 처리	로컬 입력 수집 → 네트워크 전송 → 모든 클라이언트에서 실행
이벤트 기반 통신	영웅/스킬/공격/매치/컷씬 이벤트의 중앙 관리 및 브로드캐스트

제작기능 영상

1. 캐릭터셋업, 입력 브로드캐스팅, 상태동기화





2. 사용된 기술 요소

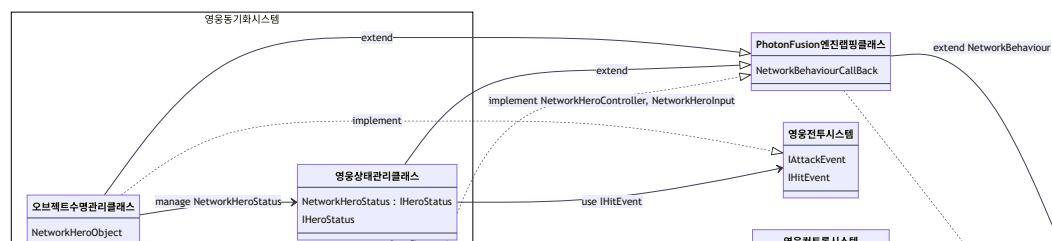
핵심 기술 요소 및 API 활용

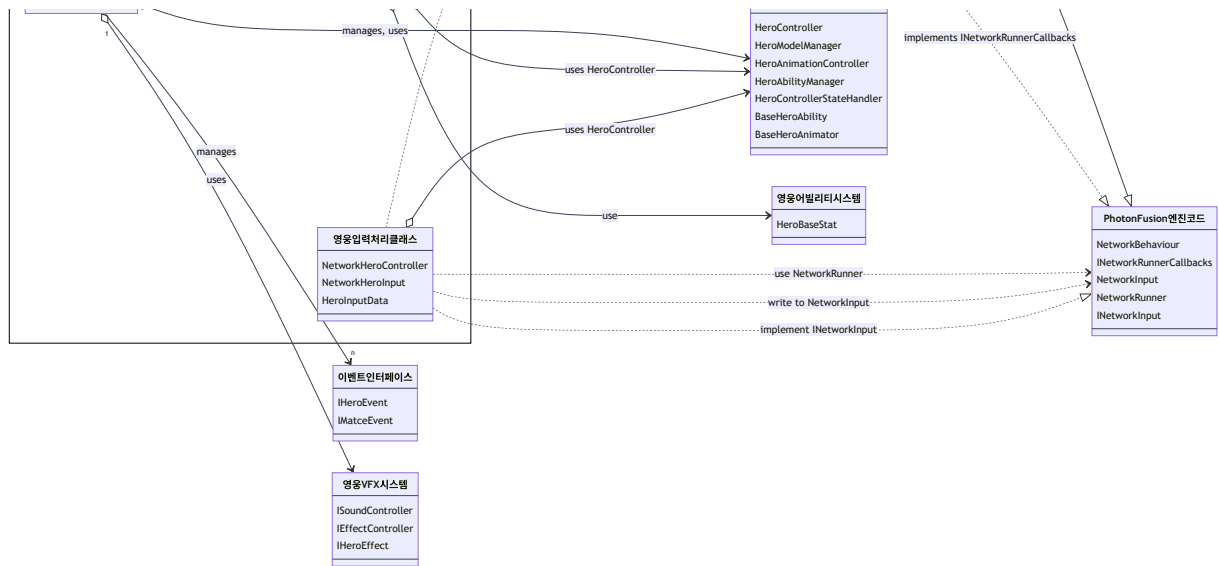
요소	설명
C#	전체 핵심 로직 및 유니티 컴포넌트 구현
Photon Fusion	실시간 네트워크 동기화 및 멀티플레이어 게임 엔진
└ NetworkBehaviour	Fusion의 네트워크 동기화 기반 클래스
└ RPC (Remote Procedure Call)	원격 프로시저 호출을 통한 네트워크 이벤트 전파
└ ChangeDetector	네트워크 상태 변경 감지 및 처리
└ NetworkInput	입력 데이터 네트워크 동기화

설계 활용 패턴

요소	설명
Facade Pattern	NetworkHeroObject 가 복잡한 하위 컴포넌트들을 캡슐화하여 단순한 인터페이스 제공
Observer Pattern	이벤트 기반 통신으로 상태 변경 시 관련 컴포넌트에 자동 알림
RPC(Network)	입력 권한을 가진 클라이언트에서 서버로, 서버에서 모든 클라이언트로 명령 전파
State Pattern Pattern(Network)	[Networked] 속성을 통한 자동 상태 동기화 및 ChangeDetector 로 변경 감지

3. 전체 시스템 구조도(간략)





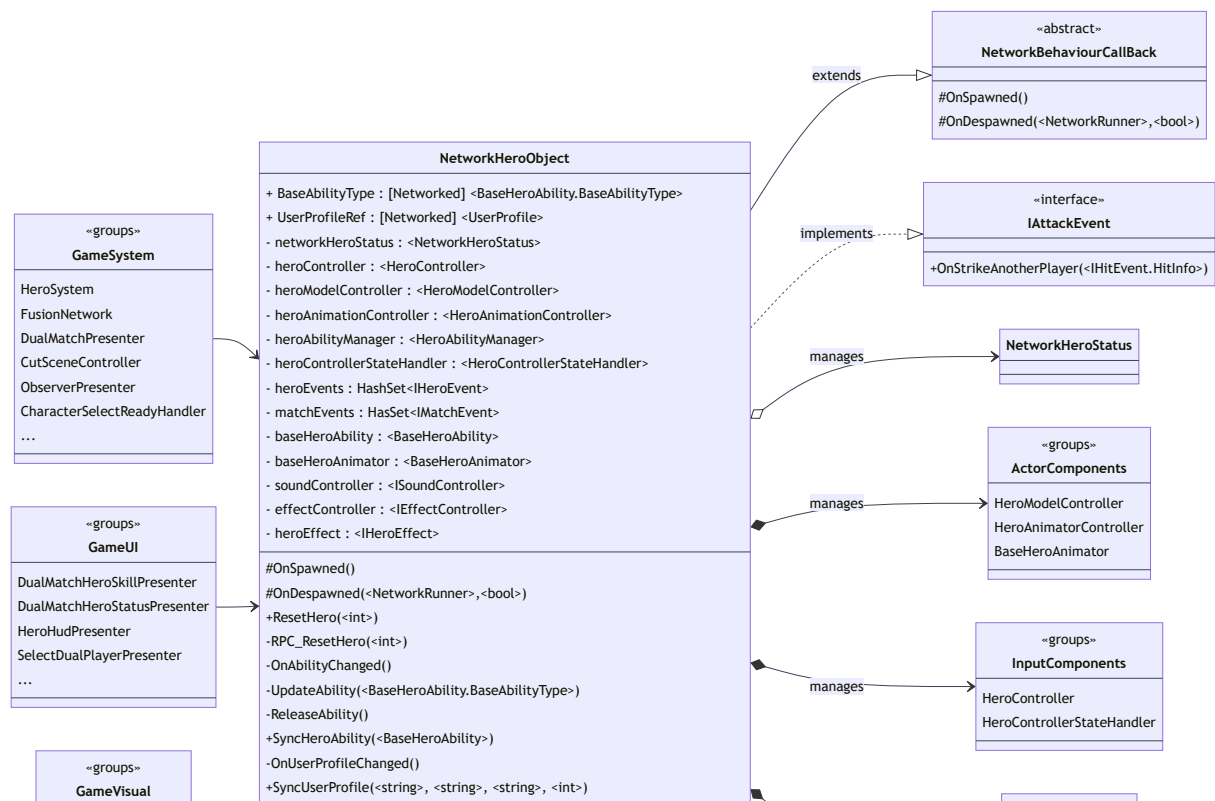
4. 주요 클래스별 역할 및 관계

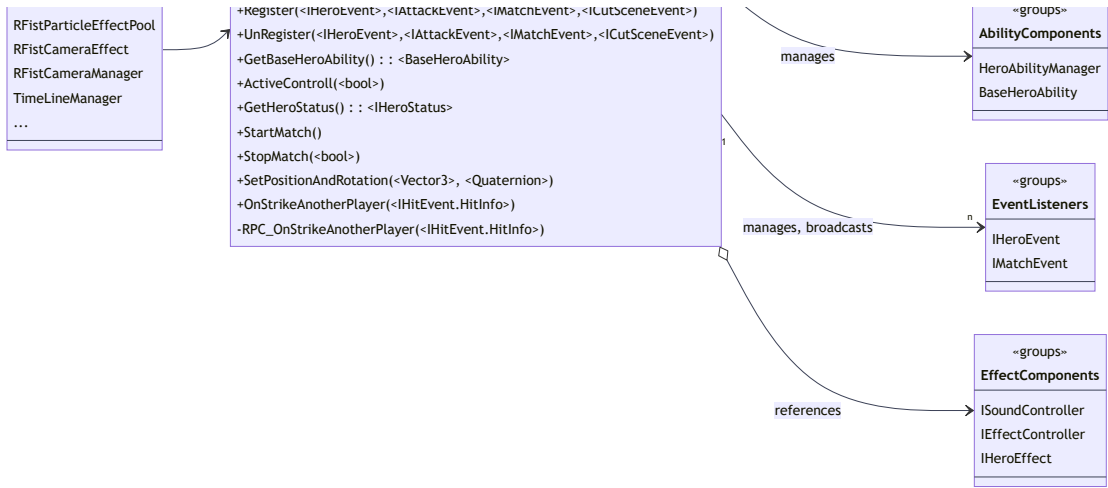
수명주기 및 네트워크 동기화 관리

클래스 역할

NetworkHeroObject

- 💡 Facade: 영웅의 서브시스템을 단순화된 통합 인터페이스로 제공
- 💡 영웅 오브젝트의 전체 수명주기(스폰/디스폰) 관리
- 💡 능력(Ability) 및 프로필 네트워크 동기화
- 💡 모든 하위 컴포넌트 초기화 및 조율
- 💡 **IHeroEvent, IMatchEvent** 이벤트 라우팅 중앙화
(**Register / UnRegister**)
- 💡 외부 시스템에서 영웅 객체에 접근하는 진입점





상태 관리

클래스

역할

IHeroStatus

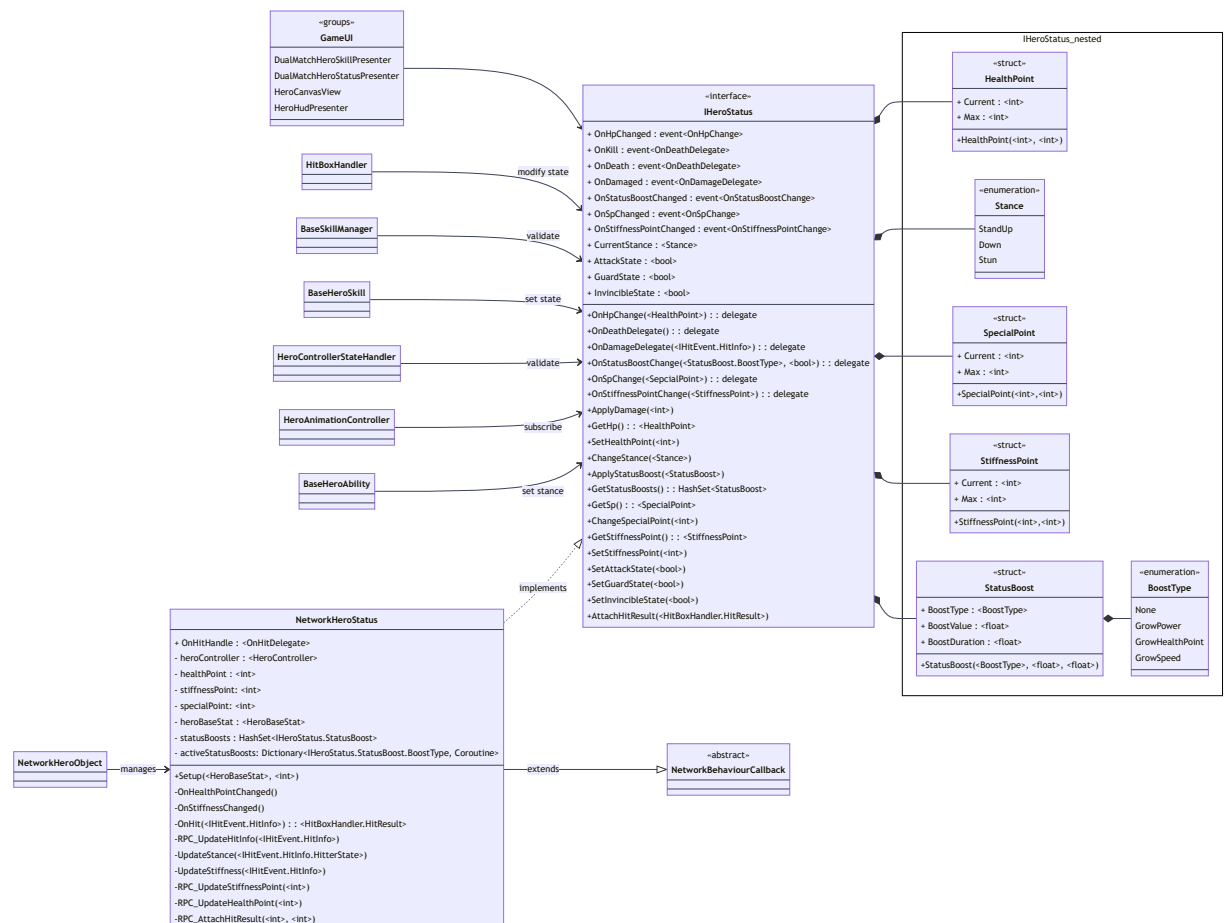
«interface»

- 💡 영웅 상태 관리의 인터페이스 계약 정의
- 💡 HP, SP, 경직 포인트, 자세(Stance), 상태 효과 (StatusBoost) 관리 메서드 정의
- 💡 상태 변경 이벤트 (**OnHpChanged**, **OnDeath** 등) 정의

NetworkHeroStatus

: NetworkBehaviourCallback,
IHeroStatus

- 💡 **IHeroStatus** 인터페이스의 네트워크 구현체
- 💡 HP/SP/경직 포인트 네트워크 동기화(RPC)
- 💡 히트 처리 및 데미지 계산
- 💡 상태 효과(Status Boost) 및 타이머 관리



인려 처리

클래스

역할

NetworkHeroInput

: *NetworkBehaviourCallback*

- 💡 로컬 플레이어 입력 수집 및 네트워크 전송
- 💡 **NetworkInput** 시스템을 통한 입력 데이터 동기화
- 💡 이동, 공격, 가드, 대시, 궁극기 등 다양한 입력 처리

NetworkHeroController

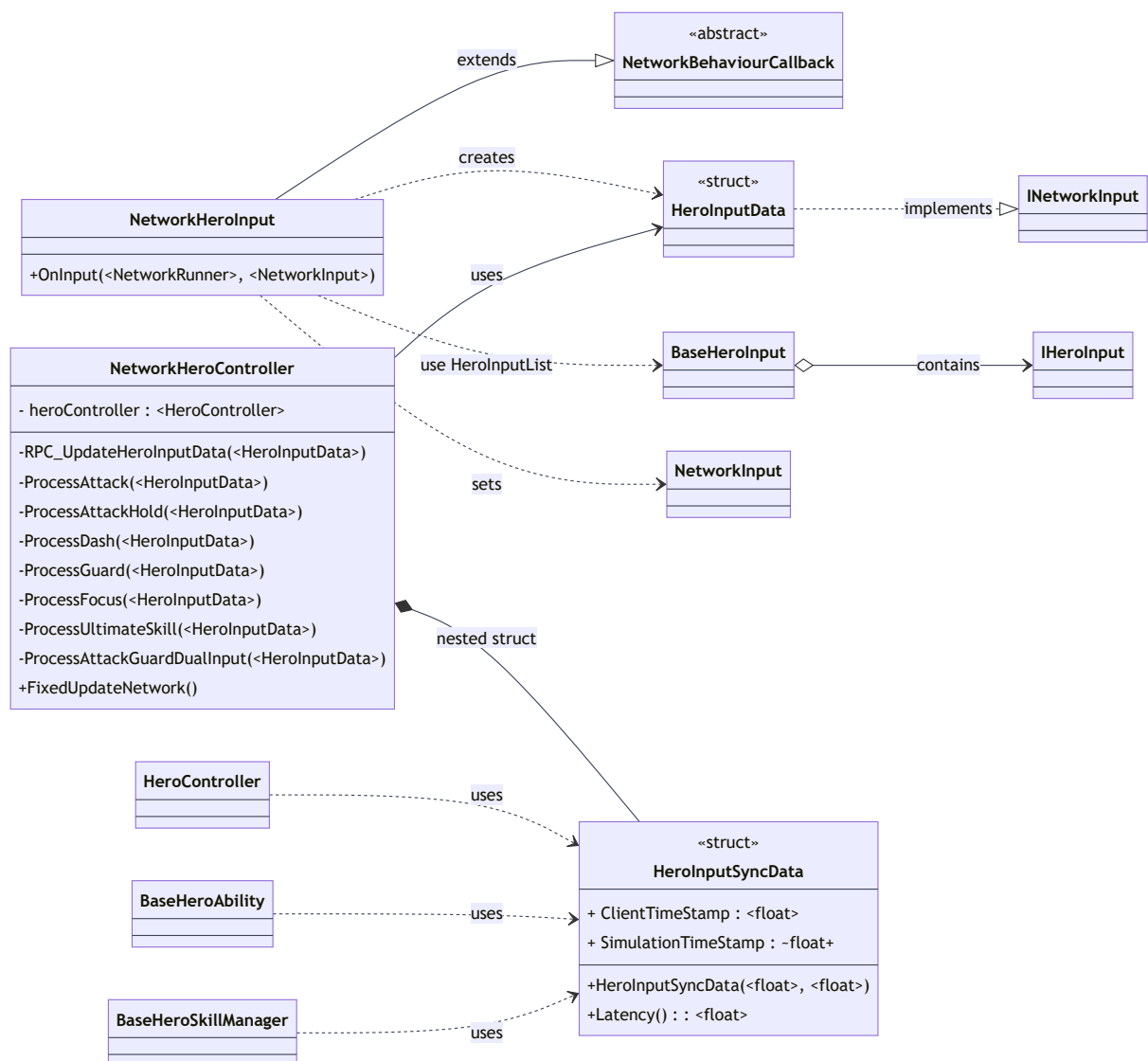
: *NetworkBehaviour*

- 💡 네트워크로부터 입력 수신 및 실제 게임 로직 실행
- 💡 RPC를 통한 입력 데이터 브로드캐스트
- 💡 네트워크 지연(Latency) 측정을 위한 타임스탬프 동기화

HeroInputData

: *INetworkInput*

- 💡 **Fusion 네트워크 전송용 입력 데이터 컨테이너**
- 💡 비트 마스크를 활용한 효율적인 버튼 상태 표현
- 💡 **INetworkInput** 구현체로 자동 직렬화 지원



5. 주요 특징

기능의 특징

- 네트워크 상태 동기화: **[Networked]** 속성과 **ChangeDetector** [🔗](#)를 활용한 자동 상태 동기화 및

변경 감지

- 이벤트 기반 아키텍처: `IHeroEvent`, `ISkillEvent`, `IAttackEvent` 등 인터페이스 기반 이벤트 시스템으로 느슨한 결합 구현
- 파사드 패턴 적용: `NetworkHeroObject`가 하위 컴포넌트들을 캡슐화하여 외부에 단순한 인터페이스 제공
- 로컬 입력 브로드캐스팅: `NetworkHeroInput` → `NetworkHeroController` → `HeroController` 흐름으로 로컬 입력을 처리
- RPC 기준 제어: 입력 권한(`InputAuthority`)을 가진 클라이언트만 RPC 호출 가능하도록 제한
- 컴포넌트 자동 초기화: `UpdateAbility()` 메서드에서 능력 변경 시 모든 관련 컴포넌트를 자동으로 재 설정

6. UseCase

영웅 스폰 및 능력 동기화 시나리오

- 1 스폰: `NetworkHeroObject`가 네트워크에 스폰되면 `OnSpawned()` 호출
- 2 프로필 동기화: `SyncUserProfile()`로 사용자 정보(이름, 팀, 프로필 이미지) 동기화
- 3 능력 동기화: `SyncHeroAbility()`로 영웅 능력 타입 동기화 → `BaseAbilityType` 변경 감지
- 4 컴포넌트 초기화: `OnAbilityChanged()` → `UpdateAbility()`에서 모든 하위 컴포넌트 설정

전투 상태 동기화 시나리오

- 1 피격: `HitBoxHandler`에서 `OnHitHandle` 델리게이트 호출
- 2 데미지 적용: `NetworkHeroStatus.OnHit()`에서 데미지 계산 및 `RPC_UpdateHitInfo()` 호출
- 3 상태 변경: 모든 클라이언트에서 HP 감소 및 자세(Stance) 변경
- 4 이벤트 발생: `OnHpChanged`, `OnDamaged` 이벤트 발생 → UI 업데이트
- 5 사망 처리: HP가 0 이하가 되면 `OnDeath` 이벤트 발생 → 컨트롤러 비활성화

네트워크 입력 처리 시나리오

- 1 입력 수집: `NetworkHeroInput.OnInput()`에서 로컬 입력 수집
- 2 입력 전송: `NetworkInput`을 통해 네트워크로 입력 데이터 전송
- 3 입력 수신: `NetworkHeroController.FixedUpdateNetwork`에서 입력 수신
- 4 RPC 브로드캐스트: `RPC_UpdateHeroInputData()`로 모든 클라이언트에 입력 동기화
- 5 동작 실행: 모든 클라이언트에서 동일한 입력으로 `HeroController` 메서드 호출

주요 사용처

- 멀티플레이어 대전 시스템: 1v1 또는 팀 대전에서 영웅 캐릭터 동기화
- 관전 시스템: 원격 플레이어의 영웅 상태 및 동작 실시간 관전
- 네트워크 지연 최소화: 클라이언트에서 영웅 상태 동기화 및 동기화

- **배치 매핑:** 더큰 디스크/하드 디스크 공간에 소기짜 컷 할기짜

NetworkHeroObject

Photon Fusion 네트워크 환경에서 영웅 오브젝트의 수명주기를 관리하는 메인 클래스

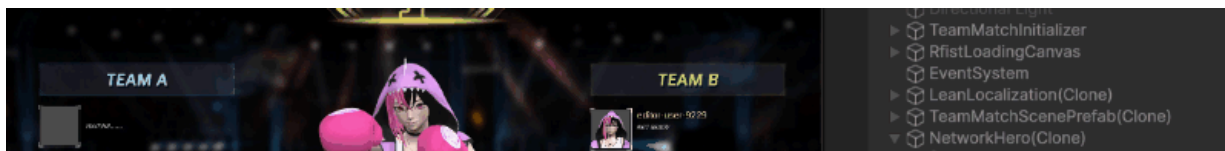
개요

- `NetworkHeroObject` 클래스는 Photon Fusion의 `NetworkBehaviour` [🔗](#)를 랩핑한 `NetworkBehaviourCallback` 클래스를 상속받아, 영웅 오브젝트의 전체 수명주기를 관리하는 메인 클래스입니다.
- 해당 클래스는 모든 영웅 컴포넌트들을 통합 관리하고 네트워크 동기화를 담당합니다. 해당 클래스는 파사드 패턴을 적용하여 `NetworkHeroStatus`, `HeroController`, `HeroModelController`, `HeroAnimationController` 등의 복잡한 하위 컴포넌트들을 캡슐화합니다.
- 이벤트 기반 통신은 `NetworkHeroObject`를 통해 중앙 관리되며, 하위 컴포넌트 직접 접근이 필요한 경우 `GetXXX()` 메서드를 통해 제한적으로 노출합니다. 네트워크를 통해 플레이어의 프로필, 능력 등을 모든 클라이언트에 동기화하며, 각 하위 컴포넌트들의 초기화와 조율을 담당합니다.

역할

- 영웅 오브젝트의 스폰/디스폰 수명주기 관리
- 모든 영웅 컴포넌트 초기화 및 설정 조율
- 네트워크를 통한 사용자 프로필 동기화
- 네트워크를 통한 영웅 능력(Ability) 동기화
- 다양한 이벤트 리스너 등록/해제 관리
- 채팅 및 이모티콘 네트워크 동기화
- 매치 시작/종료 이벤트 브로드캐스트
- 하위 컴포넌트 제한적 노출 (`GetXXX()` 메서드 제공)
 - `GetHeroStatus()`, `GetBaseHeroAbility()`, `GetHeroModelController()` 등
 - 외부에서 필요한 경우에만 직접 접근 가능
- 이벤트 라우팅 중앙화
 - `Register()/UnRegister()` 메서드를 통한 이벤트 등록/해제
 - `heroController`에 이벤트 리스너 연결 및 관리

네트워크 캐릭터 셋업





선언

```
public class NetworkHeroObject : NetworkBehaviourCallback, IHeroObjectInfo, IHeroObjectControl
```

멤버

필드

```
/// <summary>
/// 네트워크 동기화되는 영웅 상태 컴포넌트
/// </summary>
[SerializeField]
private NetworkHeroStatus networkHeroStatus;

/// <summary>
/// 영웅 동작을 제어하는 컨트롤러
/// </summary>
[SerializeField]
private HeroController heroController;

/// <summary>
/// 영웅 3D 모델을 관리하는 컨트롤러
/// </summary>
[SerializeField]
private HeroModelController heroModelController;

/// <summary>
/// 영웅 애니메이션을 관리하는 컨트롤러
/// </summary>
[SerializeField]
private HeroAnimationController heroAnimationController;

/// <summary>
/// 영웅 능력(Ability)을 생성하고 관리하는 매니저
/// </summary>
[SerializeField]
private HeroAbilityManager heroAbilityManager;

/// <summary>
/// 영웅 모델의 Transform
/// </summary>
[SerializeField]
private Transform modelTransform;
```

```

/// <summary>
/// 채팅 UI 위치를 위한 Transform
/// </summary>
[SerializeField]
private Transform chattingTransform;

/// <summary>
/// 카메라 타겟 Transform
/// </summary>
[SerializeField]
private Transform cameraTargetTransform;

/// <summary>
/// 영웅 컨트롤러의 상태를 관리하는 핸들러
/// </summary>
[SerializeField]
private HeroControllerStateHandler heroControllerStateHandler;

/// <summary>
/// 현재 영웅의 능력 컴포넌트
/// </summary>
private BaseHeroAbility _baseHeroAbility;

/// <summary>
/// 현재 영웅의 애니메이터
/// </summary>
private BaseHeroAnimator _baseHeroAnimator;

/// <summary>
/// 사운드 컨트롤러
/// </summary>
private ISoundController _soundController;

/// <summary>
/// 이펙트 컨트롤러
/// </summary>
private IEffectController _effectController;

/// <summary>
/// 영웅 이펙트 인터페이스
/// </summary>
private IHeroEffect _heroEffect;

```

네트워크 동기화 속성

```

/// <summary>
/// 네트워크 동기화되는 영웅 능력 타입
/// </summary>
[Networked]
public BaseHeroAbility.BaseAbilityType BaseAbilityType { get; private set; }

/// <summary>

```

```

/// 네트워크 동기화되는 사용자 프로필 정보
/// </summary>
[Networked]
private ref UserProfile UserProfileRef => ref MakeRef<UserProfile>();

```

메서드

```

/// <summary>
/// 영웅을 리셋합니다.
/// 입력 권한이 있는 플레이어만 호출 가능합니다.
/// </summary>
public void ResetHero(int round = 0)

/// <summary>
/// 영웅 리셋을 모든 클라이언트에 동기화하는 RPC
/// </summary>
/// <param name="round">현재 라운드 번호</param>
[Rpc(RpcSources.InputAuthority, RpcTargets.All)]
private void RPC_ResetHero(int round)

/// <summary>
/// 영웅의 HUD 스프라이트를 가져옵니다.
/// </summary>
/// <returns>영웅 스프라이트</returns>
public Sprite GetHeroSprite()

/// <summary>
/// 영웅 능력 타입이 변경되었을 때 호출됩니다.
/// </summary>
private void OnAbilityChanged()

/// <summary>
/// 영웅 능력을 업데이트합니다.
/// 기존 능력 해제 후 새로운 능력 생성 및 설정
/// </summary>
/// <param name="abilityType">설정할 능력 타입</param>
private void UpdateAbility(BaseHeroAbility.BaseAbilityType abilityType)

/// <summary>
/// 현재 영웅 능력을 해제합니다.
/// 모든 이벤트 등록 해제 및 능력 오브젝트 파괴
/// </summary>
private void ReleaseAbility()

/// <summary>
/// 영웅 능력을 네트워크에 동기화합니다.
/// </summary>
public void SyncHeroAbility(BaseHeroAbility heroAbility)

/// <summary>
/// 사용자 프로필을 네트워크에 동기화합니다.
/// </summary>
public void SyncUserProfile(string userId, string userName, string userProfile

```



```
/// <summary>
/// 원격 플레이어(상대방)인지 확인합니다.
/// </summary>
public bool IsRemote()

/// <summary>
/// 사용자 이름을 가져옵니다.
/// </summary>
/// <returns>사용자 이름</returns>
public string GetUserName();

/// <summary>
/// 사용자 프로필 이미지 URL을 가져옵니다.
/// </summary>
/// <returns>프로필 이미지 URL</returns>
public string GetUserProfileImageUrl()

/// <summary>
/// Fusion Player ID를 가져옵니다.
/// </summary>
/// <returns>플레이어 ID</returns>
public int GetFusionId()

/// <summary>
/// 현재 위치를 가져옵니다.
/// </summary>
/// <returns>월드 좌표 위치</returns>
public Vector3 GetPosition()

/// <summary>
/// 현재 회전을 가져옵니다.
/// </summary>
/// <returns>회전 값</returns>
public Quaternion GetRotation()

/// <summary>
/// 현재 영웅 능력을 가져옵니다.
/// </summary>
/// <returns>영웅 능력 컴포넌트</returns>
public BaseHeroAbility GetBaseHeroAbility()

/// <summary>
/// 컨트롤 활성화/비활성화를 설정합니다.
/// </summary>
/// <param name="isEnabled">활성화 여부</param>
public void ActiveControl(bool isEnabled)

/// <summary>
/// 모델 Transform을 가져옵니다.
/// </summary>
/// <returns>모델 Transform</returns>
public Transform GetModelTransform()

/// <summary>
/// 영웅 상태 인터페이스를 가져옵니다.
```

```

/// <summary>
/// 영웅 상태 인터페이스를 가져옵니다.
/// </summary>
/// <returns>IHeroStatus 인터페이스</returns>
public IHeroStatus GetHeroStatus()

/// <summary>
/// 영웅 기본 스탯을 가져옵니다.
/// </summary>
/// <returns>HeroBaseStat 데이터</returns>
public HeroBaseStat GetHeroBaseStat()

/// <summary>
/// 매치 시작/종료를 알립니다.
/// </summary>
public void StartMatch()
public void StopMatch(bool isWin)

/// <summary>
/// 위치와 회전을 설정합니다.
/// </summary>
/// <param name="position">설정할 위치</param>
/// <param name="rotation">설정할 회전</param>
public void SetPositionAndRotation(Vector3 position, Quaternion rotation)

/// <summary>
/// 다른 플레이어를 공격했을 때 호출됩니다.
/// </summary>
/// <param name="hitInfo">히트 정보</param>
public void OnStrikeAnotherPlayer(IHitEvent.HitInfo hitInfo)

/// <summary>
/// 공격 동작을 모든 클라이언트에 동기화하는 RPC
/// </summary>
/// <param name="hitInfo">히트 정보</param>
[Rpc(RpcSources.InputAuthority, RpcTargets.All)]
private void RPC_OnStrikeAnotherPlayer(IHitEvent.HitInfo hitInfo)

/// <summary>
/// 렌더링 프레임마다 호출되어 네트워크 상태 변경을 처리합니다.
/// </summary>
public override void Render()

```

이벤트 리스너 메서드

```

/// <summary>
/// 영웅 이벤트 리스너를 등록/해제합니다.
/// </summary>
/// <param name="heroEvent">등록할 영웅 이벤트</param>
public void Register(IHeroEvent heroEvent)
public void UnRegister(IHeroEvent heroEvent)

/// <summary>
/// 스킬 이벤트 리스너를 등록/해제합니다.
/// </summary>

```

```

/// <param name="skillEvent">등록할 스킬 이벤트</param>
public void Register(ISkillEvent skillEvent)
public void UnRegister(ISkillEvent skillEvent)

/// <summary>
/// 공격 이벤트 리스너를 등록/해제합니다.
/// </summary>
/// <param name="attackEvent">등록할 공격 이벤트</param>
public void Register(IAttackEvent attackEvent)
public void UnRegister(IAttackEvent attackEvent)

/// <summary>
/// 매치 이벤트 리스너를 등록/해제합니다.
/// </summary>
/// <param name="matchEvent">등록할 매치 이벤트</param>
private void Register(IMatchEvent matchEvent)
private void UnRegister(IMatchEvent matchEvent)

/// <summary>
/// 컷씬 이벤트 리스너를 등록/해제합니다.
/// </summary>
/// <param name="cutSceneEvent">등록할 컷씬 이벤트</param>
public void Register(ICutSceneEvent cutSceneEvent)
public void UnRegister(ICutSceneEvent cutSceneEvent)

```

코드 스니펫

능력 업데이트

```

[Networked]
public BaseHeroAbility.BaseAbilityType BaseAbilityType { get; private set; }

private ChangeDetector _changes;

public override void Render()
{
    foreach (var change in _changes.DetectChanges(this))
    {
        switch (change)
        {
            case nameof(BaseAbilityType):
                OnAbilityChanged();
                break;
            case nameof(UserProfileRef):
                OnUserProfileChanged();
                break;
            ...
        }
    }
}

private void OnAbilityChanged()
{

```

```

        UpdateAbility(BaseAbilityType);
    }

    private void UpdateAbility(BaseHeroAbility.BaseAbilityType abilityType)
    {
        // 기존 능력 해제
        if(_baseHeroAbility != null) ReleaseAbility();

        // 새로운 능력 생성
        _baseHeroAbility = heroAbilityManager.CreateAbility(abilityType, modelTransform);
        if (_baseHeroAbility == null)
        {
            Debug.LogError($"[NetworkHeroObject] Can't create ability : {abilityType}");
            return;
        }

        // 모델 업데이트
        heroModelController.UpdateModel(_baseHeroAbility, (Team)UserProfileRef.Team,
            gameObject.transform, modelTransform);

        // 애니메이터 설정
        _baseHeroAnimator = _baseHeroAbility.GetBaseHeroAnimator();

        // 컨트롤러 설정
        heroController.Setup(_baseHeroAbility, networkHeroStatus, HeroObjectInfo);

        // 애니메이션 컨트롤러 설정
        heroAnimationController.Setup(_baseHeroAnimator, networkHeroStatus);
        _baseHeroAnimator.Setup(heroModelController.GetModelAnimator(), _baseHeroAbility);

        // 능력 설정
        _baseHeroAbility.Setup(networkHeroStatus, heroController.HeroObjectController,
            this, heroController.SkillEventNotifier);
        networkHeroStatus.Setup(_baseHeroAbility.GetHeroStat(), 0);
        _baseHeroAbility.SetupHitBoxHandle(networkHeroStatus.OnHitHandle, networkHeroStatus);

        // 이펙트 설정
        _heroEffect = heroModelController.GetHeroRenderer();
        Register((IMatchEvent)_baseHeroAbility);

        // 이벤트 리스너들에게 능력 변경 알림
        foreach (var heroEvent in _heroEvents)
        {
            heroEvent.OnAbilityChanged(abilityType);
            heroEvent.OnModeChanged(IHeroEvent.HeroMode.Standard);
        }

        // 이펙트 컨트롤러 설정 코루틴 시작
        if (_setupEffectController != null) StopCoroutine(_setupEffectController);
        _setupEffectController = StartCoroutine(CoSetupEffectController());
    }
}

```

라운드 시작시 초기화 동작

```

public void ResetHero(int round = 0)
{
    if (Object.HasInputAuthority == false) return;
    RPC_ResetHero(round);
}

[Rpc(RpcSources.InputAuthority, RpcTargets.All)]
private void RPC_ResetHero(int round)
{
    // 상태 초기화
    networkHeroStatus.Setup(_baseHeroAbility.GetHeroStat(), round);
    heroControllerStateHandler.ResetState();
    heroController.ResetFocusMode();
    heroAnimationController.StartMatch();

    // 첫 라운드인 경우 특수 게이지 초기화
    if (IsFirstRound(round)) networkHeroStatus.ChangeSpecialPoint(0);
}

```

기능 설명

수명주기 관리

1 스폰 (OnSpawned)

- 이벤트 리스너 등록
- [ChangeDetector](#) 초기화
- 플레이어 프로필 및 영웅 능력 설정 대기
- 셋업 코루틴 시작

2 디스폰 (OnDespawned)

- 이벤트 리스너 해제
- 리소스 정리
- 디스폰 알림
- 셋업 코루틴 중지

게임 라운드 시작/종료시 초기화

- [NetworkHeroStatus](#) 항목을 [HeroBaseStat](#) 을 통해 초기화
 - 라운드 조건에 따라 SP초기화
- [HeroControllerStateHandler](#) 컨트롤러 상태 초기화
- Focus모드 초기화
- [HeroAnimationController](#) 애니메이션 컨트롤러 초기화

네트워크 동기화 속성

속성	타입	설명
BaseAbilityType	BaseAbilityType	영웅 능력 타입

이벤트 등록 시스템

- **IHeroEvent**: 영웅 관련 이벤트
 - 영웅의 포커스 상태, 능력 변경 사항 브로드 캐스팅
- **ISkillEvent**: 스킬 관련 이벤트
 - 스킬 사용/종료 상태 브로드 캐스팅
- **IAttackEvent**: 공격 관련 이벤트
 - 공격 성공 정보 브로드 캐스팅
- **IMatchEvent**: 매치 관련 이벤트
 - 매치 시작/종료 상태 및 승패정보 브로드 캐스팅
- **ICutSceneEvent**: 컷씬 관련 이벤트
 - 컷씬 시작/종료 상태 브로드 캐스팅

컴포넌트 초기화 흐름

- 능력 변경 시 **UpdateAbility()**에서 다음 순서로 컴포넌트 설정:
 - 1 기존 능력 해제 - **ReleaseAbility()**로 이벤트 해제 및 오브젝트 파괴
 - 2 Ability 생성 - **HeroAbilityManager**에서 새 능력 인스턴스 생성
 - 3 모델 업데이트 - **HeroModelController**에 모델/텍처 적용
 - 4 애니메이터 설정 - **BaseHeroAnimator** 연결 및 초기화
 - 5 컨트롤러 설정 - **HeroController**에 능력과 상태 주입
 - 6 이펙트 설정 - 사운드/이펙트 컨트롤러 연결
 - 7 이벤트 등록 - **IMatchEvent** 등록 및 능력 변경 알림 브로드캐스트

ChangeDetector를 통한 네트워크 동기화

- **Render()**에서 변경된 네트워크 속성을 감지하여 처리:
- **BaseAbilityType** 변경 → **OnAbilityChanged()** → **UpdateAbility()** 호출
- **UserProfileRef** 변경 → **OnUserProfileChanged()** → 프로필 설정 완료 처리

의존성/상속 관계

- **NetworkBehaviour** 를 상속받음
- **IHeroObjectInfo** 인터페이스 구현
- **IAttackEvent** 인터페이스 구현
- **INetworkChat** 인터페이스 구현
- **NetworkHeroStatus**
 - 셋업 및 이벤트 리스닝 제어
 - 주입 및 참조 관리
- **HeroController**

이벤트 및 이벤트 리스닝 제어

- 셋업 및 이벤트 리스닝 제어
 - 주입 및 참조 관리
 - 외부 호출에 따라 활성화 제어
- HeroModelController**
 - 능력 변경시 모델 업데이트 및 셋업
 - 주입 및 참조 관리
- HeroAnimationController**
 - 셋업 및 이벤트 리스닝 제어
- BaseHeroAbility**
 - BaseAbilityType** 네트워크 동기화
 - 능력 변경시 초기화
 - 셋업 및 이벤트 리스닝 제어
 - 주입 및 참조 관리
- BaseHeroAnimator** 셋업 및 참조 관리, 공격 성공시 호출
- ISoundController** 주입 및 참조 관리
- IEffectController** 주입 및 참조관리
- IHeroEffect** 주입 및 참조 관리

사용 예시

관련 클래스

- [NetworkBehaviour](#) 
- [NetworkHeroStatus](#)
- [NetworkHeroController](#)
- [NetworkHeroInput](#)
- [HeroController](#)
- HeroModelController**
- BaseHeroAbility**
- HeroAnimationController**
- BaseHeroAnimator**
- IHeroEffect**
- IEffectController**
- ISoundController**
- IAttackEvent**

NetworkHeroController

네트워크로부터 입력을 수신하여 실제 게임 로직을 실행하는 입력 실행기 클래스

개요

`NetworkHeroController` 클래스는 Photon Fusion의 `NetworkBehaviour` [🔗](#)를 래핑한 `NetworkBehaviourCallback` 클래스를 상속받아 네트워크로부터 입력을 수신하여 실제 게임 로직을 실행하는 입력 실행기입니다. `NetworkHeroInput` 으로부터 전송된 입력 데이터를 수신하고, RPC를 통해 모든 클라이언트에 동기화한 후 `HeroController` 를 호출하여 실제 게임 동작을 실행합니다.

역할

- `NetworkBehaviour` [🔗](#)를 래핑한 `NetworkBehaviourCallback` 클래스를 통해 네트워크 입력 수신
- `RPC(Remote Procedure Call)` [🔗](#)를 통한 입력 데이터 브로드캐스트
- 수신된 입력을 조건 처리 후 각종 액션(이동, 공격, 가드 등)으로 변환
- 네트워크 지연(Latency) 측정을 위한 타임스탬프 동기화
- `HeroController` 의 메서드를 호출하여 실제 게임 동작 실행

HeroInputData 수신 및 브로드캐스팅





선언

```
public class NetworkHeroController : NetworkBehaviourCallback
```

멤버

필드

```
/// <summary>
/// 실제 영웅 동작을 제어하는 로컬 게임 로직 컨트롤러
/// </summary>
[SerializeField]
protected HeroController heroController;

/// <summary>
/// FixedUpdateNetwork에서 처리된 마지막 입력 데이터
/// </summary>
protected HeroInputData FixedInputData;
```

중첩 구조체

```
/// <summary>
/// 네트워크 지연을 계산하기 위한 입력 동기화 데이터 구조체
/// 클라이언트 타임스탬프와 서버 시뮬레이션 타임을 저장하여 지연 시간을 측정합니다.
/// </summary>
public readonly struct HeroInputSyncData
{
    /// <summary>
    /// 클라이언트에서 입력이 발생한 시간
    /// </summary>
    public readonly float ClientTimeStamp;

    /// <summary>
    /// 서버에서 입력을 처리한 시뮬레이션 시간
    /// </summary>
    public readonly float SimulationTimeStamp;

    /// <summary>
    /// HeroInputSyncData 생성자
    /// </summary>
    /// <param name="clientTimeStamp">클라이언트 타임스탬프</param>
    /// <param name="simulationTimeStamp">서버 시뮬레이션 타임스탬프</param>
    public HeroInputSyncData(float clientTimeStamp, float simulationTimeStamp)
```

```

    /// <param name="simulationTimeStamp">서버 시뮬레이션 타임</param>
    public HeroInputSyncData(float clientTimeStamp, float simulationTimeStamp)

    /// <summary>
    /// 네트워크 지연 시간을 계산합니다.
    /// SimulationTimeStamp와 ClientTimeStamp의 차이를 반환합니다.
    /// </summary>
    /// <returns>지연 시간(초)</returns>
    public float Latency() => => SimulationTimeStamp - ClientTimeStamp;
}

```

메서드

```

    /// <summary>
    /// HeroInputData를 모든 클라이언트에 동기화하는 RPC 메서드
    /// 입력 권한을 가진 클라이언트에서 호출되어 모든 클라이언트에서 실행됩니다.
    /// </summary>
    /// <param name="heroInputData">동기화할 입력 데이터</param>
    [Rpc(RpcSources.InputAuthority, RpcTargets.All)]
    private void RPC_UpdateHeroInputData(HeroInputData heroInputData)

    /// <summary>
    /// 공격 입력을 처리합니다.
    /// 공격 버튼이 눌렸을 때 HeroController의 Attack 메서드를 호출합니다.
    /// </summary>
    /// <param name="inputData">입력 데이터</param>
    private void ProcessAttack(HeroInputData inputData)

    /// <summary>
    /// 공격 홀드(차지) 입력을 처리합니다.
    /// 공격 버튼을 누르고 있을 때/뮬 때 HeroController의 메서드를 호출합니다.
    /// </summary>
    /// <param name="inputData">입력 데이터</param>
    private void ProcessAttackHold(HeroInputData inputData)

    /// <summary>
    /// 대시 입력을 처리합니다.
    /// 대시 버튼이 눌렸을 때 HeroController의 Dash 메서드를 호출합니다.
    /// </summary>
    /// <param name="inputData">입력 데이터</param>
    private void ProcessDash(HeroInputData inputData)

    /// <summary>
    /// 가드 입력을 처리합니다.
    /// 가드 버튼이 눌러있을 때/떼었을 때 HeroController의 메서드를 호출합니다.
    /// </summary>
    /// <param name="inputData">입력 데이터</param>
    private void ProcessGuard(HeroInputData inputData)

    /// <summary>
    /// 포커스 모드 입력을 처리합니다.
    /// 포커스 모드 버튼이 눌렸을 때 현재 상태를 토글합니다.
    /// </summary>
    /// <param name="heroInputData">입력 데이터</param>

```

```

/// <param name="heroInputData">입력 데이터</param>
private void ProcessFocus(HeroInputData heroInputData)

/// <summary>
/// 궁극기 입력을 처리합니다.
/// 궁극기 버튼이 눌렸을 때 HeroController의 UltimateSkill 메서드를 호출합니다.
/// </summary>
/// <param name="inputData">입력 데이터</param>
private void ProcessUltimateSkill(HeroInputData inputData)

/// <summary>
/// 공격+가드 동시 입력을 처리합니다.
/// 특수 스킬 발동 입력이 감지되면 HeroController의 메서드를 호출합니다.
/// </summary>
/// <param name="inputData">입력 데이터</param>
private void ProcessAttackGuardDualInput(HeroInputData inputData)

/// <summary>
/// 네트워크 Fixed Update에서 입력을 수신하고 처리합니다.
/// Fusion의 시뮬레이션 루프에서 매 프레임 호출됩니다.
/// </summary>
public override void FixedUpdateNetwork()

```

코드 스니펫

네트워크 입력 수신 및 RPC 호출

```

public override void FixedUpdateNetwork()
{
    // 1. 네트워크에서 HeroInputData 수신
    if (GetInput<HeroInputData>(out var heroInputData))
    {
        // 2. 컨트롤러 활성화 상태 검증
        if (heroController.IsActive == false)
        {
            RPC_UpdateHeroInputData(default);
            return;
        }

        // 3. RPC를 통해 모든 클라이언트에 입력 동기화
        RPC_UpdateHeroInputData(heroInputData);
        FixedInputData = heroInputData;
    }
}

[Rpc(RpcSources.InputAuthority, RpcTargets.All)]
private void RPC_UpdateHeroInputData(HeroInputData heroInputData)
{
    // 이동 입력 처리
    heroController.Move(heroInputData.Horizontal, heroInputData.Vertical, Runn

    // 개별 스킬/액션 입력 처리
    ProcessDash(heroInputData);
}

```

```

        ProcessAttack(heroInputData);
        ProcessAttackHold(heroInputData);
        ProcessFocus(heroInputData);
        ProcessGuard(heroInputData);
        ProcessUltimateSkill(heroInputData);
        ProcessAttackGuardDualInput(heroInputData);
    }

```

입력 데이터 처리

```

private void ProcessAttack(HeroInputData inputData)
{
    if (inputData.IsAttack())
    {
        // 동기화 데이터 생성 (지연 측정용)
        var syncData = new HeroInputSyncData(
            inputData.AttackTimeStamp,
            Runner.SimulationTime
        );

        // 실제 게임 로직 실행
        heroController.Attack(syncData);
    }
}

```

기능 설명

네트워크 입력 동기화 시스템

- 1 **입력 수신:** `FixedUpdateNetwork()`에서 Fusion의 `GetInput<T>()`를 통해 로컬 입력을 수신
- 2 **입력 검증:** 컨트롤러가 활성화된 상태인지 확인
- 3 **RPC 브로드캐스트:** `RPC_UpdateHeroInputData()`를 통해 모든 클라이언트에 브로드캐스트
- 4 **동일한 처리:** 모든 클라이언트에서 동일한 입력으로 `HeroController` 메서드 호출

입력 처리 종류

메서드	입력	실행 동작
ProcessAttack	Attack	일반 공격
ProcessAttackHold	AttackHold	공격 홀드(홀드/홀드종료)
ProcessDash	Dash	대시/회피
ProcessGuard	Guard	가드(홀드/홀드종료)
ProcessFocus	FocusMode	포커스 모드 토글
ProcessUltimateSkill	UltimateAttack	궁극기 스킬

네트워크 지연 측정

- `HeroInputSyncData` 구조체 사용
- 클라이언트에서 입력 발생 시각(`ClientTimeStamp`) 기록
- 서버에서 입력 처리 시각(`SimulationTimeStamp`) 기록
- `Latency()` 메서드로 네트워크 왕복 시간 계산

의존성/상속 관계

- `NetworkBehaviour` 상속
- 동기화된 입력을 `HeroController` 클래스를 통해 실제 영웅 동작 제어
- `HeroInputData` 구조체를 통해 입력 데이터 처리
- `HeroInputSyncData` 중첩 구조체 정의

사용 예시

캐릭터별 SkillManager 클래스(`BaseSkillManager`)에서 `HeroInputSyncData` 구조체를 통해 동작 입력시간 보장

```
protected override void OnStartAttackHold(NetworkHeroController.HeroInputSyncData syncData)
{
    // AttackHold조건 판별
    if (!_isAttackHold) return;
    if (!_isGuard) return;
    if (HeroStatus.CurrentStance != IHeroStatus.Stance.StandUp) return;

    // 동작중인 공격 캔슬
    if (_currentSkill?.skillType == BaseHeroSkill.SkillType.Attack) _currentSkill.Cancel();

    // AttackHold 플래그 처리 및 지연시간 적용
    _isAttackHold = true;
    _attackHoldDuration = syncData.Latency();

    _skillQueue.Clear();
    if (_skillCoroutine != null)
    {
        if (_skillQueue.Count < 2)
        {
            _skillQueue.Enqueue(new SkillStruct(chargingSkill,
                new BaseHeroSkill.SkillExecutionData(),
                OnAttackHoldStart, OnSkillComplete, OnAttackSucceed));
        }
    }
    else
    {
        // 스킬 실행 데이터에 지연시간 데이터 포함
        skillCoroutine = StartCoroutine(DoSkill(chargingSkill, _attackHoldDuration));
    }
}
```

```
        _OnAttackHoldStart = OnAttackHoldStart;
        _OnSkillComplete = OnSkillComplete;
        _OnAttackSucceed = OnAttackSucceed;

        new BaseHeroSkill.SkillExecutionData(syncData.Latency(),
            OnAttackHoldStart, OnSkillComplete, OnAttackSucceed));
    }
}
```

관련 클래스

- [NetworkBehaviour](#) 
- [NetworkHeroObject](#)
- [HeroController](#)
- [NetworkHeroStatus](#)
- [HeroInputData](#)
- [IHeroController](#)
- [NetworkHeroInput](#)

IHeroStatus

영웅의 상태를 관리하는 인터페이스

개요

IHeroStatus 인터페이스는 RFist 게임에서 영웅 캐릭터의 모든 상태를 관리하는 핵심 인터페이스입니다. 생명력(HP), 특수 포인트(SP), 경직 포인트, 자세(Stance), 상태효과 등의 상태 정보를 제공하고, 상태 변경 시 발생하는 이벤트를 정의합니다.

역할

- 영웅의 생명력(HP) 관리 메서드 및 데미지 메서드 정의
- 특수 포인트(SP) 관리 메서드 정의
- 경직 포인트 관리 메서드 정의
- 영웅의 자세(StandUp, Down, Stun) 상태 관리 메서드 정의
- 공격/방어/무적 상태 관리 메서드 정의
- 상태효과(일시적 능력치 변경) 관리 메서드 정의
- 각각의 상태 변경 시 이벤트 정의

선언

```
public interface IHeroStatus
```

멤버

중첩 구조체 및 중첩 열거형

```
/// <summary>
/// 생명력 정보를 담는 구조체
/// </summary>
public struct HealthPoint
{
    public HealthPoint(int current, int max);
    public int Current { get; private set; }
    public int Max { get; private set; }
}

/// <summary>
```

```

/// 특수 포인트(SP) 정보를 담는 구조체
/// </summary>
public struct SpecialPoint
{
    public SpecialPoint(int current, int max);
    public int Current { get; private set; }
    public int Max { get; private set; }
}

/// <summary>
/// 경직 포인트 정보를 담는 구조체
/// </summary>
public struct StiffnessPoint
{
    public StiffnessPoint(int current, int max);
    public int Current { get; private set; }
    public int Max { get; private set; }
}

/// <summary>
/// 상태효과 정보를 담는 구조체
/// </summary>
public readonly struct StatusBoost : IEquatable<StatusBoost>
{
    public readonly BoostType boostType;
    public readonly float boostValue;
    public readonly float boostDuration;

    public StatusBoost(BoostType boostType, float boostValue, float boostDuration)
    {
        boostType = boostType;
        boostValue = boostValue;
        boostDuration = boostDuration;
    }

    public bool Equals(StatusBoost other);
}

/// <summary>
/// 영웅의 자세를 정의하는 열거형
/// </summary>
public enum Stance
{
    StandUp,
    Down,
    Stun
}

/// <summary>
/// 상태효과 타입을 정의하는 열거형
/// </summary>
public enum BoostType
{
    None = 0,
    GrowPower,
    GrowHealthPoint,
    GrowSpeed
}

```

델리게이트 및 이벤트


```

/// <summary>
/// 생명력 변경 시 호출되는 델리게이트 및 이벤트
/// </summary>
/// <param name="healthPoint">변경된 생명력 정보</param>
public delegate void OnHpChange(HealthPoint healthPoint);
public event OnHpChange OnHpChanged;

/// <summary>
/// 적 혹은 플레이어 사망시 호출되는 델리게이트 및 이벤트
/// </summary>
public delegate void OnDeathDelegate();
public event OnDeathDelegate OnDeath;
public event OnDeathDelegate OnKill;

/// <summary>
/// 피해 입었을 때 호출되는 델리게이트 및 이벤트
/// </summary>
/// <param name="hitInfo">피해 정보</param>
public delegate void OnDamageDelegate(IHitEvent.HitInfo hitInfo);
public event OnDamageDelegate OnDamaged;

/// <summary>
/// 상태효과 변경 시 호출되는 델리게이트 및 이벤트
/// </summary>
/// <param name="type">변경된 부스트 타입</param>
/// <param name="active">활성화 여부</param>
public delegate void OnStatusBoostChange(StatusBoost.BoostType type, bool active);
public event OnStatusBoostChange OnStatusBoostChanged;

/// <summary>
/// SP 변경 시 호출되는 델리게이트 및 이벤트
/// </summary>
/// <param name="specialPoint">변경된 SP 정보</param>
public delegate void OnSpChange(SpecialPoint specialPoint);
public event OnSpChange OnSpChanged;

/// <summary>
/// 경직 포인트 변경 시 호출되는 델리게이트 및 이벤트
/// </summary>
/// <param name="stiffnessPoint">변경된 경직 포인트 정보</param>
public delegate void OnStiffnessPointChange(StiffnessPoint stiffnessPoint);
public event OnStiffnessPointChange OnStiffnessPointChanged;

```

속성

```

/// <summary>
/// 현재 자세를 반환합니다.
/// </summary>
public Stance CurrentStance { get; }

```

```

/// <summary>
/// 공격 상태 여부를 반환합니다.

```

```

/// 공격 상태 여부를 반환합니다.
/// </summary>
public bool AttackState { get; }

/// <summary>
/// 방어 상태 여부를 반환합니다.
/// </summary>
public bool GuardState { get; }

/// <summary>
/// 무적 상태 여부를 반환합니다.
/// </summary>
public bool InvincibleState { get; }

```

메서드

```

/// <summary>
/// 데미지를 적용합니다.
/// </summary>
/// <param name="damage">적용할 데미지량</param>
public void ApplyDamage(int damage);

/// <summary>
/// 현재 생명력 정보를 반환합니다.
/// </summary>
/// <returns>현재 생명력 정보</returns>
public HealthPoint GetHp();

/// <summary>
/// 생명력을 설정합니다.
/// </summary>
/// <param name="hp">설정할 생명력 값</param>
/// <param name="isSync">네트워크 동기화 여부</param>
public void SetHealthPoint(int hp, bool isSync = false);

/// <summary>
/// 자세를 변경합니다.
/// </summary>
/// <param name="stance">변경할 자세</param>
public void ChangeStance(Stance stance);

/// <summary>
/// 상태효과를 적용합니다.
/// </summary>
/// <param name="statusBoost">적용할 상태효과</param>
public void ApplyStatusBoost(StatusBoost statusBoost);

/// <summary>
/// 현재 적용된 모든 상태효과를 반환합니다.
/// </summary>
/// <returns>상태효과 집합</returns>
public HashSet<StatusBoost> GetStatusBoosts();

/// <summary>

```

```

/// <summary>
/// 모든 상태효과를 초기화합니다.
/// </summary>
public void ClearStatusBoosts();

/// <summary>
/// 현재 SP 정보를 반환합니다.
/// </summary>
/// <returns>현재 SP 정보</returns>
public SpecialPoint GetSp();

/// <summary>
/// SP를 변경합니다.
/// </summary>
/// <param name="point">변경할 SP량</param>
public void ChangeSpecialPoint(int point);

/// <summary>
/// 현재 경직 포인트 정보를 반환합니다.
/// </summary>
/// <returns>현재 경직 포인트 정보</returns>
public StiffnessPoint GetStiffnessPoint();

/// <summary>
/// 경직 포인트를 설정합니다.
/// </summary>
/// <param name="point">설정할 경직 포인트 값</param>
/// <param name="isSync">네트워크 동기화 여부</param>
public void SetStiffnessPoint(int point, bool isSync = false);

/// <summary>
/// 공격 상태를 설정합니다.
/// </summary>
/// <param name="state">설정할 상태</param>
public void SetAttackState(bool state);

/// <summary>
/// 방어 상태를 설정합니다.
/// </summary>
/// <param name="state">설정할 상태</param>
public void SetGuardState(bool state);

/// <summary>
/// 무적 상태를 설정합니다.
/// </summary>
/// <param name="state">설정할 상태</param>
public void SetInvincibleState(bool state);

/// <summary>
/// 피격 정보를 적용합니다
/// </summary>
/// <param name="hitResult">히트 결과</param>
public void AttachHitResult(HitBoxHandler.HitResult hitResult);

```

기능 설명

상태 관리 시스템

- **생명력(HP) 관리**: 현재/최대 생명력 추적, 데미지 적용, 사망 처리
- **SP 관리**: 필살기 사용에 필요한 포인트 관리
- **경직 포인트**: 공격을 받을 때 쌓이는 경직 게이지 관리
- **자세(Stance)**: StandUp(서있음), Down(다운), Stun(스턴) 3가지 상태 관리
- **전투 상태**: 공격 중, 방어 중, 무적 상태 관리
- **상태효과**: 일시적인 능력치 증가 효과 관리

이벤트 시스템

- **OnHpChanged**: 생명력 변화 시 UI 업데이트 등에 사용
- **OnDamaged**: 피해 효과, 히트 스톱 등에 사용
- **OnDeath/OnAfterDeath**: 사망 처리, 게임 오버 등에 사용
- **OnStatusBoostChanged**: 상태 효과 발생/제거시 효과 적용 및 UI 표시에 사용

의존성/상속 관계

- 구현 클래스
 - **NetworkHeroStatus** 클래스에서 네트워크 환경에서의 상태 관리 구현
- **HitBoxHandler** 클래스의 **IHitEvent**를 통해 피해 이벤트 정보 전달
- **HealthPoint**, **SpecialPoint**, **StiffnessPoint**, **StatusBoost** 중첩 구조체 및 **Stance**, **BoostType** 중첩 열거형 정의

사용 예시

HeroCanvasView(HUD)에서 영웅의 체력 변경 시 HUD 정보를 수정

```
private NetworkHeroObject heroObject;

private void OnEnable()
{
    heroObject.GetHeroStatus().OnHpChanged += OnHpChanged;
    ...
}

private void OnHpChanged(IHeroStatus.HealthPoint healthPoint)
{
    if (teamAHPGaugeImage.gameObject.activeSelf)
    {
        teamAHPGaugeImage.fillAmount = (float)healthPoint.Current / healthPoint.Max;
    }
    else
    {
        teamBHPGaugeImage.fillAmount = (float)healthPoint.Current / healthPoint.Max;
    }
}
```

`BaseHeroAttackSkill`에서 영웅의 공격이 적 히트시 관련 상태 정보를 수정 및 피격결과 전달

```
// BaseHeroSkill에서 정의됨
protected IHeroStatus HeroStatus;

private void OnHitBoxDetected(Collider col, Transform targetTransform, HitBoxH
{
    ...
    // 적이 히트 가능한 상태 체크
    if (!hitHandler.CanHit(hitInfo)) return;

    OnHitInvoke?.Invoke(hitInfo);

    // 적 히트시 SP를 획득
    var spPoint = HeroStatus.GetSp().Current;
    var spGain = damageResult * SpecialPointGainFactor;
    HeroStatus.ChangeSpecialPoint((int)(spPoint + spGain));

    // hitHandler를 통해 HitResult를 반환받고 IHeroStatus로 전달
    var hitResult = hitHandler.OnHit(hitInfo);
    HeroStatus.AttachHitResult(hitResult);
    ...
}
```

`RektorSkillManager`(영웅별 스킬매니저)에서 공격이 시작 될 때 공격 상태를 변환

```
private void OnAttackStart(BaseHeroSkill.SkillStartResult skillStartResult)
{
    // 영웅의 방향키가 눌러있는 경우 해당 방향으로 공격
    if (MoveInputDir != Vector2.zero) HeroController.LookAt(new Vector3(MoveIn
    // 타겟팅 보정
    HeroController.FindAndLookTarget(_currentSkill.AutoTargetingDistance, _cur
    // 스킬 발동 이벤트 호출
    SkillEventInvoker.NotifySkillEvents(skillEvent => skillEvent?.OnSkill(skill
    // 공격상태로 전환
    HeroStatus.SetAttackState(true);
}
```

관련 클래스

- `NetworkHeroStatus`
- `IHitEvent`
- `HitBoxHandler`

NetworkHeroInput

로컬 플레이어 입력을 수집하여 네트워크로 전송하는 입력 수집기 클래스

개요

`NetworkHeroInput` 클래스는 Photon Fusion의 `NetworkBehaviour` [🔗](#)를 랩핑한 `NetworkBehaviourCallback` 클래스를 상속받아, 로컬 플레이어가 누른 키/버튼 입력을 `HeroInputData` 구조체로 변환하고 네트워크로 전송합니다. 입력 권한(`Input Authority`)을 가진 클라이언트에서만 동작하며, 실제 게임 로직은 실행하지 않습니다.

역할

- Fusion의 `NetworkInput` [🔗](#) 시스템을 통해 네트워크로 입력 데이터 전송
- `IHeroInput`을 구현한 입력소스로 부터 입력 수집
- 수집된 입력을 `HeroInputData` 구조체로 변환
- 이동(수평/수직), 공격, 가드, 대시, 포커스 등 다양한 입력 처리
- 입력 타임스탬프 기록으로 네트워크 지연 측정 지원
- 입력 권한(`Input Authority` [🔗](#)) 검증

선언

```
public class NetworkHeroInput: NetworkBehaviourCallback
```

멤버

메서드

```
/// <summary>
/// 네트워크 입력이 필요할 때 Fusion에 의해 호출되는 콜백
/// 로컬 플레이어의 입력을 수집하여 NetworkInput에 설정합니다.
/// </summary>
/// <param name="runner">NetworkRunner 인스턴스</param>
/// <param name="input">설정할 NetworkInput</param>
public override void OnInput(NetworkRunner runner, NetworkInput input)
```

코드 스니펫

입력 수집 및 네트워크 전송

```
public override void OnInput(NetworkRunner runner, NetworkInput input)
{
    // 입력 권한이 없으면 처리하지 않음
    if (!Object.HasInputAuthority) return;

    // 새로운 입력 데이터 생성
    var heroInputData = new HeroInputData();

    // 등록된 모든 입력 소스에서 입력 수집
    foreach (var heroInput in BaseHeroInput.HeroInputList)
    {
        // 수평 및 수직 이동 입력 수집
        heroInputData.Horizontal = heroInput.GetHorizontal();
        heroInputData.Vertical = heroInput.GetVertical();

        // 타임스탬프 기록
        heroInputData.AttackTimeStamp = runner.SimulationTime;

        // 대시 입력 수집
        if (heroInput.IsDash())
        {
            heroInputData.Buttons |= HeroInputData.Dash;
        }

        // 공격 입력 수집
        if (heroInput.IsAttack())
        {
            heroInputData.Buttons |= HeroInputData.Attack;
        }

        // 공격 홀드(차지) 입력 수집
        if (heroInput.IsAttackHold())
        {
            heroInputData.Buttons |= HeroInputData.AttackHold;
        }

        // 가드 입력 수집
        if (heroInput.IsGuard())
        {
            heroInputData.Buttons |= HeroInputData.Guard;
        }

        // 포커스 모드 입력 수집
        if (heroInput.IsFocusMode())
        {
            heroInputData.Buttons |= HeroInputData.FocusMode;
        }

        // 포커스 변경 입력 수집
        if (heroInput.IsFocusChange())
        {
            heroInputData.Buttons |= HeroInputData.FocusChange;
        }
    }

    runner.SendNetworkVariable(heroInputData);
}
```

```

    {
        heroInputData.Buttons |= HeroInputData.FocusChange;
    }

    // 궁극기 입력 수집
    if (heroInput.IsUltimateAttack())
    {
        heroInputData.Buttons |= HeroInputData.UltimateAttack;
    }

    // 공격+가드 동시 입력 수집
    if (heroInput.IsAttackGuardDualInput())
    {
        heroInputData.Buttons |= HeroInputData.AttackGuardDualInput;
    }
}

// 수집된 입력 데이터를 네트워크 입력으로 설정
input.Set(heroInputData);
}

```

기능 설명

입력 수집 흐름

- 1 권한 검증: [HasInputAuthority](#) 를 확인하여 로컬 플레이어만 입력 수집
- 2 데이터 초기화: 새로운 [HeroInputData](#) 구조체 생성
- 3 입력 소스 순회: [IHeroInput](#)을 구현한 입력소스로 부터 입력 수집
- 4 이동 입력: 수평(Horizontal) 및 수직(Vertical) 이동 입력 수집
- 5 버튼 입력: 비트 플래그를 사용하여 다양한 액션 입력 수집
- 6 타임스탬프 기록: 공격 관련 입력에 시뮬레이션 타임 기록
- 7 네트워크 설정: 수집된 데이터를 [NetworkInput](#) 에 설정

지원하는 입력 타입

입력	상수	설명
이동	Horizontal, Vertical	WASD/스틱 이동 입력
공격	Attack	공격 버튼
공격 홀드	AttackHold	공격 버튼 홀드(차지)
가드	Guard	가드/방어 버튼
대시	Dash	회피/대시 버튼
포커스 모드	FocusMode	타겟 고정 모드 토글
포커스 변경	FocusChange	타겟 변경
궁극기	UltimateAttack	궁극기 스킬 버튼

Component	Component Name	Component Role
공격+가드	AttackGuardDualInput	특수 스킬 발동 입력

타임스탬프 동기화

- ``runner.SimulationTime`` 을 기록
- `NetworkHeroController.HeroInputSyncData` 를 통해 네트워크 지연 계산에 활용
- 클라이언트-서버 간 시간 동기화에 사용

의존성/상속 관계

- `NetworkBehaviourCallback` 랩핑 클래스를 상속함
- `BaseHeroInput` 클래스를 통해 입력 소스 수집
- `HeroInputData` 구조체가 `NetworkInput` 구현 및 네트워크 동기화됨

관련 클래스

- `NetworkRunner`
- `NetworkInput`
- `NetworkHeroController`
- `BaseHeroInput`
- `HeroInputData`
- `IHeroInput`
- `HeroKeyInput`

NetworkHeroStatus

Photon Fusion 네트워크 환경에서 **IHeroStatus**를 구현한 영웅의 상태를 관리하는 클래스

개요

NetworkHeroStatus 클래스는 Photon Fusion 네트워크 환경에서 영웅의 상태(HP, SP, 경직 포인트 등)를 관리하는 클래스입니다. **IHeroStatus** 인터페이스를 구현하여 다양한 상태 변화 이벤트를 제공하며, 모든 상태 변경은 네트워크를 통해 동기화됩니다. HP(생명력), SP(특수 포인트), 경직 포인트(Stiffness), 자세(Stance) 등의 상태를 관리하고, 상태 효과(Status Boost) 시스템을 통해 일시적인 능력치 변화도 처리합니다.

역할

- 영웅의 HP(생명력) 관리 및 네트워크 동기화
- SP(특수 포인트) 관리
- 경직 포인트(Stiffness Point) 관리
- 자세(Stance) 상태 관리
- 공격/가드/무적 상태 관리
- 포커스 모드 상태 관리
- 상태 효과(Status Boost) 시스템 관리
- 히트 처리 및 데미지 계산
- 사망/처치 이벤트 발생

네트워크 캐릭터 상태 동기화(HP,경직포인트 및 자세)





선언

```
[RequireComponent(typeof(NetworkObject))]  
public class NetworkHeroStatus : NetworkBehaviourCallback, IHeroStatus
```

멤버

이벤트

```
/// <summary>  
/// HP 변경 시 발생하는 이벤트  
/// </summary>  
public event IHeroStatus.OnHpChange OnHpChanged;  
  
/// <summary>  
/// 적 혹은 플레이어 사망시 발생하는 이벤트  
/// </summary>  
public event IHeroStatus.OnDeathDelegate OnKill;  
public event IHeroStatus.OnDeathDelegate OnDeath;  
  
/// <summary>  
/// SP(특수 포인트) 변경 시 발생하는 이벤트  
/// </summary>  
public event IHeroStatus.OnSpChange OnSpChanged;  
  
/// <summary>  
/// 피해를 입었을 때 발생하는 이벤트  
/// </summary>  
public event IHeroStatus.OnDamageDelegate OnDamaged;  
  
/// <summary>  
/// 상태 효과 변경 시 발생하는 이벤트  
/// </summary>  
public event IHeroStatus.OnStatusBoostChange OnStatusBoostChanged;  
  
/// <summary>  
/// 경직 포인트 변경 시 발생하는 이벤트  
/// </summary>  
public event IHeroStatus.OnStiffnessPointChange OnStiffnessPointChanged;
```

필드

```
/// <summary>  
/// 영웅 컨트롤러 참조
```

```

/// </summary>
[SerializeField]
private HeroController heroController;

/// <summary>
/// 현재 처랭(HP)
/// </summary>
private int _healthPoint;

/// <summary>
/// 현재 경직 포인트(Stiffness Point)
/// </summary>
private int _stiffnessPoint;

/// <summary>
/// 현재 특수 포인트(SP)
/// </summary>
private int _specialPoint;

/// <summary>
/// 영웅 기본 스탯
/// </summary>
private HeroBaseStat _heroBaseStat;

/// <summary>
/// 포커스 모드 활성화 여부
/// </summary>
private bool _isFocusMode;

/// <summary>
/// 적용 중인 상태 효과 집합
/// </summary>
private HashSet<IHeroStatus.StatusBoost> _statusBoosts;

/// <summary>
/// 활성화된 상태 효과 코루틴 관리 딕셔너리
/// </summary>
private Dictionary<IHeroStatus.StatusBoost.BoostType, Coroutine> _activeStatus

```

속성

```

/// <summary>
/// 현재 자세(Stance) 상태
/// </summary>
public IHeroStatus.Stance CurrentStance { get; private set; }

/// <summary>
/// 공격 상태 여부
/// </summary>
public bool AttackState { get; private set; }

/// <summary>
/// 가드 상태 여부

```

```

/// </summary>
public bool GuardState { get; private set; }

/// <summary>
/// 무적 상태 여부
/// </summary>
public bool InvincibleState { get; private set; }

/// <summary>
/// 히트 처리 델리게이트 인스턴스
/// </summary>
public OnHitDelegate OnHitHandle => OnHit;

```

메서드

```

/// <summary>
/// 포커스 모드 상태를 설정합니다.
/// 입력 권한이 있는 플레이어만 설정 가능합니다.
/// </summary>
private void SetFocusState(bool isFocusMode)

/// <summary>
/// 포커스 모드 상태를 모든 클라이언트에 동기화하는 RPC
/// </summary>
[Rpc(RpcSources.InputAuthority, RpcTargets.All)]
private void RPC_SetFocusState(bool isFocusMode)

/// <summary>
/// 영웅 상태를 초기 설정합니다.
/// </summary>
/// <param name="defaultBaseHeroStat">기본 영웅 스탯</param>
/// <param name="round">현재 라운드</param>
public void Setup(HeroBaseStat defaultBaseHeroStat, int round)

/// <summary>
/// HP 변경 시 호출됩니다.
/// 이벤트를 발생시키고 사망 체크를 수행합니다.
/// </summary>
private void OnHealthPointChanged()

/// <summary>
/// 경직 포인트 변경 시 호출됩니다.
/// </summary>
private void OnStiffnessChanged()

/// <summary>
/// 히트 처리 시 호출됩니다.
/// </summary>
/// <param name="hitInfo">히트 정보</param>
/// <returns>히트 결과</returns>
private HitBoxHandler.HitResult OnHit(IHitEvent.HitInfo hitInfo)

/// <summary>
/// 히트 처리를 담당합니다.

```

```

/// 자세를 변경합니다.
/// </summary>
/// <param name="stance">변경할 자세</param>
public void ChangeStance(IHeroStatus.Stance stance)

/// <summary>
/// 히트 정보를 모든 클라이언트에 동기화하는 RPC
/// </summary>
[Rpc(RpcSources.All, RpcTargets.All)]
private void RPC_UpdateHitInfo(IHitEvent.HitInfo hitInfo)

/// <summary>
/// 자세를 업데이트합니다.
/// </summary>
private void UpdateStance(IHitEvent.HitInfo.HitterState hitterState)

/// <summary>
/// 경직 포인트를 업데이트합니다.
/// </summary>
private void UpdateStiffness(IHitEvent.HitInfo hitInfo)

/// <summary>
/// 경직 포인트를 모든 클라이언트에 동기화하는 RPC
/// </summary>
[Rpc(RpcSources.All, RpcTargets.All)]
private void RPC_UpdateStiffnessPoint(int stiffnessPoint , bool executeLocal)

/// <summary>
/// 데미지를 적용합니다.
/// </summary>
/// <param name="damage">적용할 데미지</param>
public void ApplyDamage(int damage)

/// <summary>
/// HP를 모든 클라이언트에 동기화하는 RPC
/// </summary>
[Rpc(RpcSources.All, RpcTargets.All)]
private void RPC_UpdateHealthPoint(int healthPoint , bool executeLocal)

/// <summary>
/// HP를 설정합니다.
/// </summary>
public void SetHealthPoint(int hp, bool isSync = false)

/// <summary>
/// 경직도를 설정합니다.
/// </summary>
public void SetStiffnessPoint(int point, bool isSync = false)

/// <summary>
/// 상태 효과를 적용합니다.
/// </summary>
/// <param name="statusBoost">적용할 상태 효과</param>
public void ApplyStatusBoost(IHeroStatus.StatusBoost statusBoost)

/// <summary>

```

```

/// 상태 효과 적용 코루틴
/// 지정된 시간 동안 부스트 효과를 적용하고 자동으로 해제합니다.
/// </summary>
private IEnumerator CoUpdateStatusBoost(IHeroStatus.StatusBoost statusBoost)

/// <summary>
/// 상태 효과를 제거합니다.
/// </summary>
private void RemoveStatusBoost(IHeroStatus.StatusBoost.BoostType statusBoost)

/// <summary>
/// 현재 SP를 가져옵니다.
/// </summary>
public IHeroStatus.SpecialPoint GetSp() => new IHeroStatus.SpecialPoint(_specialPoint)

/// <summary>
/// SP를 변경합니다.
/// </summary>
public void ChangeSpecialPoint(int point)

/// <summary>
/// 적용 중인 상태 효과 목록을 가져옵니다.
/// </summary>
public HashSet<IHeroStatus.StatusBoost> GetStatusBoosts() => _statusBoosts;

/// <summary>
/// 현재 경직도를 가져옵니다.
/// </summary>
public IHeroStatus.StiffnessPoint GetStiffnessPoint() => new IHeroStatus.StiffnessPoint(_stiffnessPoint)

/// <summary>
/// 공격 상태를 설정합니다.
/// </summary>
public void SetAttackState(bool attackState)

/// <summary>
/// 가드 상태를 설정합니다.
/// </summary>
public void SetGuardState(bool state)

/// <summary>
/// 무적 상태를 설정합니다.
/// </summary>
public void SetInvincibleState(bool state)

/// <summary>
/// 히트 결과를 첨부합니다.
/// </summary>
public void AttachHitResult(HitBoxHandler.HitResult hitResult)

/// <summary>
/// 히트 결과를 모든 클라이언트에 동기화하는 RPC
/// </summary>
[Rpc(RpcSources.InputAuthority, RpcTargets.All)]
private void RPC_AttachHitResult(int applyDamage, int healPoint)

```

코드 스니펫

상태 초기화

```
public void Setup(HeroBaseStat defaultBaseHeroStat, int round)
{
    _heroBaseStat = defaultBaseHeroStat;
    ClearHitData();

    // HP 초기화
    _healthPoint = _heroBaseStat.maxHealthPoint;
    OnHealthPointChanged();

    // 경직도 초기화
    _stiffnessPoint = 0;
    OnStiffnessChanged();

    // 기타 상태 초기화
    CurrentStance = IHeroStatus.Stance.StandUp;
    SetFocusState(false);
    AttackState = false;
    GuardState = false;
    InvincibleState = false;
}
```

상태 효과 시스템

```
public void ApplyStatusBoost(IHeroStatus.StatusBoost statusBoost)
{
    Coroutine coroutine;
    // 이미 동일한 타입의 부스트가 적용 중이면 갱신
    if (_statusBoosts.Any(boost => boost.boostType == statusBoost.boostType))
    {
        if (_activeStatusBoostCoroutines.TryGetValue(statusBoost.boostType, out coroutine))
        {
            Debug.Log($"#UpdateStatusBoost {statusBoost.boostType}");
            StopCoroutine(coroutine);
        }
        else
        {
            _activeStatusBoostCoroutines.Add(statusBoost.boostType, null);
        }
    }
    else
    {
        _statusBoosts.Add(statusBoost);

        // 새로운 부스트 코루틴 시작
        coroutine = StartCoroutine(CoUpdateStatusBoost(statusBoost));
    }
}
```



```

        coroutine = StartCoroutine(CoUpdateStatusBoost(statusBoost));
        _activeStatusBoostCoroutines[statusBoost.boostType] = coroutine;

        Debug.Log($"#ApplyStatusBoost {statusBoost.boostType} {statusBoost.boostValue}");
        OnStatusBoostChanged?.Invoke(statusBoost.boostType, StatusBoostStarted);
    }

    private IEnumerator CoUpdateStatusBoost(IHeroStatus.StatusBoost statusBoost)
    {
        // 부스트 효과 적용
        switch (statusBoost.boostType)
        {
            case IHeroStatus.StatusBoost.BoostType.GrowPower :
                var boostedPower = Mathf.RoundToInt(_heroBaseStat.power * statusBoost.boostValue);
                Debug.Log($"#Apply Power Boost {_heroBaseStat.power} => {boostedPower}");
                _heroBaseStat.ApplyPowerBoost(boostedPower);
                break;
            case IHeroStatus.StatusBoost.BoostType.GrowHealthPoint:
            case IHeroStatus.StatusBoost.BoostType.GrowSpeed:
                break;
        }

        // 지정된 시간 대기
        yield return new WaitForSeconds(statusBoost.boostDuration);

        // 부스트 효과 해제
        switch (statusBoost.boostType)
        {
            case IHeroStatus.StatusBoost.BoostType.GrowPower :
                _heroBaseStat.RevertPowerBoost();
                RemoveStatusBoost(IHeroStatus.StatusBoost.BoostType.GrowPower);
                break;
            case IHeroStatus.StatusBoost.BoostType.GrowHealthPoint:
            case IHeroStatus.StatusBoost.BoostType.GrowSpeed:
                break;
        }
    }

    private void RemoveStatusBoost(IHeroStatus.StatusBoost.BoostType statusBoost)
    {
        var targetBoost = _statusBoosts.FirstOrDefault(boost => boost.boostType == statusBoost);
        if (targetBoost.boostType == IHeroStatus.StatusBoost.BoostType.None) return;
        _statusBoosts.Remove(targetBoost);

        StopCoroutine(_activeStatusBoostCoroutines[statusBoost]);
        _activeStatusBoostCoroutines[statusBoost] = null;

        Debug.Log($"#RemoveStatusBoost {statusBoost}");
        OnStatusBoostChanged?.Invoke(statusBoost, StatusBoostEnded);
    }

    public void ClearStatusBoosts()
    {
        _statusBoosts.Clear();
        foreach (var activeStatusBoostCoroutine in _activeStatusBoostCoroutines)
        {
            StopCoroutine(activeStatusBoostCoroutine);
            activeStatusBoostCoroutine = null;
        }
    }

```

```

        StopCoroutine(activeStatusBoostCoroutine.Value);
        _activeStatusBoostCoroutines[activeStatusBoostCoroutine.Key] = null;
    }

    _heroBaseStat.RevertPowerBoost();
}

```

히트 처리

```

public OnHitDelegate OnHitHandle => OnHit;

private HitBoxHandler.HitResult OnHit(IHitEvent.HitInfo hitInfo)
{
    int savedHealthPoint = _healthPoint;
    RPC_UpdateHitInfo(hitInfo);
    int applyDamage = savedHealthPoint - _healthPoint;
    return new HitBoxHandler.HitResult
    {
        ApplyDamage = applyDamage,
        HealthPoint = _healthPoint,
    };
}

[Rpc(RpcSources.All, RpcTargets.All)]
private void RPC_UpdateHitInfo(IHitEvent.HitInfo hitInfo)
{
    heroController.OnHitReceive(hitInfo);
    UpdateStance(hitInfo.HitterInfo);
    UpdateStiffness(hitInfo);
}

private void UpdateStance(IHitEvent.HitInfo.HitterState hitterState)
{
    if (CurrentStance == hitterState.HitterStance) return;
    ChangeStance(hitterState.HitterStance);
    heroController.OnStanceChanged(hitterState.HitterStance);
}

private void UpdateStiffness(IHitEvent.HitInfo hitInfo)
{
    _stiffnessPoint = hitInfo.Stiffness;
    OnStiffnessChanged();
}

```

기능 설명

상태 관리 시스템

상태	타입	설명
HP	int	생명력 (0 ~ maxHealthPoint)

SP	int	특수 포인트 (0 ~ maxSpecialPoint)
경직 포인트	int	자세 변경 여부에 영향
자세	Stance	StandUp, Down, Stun 등
공격 상태	bool	현재 공격 중 여부
가드 상태	bool	현재 가드 중 여부
무적 상태	bool	피해 무효화 여부
포커스 모드	bool	타겟 고정 모드

상태 효과 타입

타입	효과
GrowPower	공격력 일시 증가
GrowHealthPoint	최대 생명력 일시 증가
GrowSpeed	이동 속도 일시 증가

이벤트 발생 조건

- **OnHpChanged**: HP가 변경될 때
- **OnDeath**: HP가 0 이하가 될 때
- **OnKill**: 적을 처치했을 때
- **OnSpChanged**: SP가 변경될 때
- **OnStiffnessPointChanged**: 경직 포인트가 변경될 때
- **OnStatusBoostChanged**: 상태 효과가 시작/종료될 때

네트워크 동기화 RPC

- `RPC_UpdateHealthPoint`: HP 동기화
- `RPC_UpdateStiffnessPoint`: 경직 포인트 동기화
- `RPC_UpdateHitInfo`: 히트 정보 동기화
- `RPC_SetFocusState`: 포커스 모드 동기화

의존성/상속 관계

- `NetworkBehaviour` 를 상속받음
- `IHeroStatus` 인터페이스 구현
- `HeroController` 클래스로 피격 정보 및 자세 변경 호출
- `HeroBaseStat` 스크립터를 오브젝트를 통해 플레이어 스탯 설정 및 변경
- `HitBoxHandler` 클래스에서 `OnHit` 메서드를 `OnHitHandle` 델리게이트를 통해 호출

사용 예시

NetworkHeroObject에서 상태 초기화 및 셋업

```
private void UpdateAbility(BaseHeroAbility.BaseAbilityType abilityType)
{
    ...

    // 능력 설정
    _baseHeroAbility.Setup(networkHeroStatus, heroController.HeroObjectControl
    networkHeroStatus.Setup(_baseHeroAbility.GetHeroStat(), 0);
    ...
}

[Rpc(RpcSources.InputAuthority, RpcTargets.All)]
private void RPC_ResetHero(int round)
{
    // 상태 초기화
    networkHeroStatus.Setup(_baseHeroAbility.GetHeroStat(), round);
    heroControllerStateHandler.ResetState();
    heroController.ResetFocusMode();
    heroAnimationController.StartMatch();

    // 첫 라운드인 경우 특수 게이지 초기화
    if (IsFirstRound(round)) networkHeroStatus.ChangeSpecialPoint(0);
}
```

각 캐릭터별 SkillManager에서 NetworkHeroStatus의 SP로 궁극기 준비 상태 확인

```
// BaseHeroAbility
public void UltimateSkill(NetworkHeroController.HeroInputSyncData syncData)
{
    if (baseSkillManager.IsUltimateAttackReady() == false) return;
    baseSkillManager.UltimateAttack(syncData);
    OnResetStiffnessRecovery();
}

// BaseSkillManager
public bool IsUltimateAttackReady()
{
    return OnUltimateAttackReady();
}

// RektorSkillManager
protected override bool OnUltimateAttackReady()
{
    var sp = HeroStatus.GetSp();
    if (sp.Current == sp.Max) return true;

    var result = new BaseHeroSkill.SkillFinishResult(ultimateAtk, BaseHeroSkill
    SkillEventInvoker.NotifySkillEvents(skillEvent => skillEvent?.OnSkillFinis
    return false;
}
```

OnHitHandle 델리게이트를 HitHandler에 셋팅

```
// NetworkHeroObject
private void UpdateAbility(BaseHeroAbility.BaseAbilityType abilityType)
{
    ...
    // 능력 설정
    _baseHeroAbility.Setup(networkHeroStatus, heroController.HeroObjectControl
networkHeroStatus.Setup(_baseHeroAbility.GetHeroStat() , 0);
    _baseHeroAbility.SetupHitBoxHandle(networkHeroStatus.OnHitHandle, networkH
    ...
}

// BaseHeroAbility
public void SetupHitBoxHandle(NetworkHeroStatus.OnHitDelegate hitDelegate, IHe
{
    hitBoxHandler.SetupHitBoxHandle(hitDelegate, heroStatus);
}

// HitBoxHandler
public void SetupHitBoxHandle(NetworkHeroStatus.OnHitDelegate hitDelegate, IHe
{
    _hitDelegate = hitDelegate;
    _heroStatus = heroStatus;
}

public HitResult OnHit(IHitEvent.HitInfo hitInfo)
{
    return _hitDelegate?.Invoke(hitInfo) ?? new HitResult();
}
```

관련 클래스

- [IHeroStatus](#)
- [NetworkHeroObject](#)
- [NetworkHeroController](#)
- [HeroController](#)
- [HeroBaseStat](#)
- [HitBoxHandler](#)

HeroInputData

Fusion 네트워크 전송을 위한 입력 데이터 컨테이너

개요

HeroInputData 구조체는 RFist 게임의 Fusion 네트워크 전송을 위한 입력 데이터를 담는 컨테이너입니다. 비트 마스크(Bit Mask)를 사용하여 효율적인 네트워크 전송을 지원하며, 이동 및 전투 입력 데이터를 캡슐화합니다.

역할

- 동기화할 입력 데이터 캡슐화
- 비트 마스크를 활용한 효율적인 버튼 상태 표현
- Fusion NetworkRunner에 의한 자동 직렬화/전송 대상 데이터 제공
- 이동 및 전투 입력 데이터 보관
- **INetworkInput** [구현체](#)

선언

```
public struct HeroInputData: INetworkInput
```

멤버

버튼 비트 마스크 정의

```
/// <summary>
/// 공격 버튼 비트 마스크
/// </summary>
public const uint Attack = 0x0000_0001;

/// <summary>
/// 공격 홀드 버튼 비트 마스크
/// </summary>
public const uint AttackHold = 0x0000_0002;

/// <summary>
/// 가드 버튼 비트 마스크
/// </summary>
public const uint Guard = 0x0000_0004;
```

```

public const uint Guard = 0x0000_0004;

/// <summary>
/// 공격+가드 동시 입력 버튼 비트 마스크
/// </summary>
public const uint AttackGuardDualInput = 0x0000_0005;

/// <summary>
/// 궁극기 버튼 비트 마스크
/// </summary>
public const uint UltimateAttack = 0x0000_0100;

/// <summary>
/// 대시 버튼 비트 마스크
/// </summary>
public const uint Dash = 0x0001_0000;

/// <summary>
/// 포커스 모드 버튼 비트 마스크
/// </summary>
public const uint FocusMode = 0x0010_0000;

/// <summary>
/// 포커스 변경 버튼 비트 마스크
/// </summary>
public const uint FocusChange = 0x0020_0000;

```

속성

```

/// <summary>
/// 수평 이동 입력값 (-1 ~ 1)
/// </summary>
public float Horizontal { get; set; }

/// <summary>
/// 수직 이동 입력값 (-1 ~ 1)
/// </summary>
public float Vertical { get; set; }

/// <summary>
/// 공격 타임스탬프
/// </summary>
public float AttackTimeStamp { get; set; }

/// <summary>
/// 버튼 상태 비트 마스크
/// </summary>
public uint Buttons { get; set; }

```

메서드

```

/// <summary>

```

```

/// <summary>
/// 공격 입력 여부 확인
/// </summary>
/// <returns>공격 입력 여부</returns>
public bool IsAttack() => (Buttons & Attack) == Attack;

/// <summary>
/// 공격 홀드 입력 여부 확인
/// </summary>
/// <returns>공격 홀드 입력 여부</returns>
public bool IsAttackHold() => (Buttons & AttackHold) == AttackHold;

/// <summary>
/// 가드 입력 여부 확인
/// </summary>
/// <returns>가드 입력 여부</returns>
public bool IsGuard() => (Buttons & Guard) == Guard;

/// <summary>
/// 대시 입력 여부 확인
/// </summary>
/// <returns>대시 입력 여부</returns>
public bool IsDash() => (Buttons & Dash) == Dash;

/// <summary>
/// 포커스 모드 입력 여부 확인
/// </summary>
/// <returns>포커스 모드 입력 여부</returns>
public bool IsFocusMode() => (Buttons & FocusMode) == FocusMode;

/// <summary>
/// 포커스 변경 입력 여부 확인
/// </summary>
/// <returns>포커스 변경 입력 여부</returns>
public bool IsFocusChange() => (Buttons & FocusChange) == FocusChange;

/// <summary>
/// 궁극기 입력 여부 확인
/// </summary>
/// <returns>궁극기 입력 여부</returns>
public bool IsUltimateSkill() => (Buttons & UltimateAttack) == UltimateAttack;

/// <summary>
/// 공격+가드 동시 입력 여부 확인
/// </summary>
/// <returns>공격+가드 동시 입력 여부</returns>
public bool IsAttackGuardDualInput() => (Buttons & AttackGuardDualInput) == At

```

기능 설명

비트 마스크 구조

- `HeroInputData`는 uint 타입의 `Buttons` 필드에 여러 버튼 상태를 비트 단위로 저장

상수	비트 위치	16진수 값	설명
Attack	0	0x0000_0001	공격
AttackHold	1	0x0000_0002	공격 홀드
Guard	2	0x0000_0004	가드
AttackGuardDualInput	3	0x0000_0005	공격+가드 동시 입력
UltimateAttack	8	0x0000_0100	궁극기
Dash	16	0x0001_0000	대시
FocusMode	20	0x0010_0000	포커스 모드
FocusChange	21	0x0020_0000	포커스 변경

네트워크 전송 구조체

- `HeroInputData`는 `INetworkInput` [↗](#) 인터페이스를 구현하여 Fusion의 네트워크 시스템에서 사용됨
- 클라이언트에서 입력 데이터 생성
- `Buttons` 비트 마스크로 효율적인 직렬화

입력 데이터 구성

- Horizontal / Vertical(float) : 8bytes
- AttackTimeStamp(float) : 4bytes
- Buttons(uint) : 4bytes

의존성/상속 관계

- `INetworkInput` [↗](#) (Fusion 인터페이스) 구현
- `NetworkHeroController`에서 입력 수집 및 동기화
- `NetworkHeroInput`에서 로컬 입력을 `HeroInputData`로 변환

사용 예시

`NetworkHeroInput`에서 `HeroInputData` 생성

```
public override void OnInput(NetworkRunner runner, NetworkInput input)
{
    // 입력 권한이 없으면 처리하지 않음
    if (!Object.HasInputAuthority) return;

    // 새로운 입력 데이터 생성
    var heroInputData = new HeroInputData();

    // 등록된 모든 입력 소스에서 입력 수집
```

```

foreach (var heroInput in BaseHeroInput.HeroInputList)
{
    // 수평 이동 입력 수집 (이미 값이 있으면 건드리지 않음)
    if (heroInputData.Horizontal == 0)
    {
        heroInputData.Horizontal = heroInput.GetHorizontal();
    }

    // 수직 이동 입력 수집 (이미 값이 있으면 건드리지 않음)
    if (heroInputData.Vertical == 0)
    {
        heroInputData.Vertical = heroInput.GetVertical();
    }

    // 대시 입력 수집
    if (heroInput.IsDash())
    {
        heroInputData.Buttons |= HeroInputData.Dash;
    }

    // 공격 입력 수집 및 타임스탬프 기록
    if (heroInput.IsAttack())
    {
        heroInputData.Buttons |= HeroInputData.Attack;
        heroInputData.AttackTimeStamp = runner.SimulationTime;
    }

    // 공격 홀드(차지) 입력 수집 및 타임스탬프 기록
    if (heroInput.IsAttackHold())
    {
        heroInputData.Buttons |= HeroInputData.AttackHold;
        heroInputData.AttackTimeStamp = runner.SimulationTime;
    }
    else
    {
        // 홀드 해제 시에도 타임스탬프 갱신
        heroInputData.AttackTimeStamp = runner.SimulationTime;
    }

    // 가드 입력 수집 및 타임스탬프 기록
    if (heroInput.IsGuard())
    {
        heroInputData.Buttons |= HeroInputData.Guard;
        heroInputData.AttackTimeStamp = runner.SimulationTime;
    }

    // 포커스 모드 입력 수집
    if (heroInput.IsFocusMode())
    {
        heroInputData.Buttons |= HeroInputData.FocusMode;
    }

    // 포커스 변경 입력 수집
    if (heroInput.IsFocusChange())
    {
        heroInputData.Buttons |= HeroInputData.FocusChange;
    }
}

```

```

    }

    // 궁극기 입력 수집 및 타임스탬프 기록
    if (heroInput.IsUltimateAttack())
    {
        heroInputData.Buttons |= HeroInputData.UltimateAttack;
        heroInputData.AttackTimeStamp = runner.SimulationTime;
    }

    // 공격+가드 동시 입력 수집 및 타임스탬프 기록
    if (heroInput.IsAttackGuardDualInput())
    {
        heroInputData.Buttons |= HeroInputData.AttackGuardDualInput;
        heroInputData.AttackTimeStamp = runner.SimulationTime;
    }
}

// 수집된 입력 데이터를 네트워크 입력으로 설정
input.Set(heroInputData);
}

```

NetworkHeroController에서 입력데이터 수신 및 전송

```

public override void FixedUpdateNetwork()
{
    if (GetInput<HeroInputData>(out var heroInputData))
    {
        // 컨트롤러가 비활성화된 경우 기본값 전송
        if (heroController.IsActive == false)
        {
            RPC_UpdateHeroInputData(default);
            return;
        }
        // 입력 데이터를 모든 클라이언트에 동기화
        RPC_UpdateHeroInputData(heroInputData);
        FixedInputData = heroInputData;
    }
}

[Rpc(RpcSources.InputAuthority, RpcTargets.All)]
private void RPC_UpdateHeroInputData(HeroInputData heroInputData)
{
    // 이동 입력 처리
    heroController.Move(heroInputData.Horizontal, heroInputData.Vertical, Runn

    // 개별 스킬/액션 입력 처리
    ProcessDash(heroInputData);
    ProcessAttack(heroInputData);
    ProcessAttackHold(heroInputData);
    ProcessFocus(heroInputData);
    ProcessGuard(heroInputData);
    ProcessUltimateSkill(heroInputData);
}

```

관련 클래스

- [NetworkHeroController](#)
- [NetworkHeroInput](#)
- [HeroKeyInput](#)
- [BaseHeroInput](#)
- [IHeroInput](#)

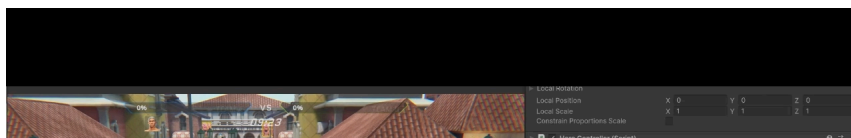
RFist 게임의 영웅 캐릭터 입력, 이동, 상태 관리를 통합 제어하는 시스템

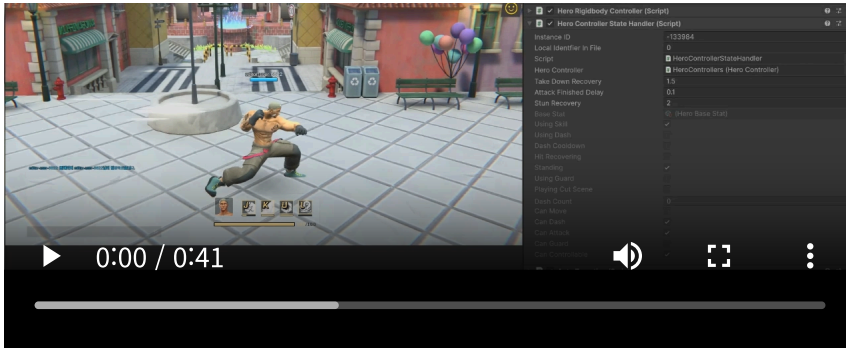
- **HeroControlSystem**은 RFist 게임의 **영웅 캐릭터 제어 시스템**으로, **입력 처리, 이동 제어, 상태 관리, 컷씬 제어**를 통합적으로 관리하는 시스템입니다. 6가지 이벤트 타입을 기반으로 한 움저버 패턴을 통해 애니메이션, 스킬, 피격 등의 시스템과 느슨하게 결합되어 동작합니다.

- **다양한 입력 방식 지원:** 키보드, 게임패드 등 다양한 입력 디바이스를 추상화하여 일관된 인터페이스 제공
- **물리 기반 + 트윈 기반 이동:** Rigidbody 기반이동과 DoTween 기반 스킬 동작 이동 지원
- **상태 기반 행동 제어:** 스킬 사용, 피격, 다운, 쿨다운 등 상태에 따라 입력 가능 여부를 동적으로 제어
- **네트워크 동기화 지원:** Fusion 네트워크 환경에서 입력 데이터를 효율적으로 동기화
- **컷씬(궁극기) 제어:** Timeline 기반 특수 스킬 실행 중 캐릭터 제어 및 데미지 처리

기능	설명
입력 처리	Input System 기반 키보드 입력 처리 및 네트워크 동기화용 데이터 구조 제공
이동 제어	Rigidbody 기반 이동 + DoTween 트윈 기반 이동 지원
상태 관리	스킬/피격/가드/대시/컷씬/쿨다운 상태에 따른 행동 가능 여부 동적 관리
이벤트 관리 및 통지	6가지 이벤트 타입(MoveControl, Hit, Skill, Attack, Stance, CutScene) 옱저버 패턴 구현
컷씬 유틸리티	Timeline 기반 궁극기 컷씬 중 이벤트 처리 및 데미지 적용

1. 이동제어, 입력 상태관리, 컷싹 제어





2. 사용된 기술 요소

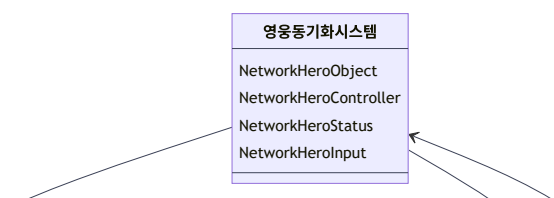
핵심 기술 요소 및 API 활용

요소	설명
C#	전체 핵심 로직 및 유니티 컴포넌트 구현
Input System	Action 기반 입력 처리 및 디바이스 독립적 입력 추상화
DoTween	트윈 기반 이동(대시, 넉백, 스킬 동작) 구현
Fusion	네트워크 입력 동기화 및 RPC 기반 입력 데이터 전송
↳ NetworkInput	입력 데이터 네트워크 동기화

설계 활용 패턴

요소	설명
옵저버 패턴 (Observer Pattern)	HeroController 에서 6가지 이벤트 타입을 관리하고, 관련 시스템(애니메이션, 스킬 등)에 이벤트 통지
템플릿 메서드 패턴 (Template Method Pattern)	BaseHeroInput , BaseHeroMoveController 에서 공통 인터페이스 정의 및 하위 클래스에서 구체적인 구현
전략 패턴 (Strategy Pattern)	IHeroInput 인터페이스를 통해 다양한 입력 구현체를 런타임에 교체 가능

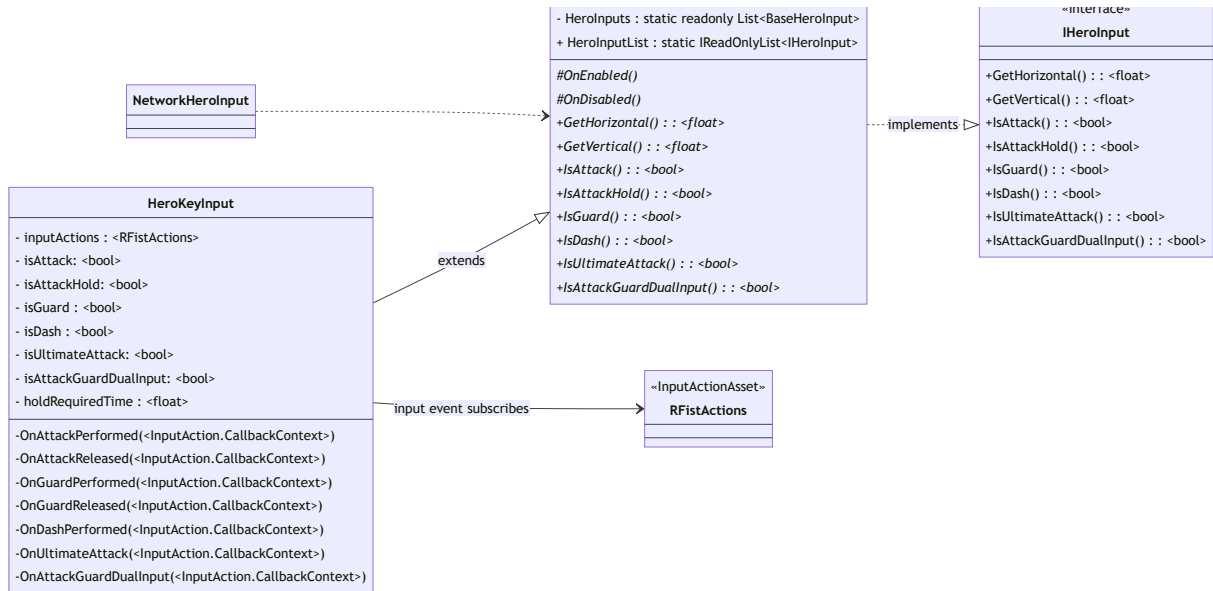
3. 전체 시스템 구조도(간략)





```

classDiagram
    class BaseHeroInput {
        <<abstract>>
    }
    class HeroInput {
    }
    BaseHeroInput <|-- HeroInput
    
```

동작 제어 클래스

클래스

역할

IHeroController

«interface»

- 💡 영웅 이동 및 제어 기능 계약 정의
- 💡 DoTween 기반 절대/상대 이동, 넉백, 대시 인터페이스 제공
- 💡 타겟팅 및 방향전환 기능 정의
- 💡 Tween 동작 취소 기능 정의

BaseHeroMoveController

«abstract»

- 💡 이동 제어의 **기반 추상 클래스**
- 💡 템플릿 메서드 패턴을 통한 구현체 확장 지원
- 💡 이동, 회전, 순간 이동, 넉백 등 기본 동작 정의

HeroRigidbodyController

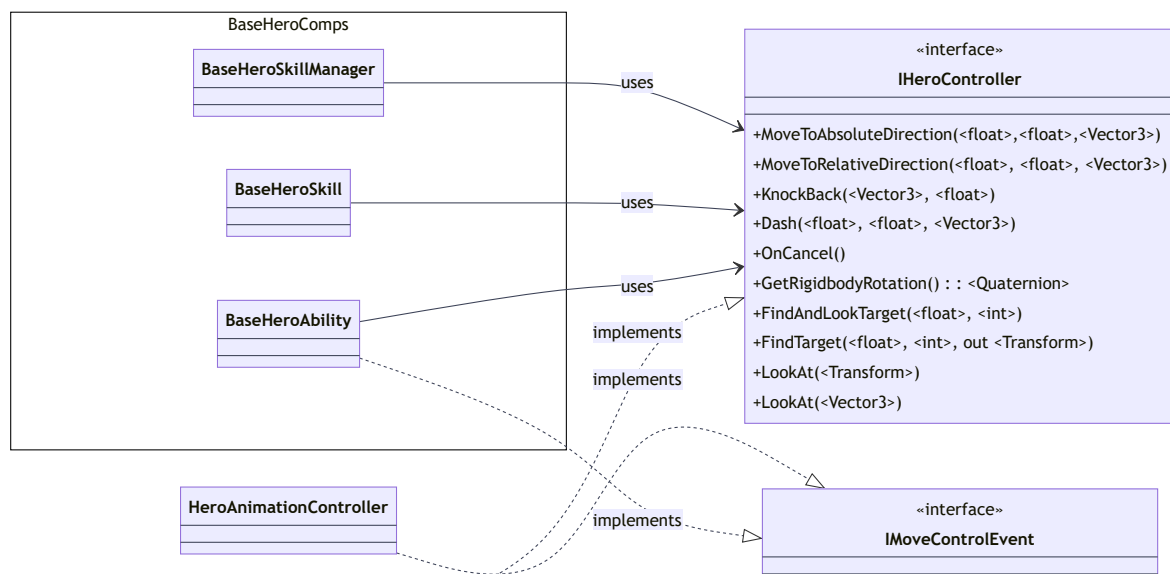
: BaseHeroMoveController

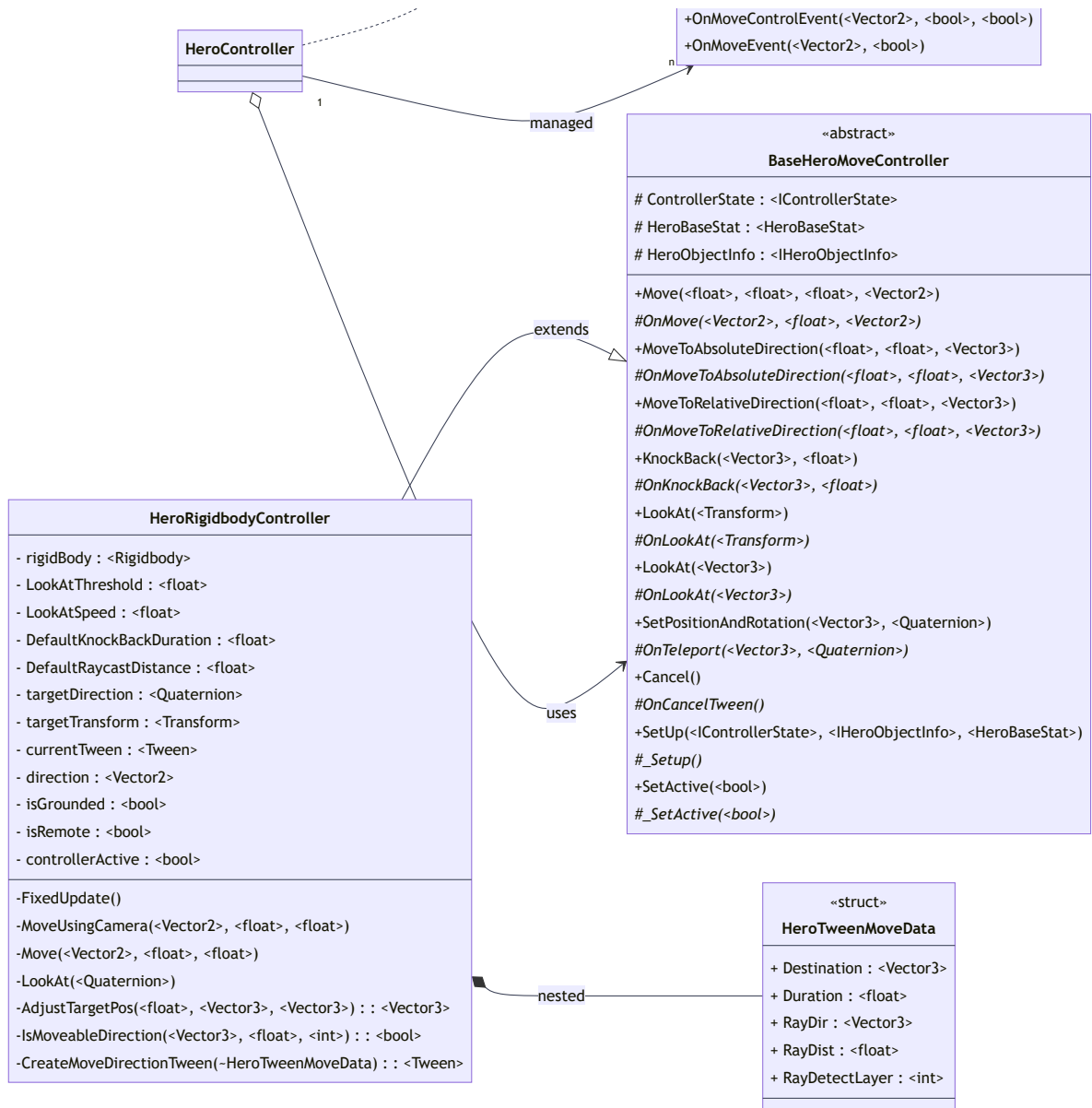
- 💡 **Rigidbody/Tween 기반 이동 구현체**
- 💡 카메라 기준 상대 이동 및 회전 처리
- 💡 DoTween 트윈 이동 + 벽/아바타 충돌 감지
- 💡 넉백, 대시 등 특수 이동 구현

IMoveControlEvent

«interface»

- 💡 이동 입력 이벤트 및 실제 이동 발생 이벤트 정의
- 💡 이동 방향, 포커스 모드, 제어 가능 여부 전달





상태 관리 클래스

클래스

IControllerState

«interface»

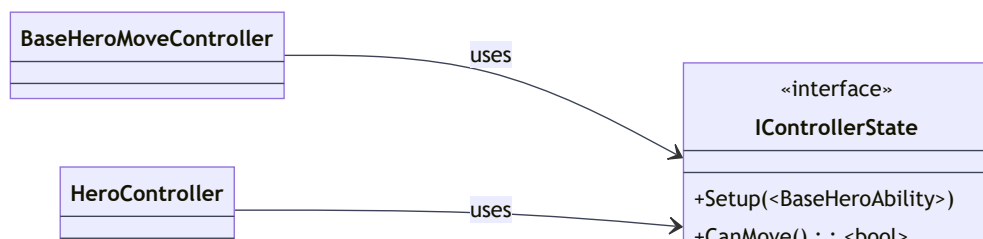
HeroControllerStateHandler

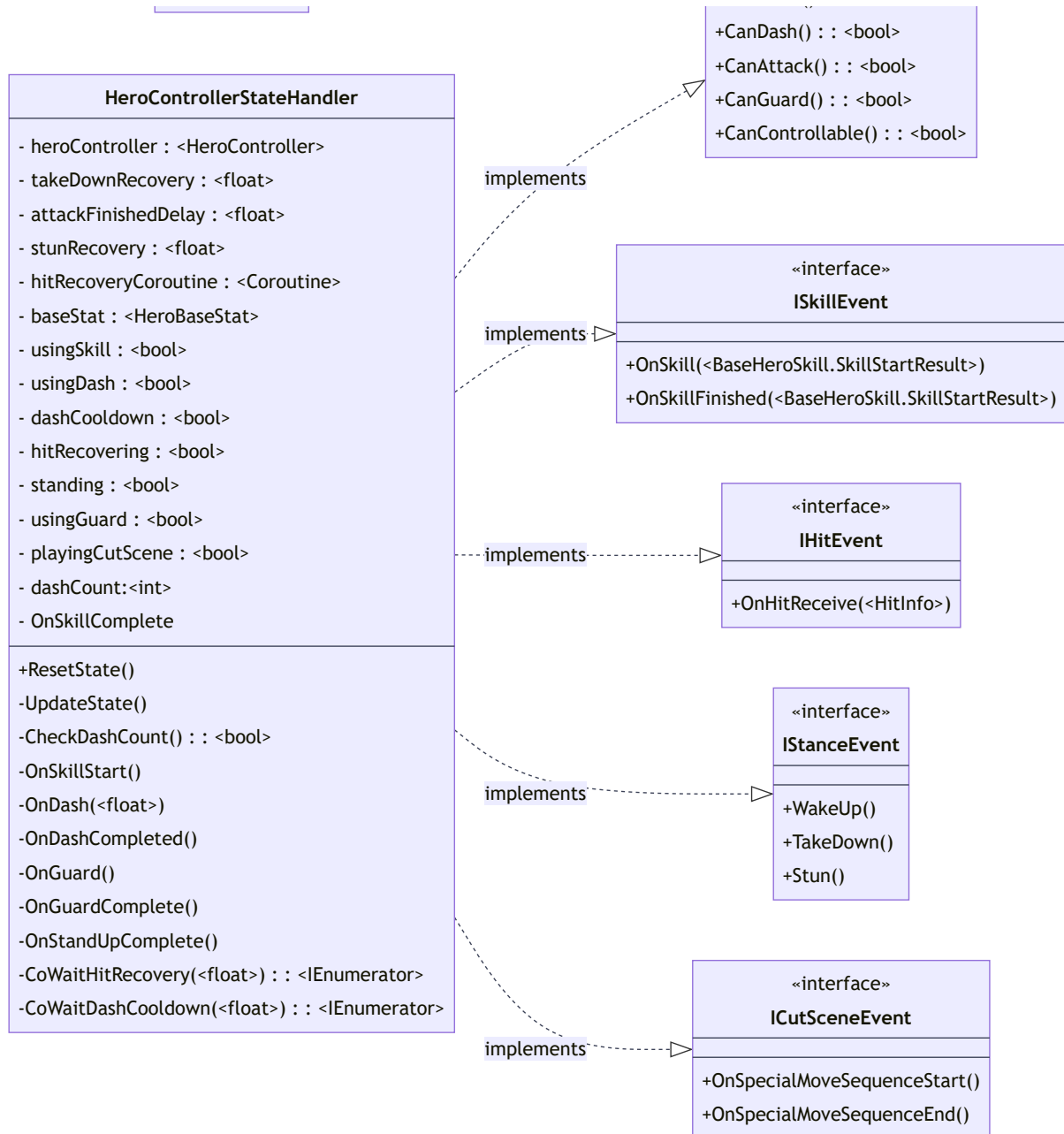
: IControllerState, ISkillEvent, IHitEvent, IStanceEvent, ICutSceneEvent

역할

- 💡 컨트롤러 상태 계약 정의 (CanMove, CanAttack, CanGuard, CanDash, CanControllable)
- 💡 상태 기반 행동 제어의 표준 인터페이스

- 💡 상태 기반 행동 제어의 핵심 구현체
- 💡 스킬 시작/종료, 피격, 자세 변경, 컷씬 등의 이벤트 수신 및 컨트롤 제한
- 💡 대시 횟수 및 쿨다운 관리





컷씬 관리 클래스

클래스

역할

ICutSceneEvent

«interface»

- 💡 컷씬(특수 스킬) 시작/종료 이벤트 정의

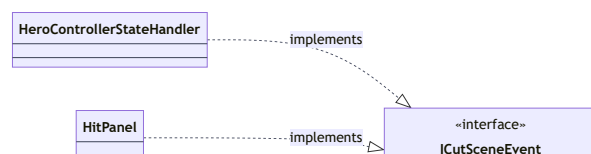
ICutSceneEventInvoker

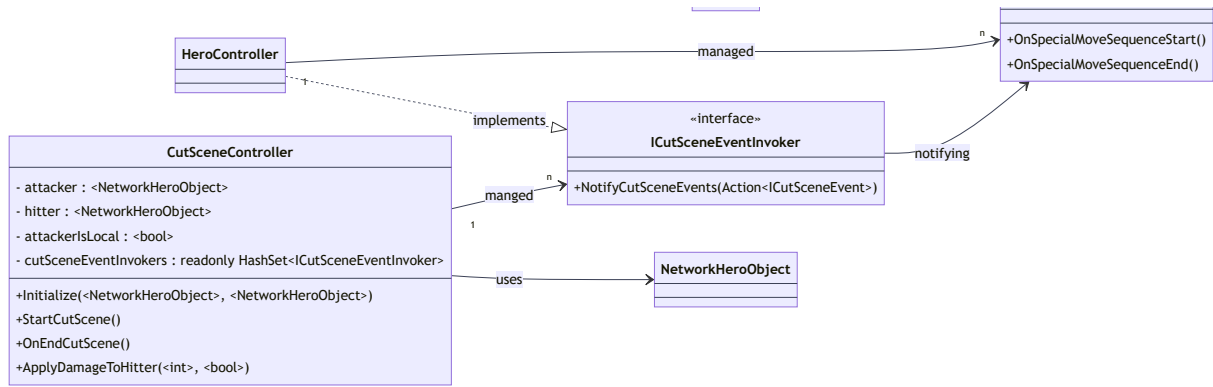
«interface»

- 💡 컷씬 이벤트 발신자 역할 정의
- 💡 `NotifyCutSceneEvents()` 로 모든 구독자에게 일괄 발신

CutSceneController

- 💡 `TimelineManager` 에서 발생하는 컷씬 이벤트 처리 (시작/종료)
- 💡 컷씬 중 피격자에게 데미지 적용
- 💡 로컬/원격 플레이어 구분에 따른 공격 이벤트 처리





5. 주요 특징

기능의 특징

- **입력 추상화 계층:** **IHeroInput** 인터페이스와 **BaseHeroInput** 추상 클래스를 통해 키보드, 게임패드, 모바일 등 다양한 입력 방식을 추상화하고, **HeroInputData** 비트 마스크로 네트워크 효율성 극대화
- **이중 이동 시스템:** Rigidbody 기반 물리 이동(자유로운 이동)과 DoTween 기반 트윈 이동(정밀한 스킬 동작)을 상황에 맞게 선택 가능
- **상태 기반 행동 제어:** **IControllerState** 인터페이스를 통해 스킬 사용, 피격, 다운, 스톤, 컷씬 등의 상태에 따라 이동/공격/가드/대시 가능 여부를 정밀하게 제어
- **6중 이벤트 옵저버 패턴:** MoveControl, Hit, Skill, Attack, Stance, CutScene 6가지 이벤트 타입을 독립적으로 관리하여 애니메이션, 스킬, 피격 시스템과 느슨한 결합 유지
- **컷씬 제어 통합:** Timeline 기반 궁극기 컷씬 실행 중에도 캐릭터 상태 관리 및 데미지 처리가 가능한 통합 컨트롤 시스템

6. UseCase

전투 입력 처리 시나리오

- 1 **입력 수집:** **HeroKeyInput** 이 Input System으로부터 키 입력을 감지
- 2 **네트워크 동기화:** **NetworkHeroInput** 이 **BaseHeroInput.HeroInputList** 에서 모든 입력 소스의 데이터를 수집하여 **HeroInputData** 로 변환
- 3 **입력 전송:** Fusion **NetworkRunner** 를 통해 모든 클라이언트에 입력 데이터 동기화
- 4 **입력 처리:** **HeroController** 가 동기화된 입력을 수신하여 상태 확인 (**IControllerState.CanAttack** 등)
- 5 **동작 실행:** 상태가 허용하면 **BaseHeroAbility** 를 통해 공격/가드/대시 등의 스킬 실행
- 6 **이벤트 통지:** 스킬 시작/종료 시 **HeroController.NotifySkillEvents()** 로 관련 시스템에 이벤트 발신

피격 및 넉백 시나리오

- 1 피격 수신: `HeroController.OnHitReceive()`로 피격 정보 수신
- 2 상태 전환: `HeroControllerStateHandler.OnHitReceive()`에서 피격 회복 상태로 전환 및 `CanMove/CanAttack` 비활성화
- 3 넉백 실행: `HeroController.KnockBack()` → `HeroRigidbodyController.OnKnockBack()`으로 DoTween 기반 넉백 동작 실행
- 4 상태 복원: 회복 시간 경과 후 `UpdateState()`로 행동 가능 상태 복원

궁극기(컷씬) 실행 시나리오

- 1 컷씬 시작: `CutSceneController.StartCutScene()` 호출
- 2 이벤트 발신: `ICutSceneEventInvoker.NotifyCutSceneEvents()`로 모든 구독자에게 컷씬 시작 통지
- 3 상태 제한: `HeroControllerStateHandler.OnSpecialMoveSequenceStart()` (`/docs/projects/rfist/herocontrolsystem/herocontrollerstatehandler`)에서 `_playingCutScene = true` 설정으로 모든 행동 제한
- 4 데미지 처리: Timeline 프레임 이벤트에서 `CutSceneController.ApplyDamageToHitter()`로 피격자 데미지 적용
- 5 컷씬 종료: `OnEndCutScene()`에서 `_playingCutScene = false` 설정으로 행동 가능 상태 복원

주요 사용처

- PvP 전투 게임의 캐릭터 제어: 입력 → 이동 → 스킬 → 피격의 전체 흐름 관리
- 네트워크 동기화 환경: Fusion 기반 멀티플레이어 게임의 입력 동기화
- 복잡한 상태 머신이 필요한 캐릭터: 스킬 캔슬, 피격 경직, 다운/스턴 등 다양한 상태 전환이 필요한 캐릭터
- 컷씬 기반 특수 스킬: Timeline 연동 궁극기 시스템이 필요한 게임

HeroController

영웅 캐릭터의 전반적인 제어를 담당하는 핵심 컨트롤러 클래스

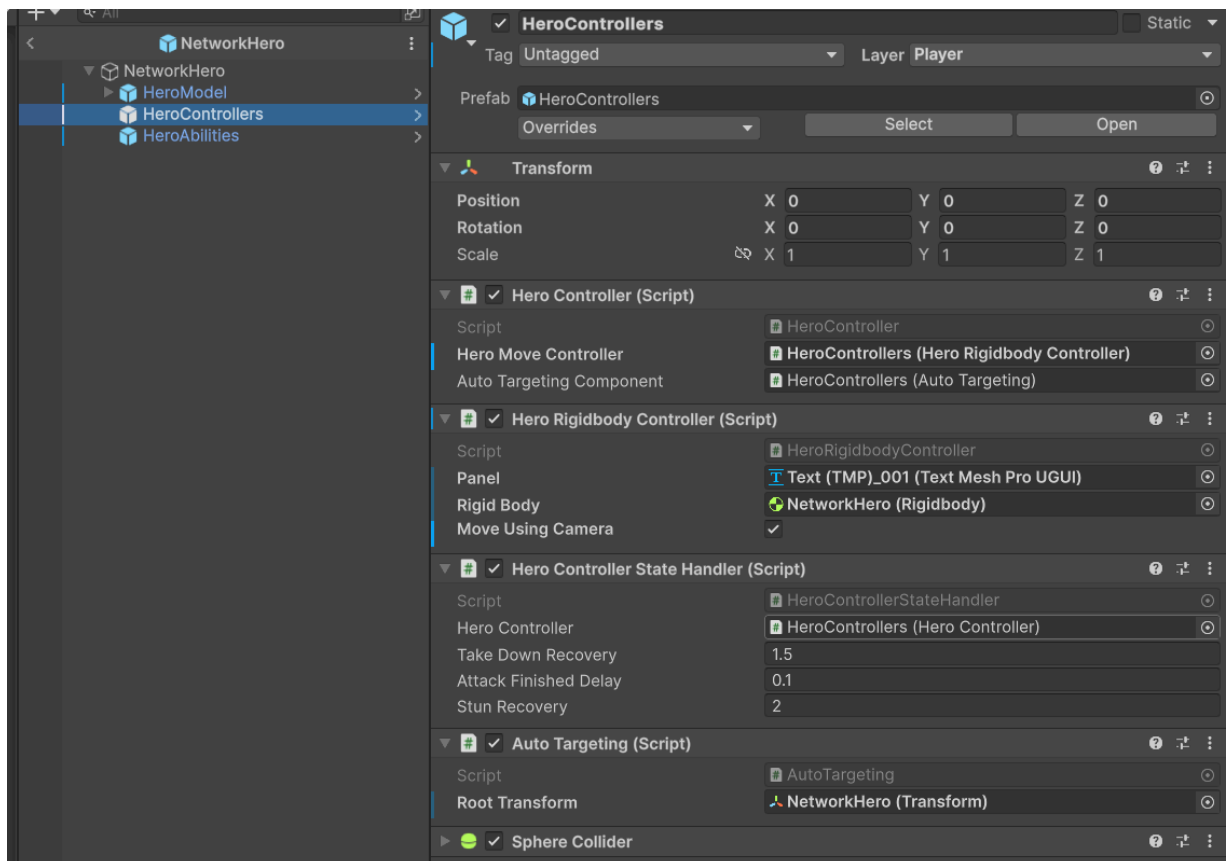
개요

HeroController 클래스는 RFist 게임의 영웅 캐릭터 전반을 제어하는 핵심 컨트롤러입니다. 이동, 전투 액션, 이벤트 통지 시스템 등의 기능을 통합 관리하며, 다양한 이벤트 인터페이스를 통해 애니메이션, 스킬, 피격 등을 담당하는 시스템과 연동됩니다.

역할

- 영웅 캐릭터의 이동 및 전투 액션 제어
- 6가지 이벤트 타입(MoveControl, Hit, Skill, Attack, Stance, CutScene)의 등록 및 통지
- 컨트롤러 상태(IControllerState) 기반 행동 제어
- 자동 타겟팅 및 방향 전환 처리

HeroController 오브젝트 구성 및 Hierarchy 구조



서언

```
public class HeroController : MonoBehaviour, IHeroController, IHeroDetector, IHeroAbilityInvoker
```

멤버

속성

```
/// <summary>
/// 스킬 이벤트 발신자 인터페이스
/// </summary>
public ISkillEventInvoker SkillEventNotifier => this;

/// <summary>
/// 영웅 컨트롤러 인터페이스
/// </summary>
public IHeroController HeroObjectController => this;

/// <summary>
/// 컨트롤러 활성화 여부
/// </summary>
public bool IsActive => _isActive;

/// <summary>
/// 영웅 능력치 컴포넌트
/// </summary>
private BaseHeroAbility _baseHeroAbility;

/// <summary>
/// 이동 제어 이벤트 리스너 목록
/// </summary>
private readonly HashSet<IMoveControlEvent> _moveControlEvents = new HashSet<IMoveControlEvent>();

/// <summary>
/// 피격 이벤트 리스너 목록
/// </summary>
private readonly HashSet<IHitEvent> _hitEvents = new HashSet<IHitEvent>();

/// <summary>
/// 스킬 이벤트 리스너 목록
/// </summary>
private readonly HashSet<ISkillEvent> _skillEvents = new HashSet<ISkillEvent>();

/// <summary>
/// 공격 이벤트 리스너 목록
/// </summary>
private readonly HashSet<IAttackEvent> _attackEvents = new HashSet<IAttackEvent>();

/// <summary>
/// 자세 변경 이벤트 리스너 목록
/// </summary>
private readonly HashSet<IStanceEvent> _stanceEvents = new HashSet<IStanceEvent>();
```

```

/// <summary>
/// 컷씬 이벤트 리스너 목록
/// </summary>
private readonly HashSet<ICutSceneEvent> _cutSceneEvents = new HashSet<ICutSceneEvent>();

/// <summary>
/// 컨트롤러 상태 핸들러
/// </summary>
private IControllerState _controllerState;

```

메서드

```

/// <summary>
/// 컨트롤러 상태 핸들러 설정
/// </summary>
/// <param name="controllerState">컨트롤러 상태 인터페이스</param>
public void SetupControllerStateHandler(IControllerState controllerState)

/// <summary>
/// 컨트롤러 초기 설정
/// </summary>
/// <param name="heroAbility">영웅 능력치 컴포넌트</param>
/// <param name="heroStatus">영웅 상태 인터페이스</param>
/// <param name="heroObjectInfo">영웅 오브젝트 정보</param>
public void Setup(BaseHeroAbility heroAbility, IHeroStatus heroStatus, IHeroObjectInfo heroObjectInfo)

/// <summary>
/// 로컬 플레이어 설정
/// </summary>
/// <param name="hasInputAuthority">입력 권한 여부</param>
public void SetLocalPlayer(bool hasInputAuthority)

/// <summary>
/// 방향키 이동 처리
/// </summary>
/// <param name="horizontal">수평 입력값</param>
/// <param name="vertical">수직 입력값</param>
/// <param name="deltaTime">델타 타임</param>
public void Move(float horizontal, float vertical, float deltaTime)

/// <summary>
/// DoTween을 사용하여 절대 방향으로 이동
/// </summary>
/// <param name="distance">이동 거리</param>
/// <param name="time">이동 시간</param>
/// <param name="direction">이동 방향</param>
public void MoveToAbsoluteDirection(float distance, float time, Vector3 direction)

/// <summary>
/// DoTween을 사용하여 상대 방향으로 이동
/// </summary>
/// <param name="distance">이동 거리</param>
/// <param name="time">이동 시간</param>

```



```

/// <param name="time">이동 시간</param>
/// <param name="direction">이동 방향</param>
public void MoveToRelativeDirection(float distance, float time, Vector3 direction)

/// <summary>
/// DoTween을 사용하여 넉백 이동
/// </summary>
/// <param name="attackerPos">공격자 위치 (넉백 방향 계산용)</param>
/// <param name="force">넉백 힘</param>
public void KnockBack(Vector3 attackerPos, float force)

/// <summary>
/// 대시 이동(IHeroController 구현)
/// </summary>
/// <param name="distance">이동 거리</param>
/// <param name="time">이동 시간</param>
/// <param name="direction">이동 방향</param>
public void Dash(float distance, float time, Vector3 direction)

/// <summary>
/// 타겟을 찾아 바라보기
/// </summary>
/// <param name="autoTargetingDistance">자동 타겟팅 거리</param>
/// <param name="autoTargetingAngle">자동 타겟팅 각도</param>
public void FindAndLookTarget(float autoTargetingDistance = 0, int autoTargetingAngle = 0)

/// <summary>
/// 타겟 찾기
/// </summary>
/// <param name="targetDistanceRange">타겟 감지 거리</param>
/// <param name="targetAngleRange">타겟 감지 각도</param>
/// <param name="target">찾은 타겟 트랜스폼</param>
/// <returns>타겟 찾기 성공 여부</returns>
public bool FindTarget(float targetDistanceRange, int targetAngleRange, out Transform target)

/// <summary>
/// 특정 방향 바라보기
/// </summary>
/// <param name="direction">바라볼 방향</param>
public void LookAt(Vector3 direction)

/// <summary>
/// 특정 타겟 바라보기
/// </summary>
/// <param name="target">바라볼 타겟 트랜스폼</param>
public void LookAt(Transform target)

/// <summary>
/// 피격 수신 처리
/// </summary>
/// <param name="hitInfo">피격 정보</param>
public void OnHitReceive(IHitEvent.HitInfo hitInfo)

// 전투입력 메서드
public void Dash(float vertical, float horizontal)
public void Attack(NetworkHeroController.HeroInputSyncData syncData)

```

```

public void AttackHold(NetworkHeroController.HeroInputSyncData syncData)
public void StopAttackHold(NetworkHeroController.HeroInputSyncData syncData)
public void Guard(NetworkHeroController.HeroInputSyncData syncData)
public void StopGuard(NetworkHeroController.HeroInputSyncData syncData)
public void UltimateSkill(NetworkHeroController.HeroInputSyncData syncData)
public void AttackGuardDualInput(NetworkHeroController.HeroInputSyncData syncData)

// 이벤트 발신 메서드
public void NotifySkillEvents(Action<ISkillEvent> action)
private void NotifyHitEvents(Action<IHitEvent> action)
private void NotifyMoveControlEvents(Action<IMoveControlEvent> action)
public void NotifyAttackEvents(Action<IAttackEvent> action)
private void NotifyStanceEvents(Action<IStanceEvent> action)
public void NotifyCutSceneEvents(Action<ICutSceneEvent> action)

// 이벤트 레지스터 메서드
public void Register(IMoveControlEvent moveControlEvent)
public void UnRegister(IMoveControlEvent moveControlEvent)
public void Register(IHitEvent hitEvent)
public void UnRegister(IHitEvent hitEvent)
public void Register(ISkillEvent skillEvent)
public void UnRegister(ISkillEvent skillEvent)
public void Register(IAttackEvent attackEvent)
public void UnRegister(IAttackEvent attackEvent)
public void Register(IStanceEvent stanceEvent)
public void UnRegister(IStanceEvent stanceEvent)
public void Register(ICutSceneEvent cutSceneEvent)
public void UnRegister(ICutSceneEvent cutSceneEvent)

```

코드 스니펫

초기 설정

```

/// <summary>
/// 컨트롤러 초기 설정
/// </summary>
public void Setup(BaseHeroAbility heroAbility, IHeroStatus heroStatus, IHeroObject heroObject)
{
    _baseHeroAbility = heroAbility;

    // 이벤트 리스너 등록
    Register(_baseHeroAbility.HitEvent);
    Register(_baseHeroAbility.MoveControlEvent);
    Register(_baseHeroAbility.StanceEvent);
    Register(_baseHeroAbility.SkillEvent);

    // 컨트롤러 상태 및 이동 컨트롤러 설정
    _controllerState.Setup(_baseHeroAbility);
    heroMoveController.Setup(_controllerState, heroObjectInfo, heroAbility.GetStatus);
}

/// <summary>
/// 로컬 플레이어 설정

```

```

/// <summary>
/// </summary>
/// <param name="hasInputAuthority">입력 권한 여부</param>
public void SetLocalPlayer(bool hasInputAuthority)
{
    _isLocalPlayer = hasInputAuthority;
}

/// <summary>
/// 컨트롤러 상태 핸들러 설정
/// </summary>
/// <param name="controllerState">컨트롤러 상태 인터페이스</param>
public void SetupControllerStateHandler(IControllerState controllerState)
{
    _controllerState = controllerState;
}

```

이동 처리 로직

```

/// <summary>
/// 이동 처리
/// </summary>
public void Move(float horizontal, float vertical, float deltaTime)
{
    _moveDirection = new Vector2(horizontal, vertical);

    // 이동 제어 이벤트 통지
    NotifyMoveControlEvents(moveEvent ⇒ moveEvent?.OnMoveControlEvent(_moveDirection));
    if (_controllerState.CanMove == false) return;

    if (!_isFocusMode)
    {
        // 일반 이동 모드
        if (_lastMoveDirection != _moveDirection)
        {
            // 방향이 변경된 경우
            NotifyMoveControlEvents(moveEvent ⇒ moveEvent?.OnMoveEvent(_moveDirection));
            heroMoveController.Move(horizontal, vertical, deltaTime, _moveDirection);
            _lastMoveDirection = _moveDirection;
        }
        else
        {
            // 동일한 방향 유지
            heroMoveController.Move(horizontal, vertical, deltaTime, Vector2.zero);
        }
    }
    else
    {
        // 포커스 모드 이동 (타겟을 바라보며 이동)
        if (_focusTarget != null)
        {
            heroMoveController.FocusMove(horizontal, vertical, deltaTime, _focusTarget);
        }
        NotifyMoveControlEvents(moveEvent ⇒ moveEvent?.OnMoveEvent(_moveDirection));
    }
}

```

```
}  
}
```

트윈 이동 호출

```
/// <summary>  
/// DoTween을 사용하여 절대 방향으로 이동  
/// </summary>  
/// <param name="distance">이동 거리</param>  
/// <param name="time">이동 시간</param>  
/// <param name="direction">이동 방향</param>  
public void MoveToAbsoluteDirection(float distance, float time, Vector3 direction)  
{  
    heroMoveController.MoveToAbsoluteDirection(distance, time, direction);  
}  
  
/// <summary>  
/// DoTween을 사용하여 상대 방향으로 이동  
/// </summary>  
/// <param name="distance">이동 거리</param>  
/// <param name="time">이동 시간</param>  
/// <param name="direction">이동 방향</param>  
public void MoveToRelativeDirection(float distance, float time, Vector3 direction)  
{  
    heroMoveController.MoveToRelativeDirection(distance, time, direction);  
}  
  
/// <summary>  
/// DoTween을 사용하여 넉백 처리  
/// </summary>  
/// <param name="attackerPos">공격자 위치 (넉백 방향 계산용)</param>  
/// <param name="force">넉백 힘</param>  
public void KnockBack(Vector3 attackerPos, float force)  
{  
    heroMoveController.KnockBack(attackerPos, force);  
}
```

타겟팅 및 방향전환 메서드

```
/// <summary>  
/// 타겟을 찾아 바라보기  
/// </summary>  
/// <param name="autoTargetingDistance">자동 타겟팅 거리</param>  
/// <param name="autoTargetingAngle">자동 타겟팅 각도</param>  
public void FindAndLookTarget(float autoTargetingDistance = 0, int autoTargetingAngle)  
{  
    if (_isLocalPlayer == false) return;  
    if (FindTarget(autoTargetingDistance, autoTargetingAngle, out var target))  
    {  
        LookAt(target);  
    }  
}
```

```

}

/// <summary>
/// 타겟 찾기
/// </summary>
/// <param name="targetDistanceRange">타겟 감지 거리</param>
/// <param name="targetAngleRange">타겟 감지 각도</param>
/// <param name="target">찾은 타겟 트랜스폼</param>
/// <returns>타겟 찾기 성공 여부</returns>
public bool FindTarget(float targetDistanceRange, int targetAngleRange, out Transform target)
{
    return autoTargetingComponent.FindTarget(targetAngleRange, targetAngleRange, targetDistanceRange, out target);
}

/// <summary>
/// 특정 방향 바라보기
/// </summary>
/// <param name="direction">바라볼 방향</param>
public void LookAt(Vector3 direction)
{
    heroMoveController.LookAt(direction);
}

/// <summary>
/// 특정 타겟 바라보기
/// </summary>
/// <param name="target">바라볼 타겟 트랜스폼</param>
public void LookAt(Transform target)
{
    heroMoveController.LookAt(target);
}

```

자세 변경 처리

```

/// <summary>
/// 자세 변경 시 처리
/// </summary>
public void OnStanceChanged(IHeroStatus.Stance stance)
{
    switch (stance)
    {
        case IHeroStatus.Stance.StandUp:
            NotifyStanceEvents(wakeup ⇒ wakeup?.WakeUp());
            break;
        case IHeroStatus.Stance.Down:
            NotifyStanceEvents(down ⇒ down?.TakeDown());
            break;
        case IHeroStatus.Stance.Stun:
            NotifyStanceEvents(stun ⇒ stun?.Stun());
            break;
    }
}

```

이벤트 통지 시스템

```
/// <summary>
/// 스킬 이벤트 리스너들에게 이벤트 통지
/// </summary>
/// <param name="action">실행할 이벤트 액션</param>
public void NotifySkillEvents(Action<ISkillEvent> action)
{
    foreach (var skillEvent in _skillEvents)
    {
        action?.Invoke(skillEvent);
    }
}

/// <summary>
/// 피격 이벤트 리스너들에게 이벤트 통지
/// </summary>
/// <param name="action">실행할 이벤트 액션</param>
private void NotifyHitEvents(Action<IHitEvent> action)
{
    foreach (var hitEvent in _hitEvents)
    {
        action?.Invoke(hitEvent);
    }
}

/// <summary>
/// 이동 제어 이벤트 리스너들에게 이벤트 통지
/// </summary>
/// <param name="action">실행할 이벤트 액션</param>
private void NotifyMoveControlEvents(Action<IMoveControlEvent> action)
{
    foreach (var moveControlEvent in _moveControlEvents)
    {
        action?.Invoke(moveControlEvent);
    }
}

/// <summary>
/// 공격 이벤트 리스너들에게 이벤트 통지
/// </summary>
/// <param name="action">실행할 이벤트 액션</param>
public void NotifyAttackEvents(Action<IAttackEvent> action)
{
    foreach (var attackEvent in _attackEvents)
    {
        action?.Invoke(attackEvent);
    }
}

/// <summary>
/// 자세 변경 이벤트 리스너들에게 이벤트 통지
/// </summary>
/// <param name="action">실행할 이벤트 액션</param>
```

```

/// <param name="action">실행할 이벤트 액션</param>
private void NotifyStanceEvents(Action<IStanceEvent> action)
{
    foreach (var stanceEvent in _stanceEvents)
    {
        action?.Invoke(stanceEvent);
    }
}

/// <summary>
/// 컷씬 이벤트 리스너들에게 이벤트 통지
/// </summary>
/// <param name="action">실행할 이벤트 액션</param>
public void NotifyCutSceneEvents(Action<ICutSceneEvent> action)
{
    foreach (var cutSceneEvent in _cutSceneEvents)
    {
        action?.Invoke(cutSceneEvent);
    }
}

```

전투 입력 처리

```

/// <summary>
/// 공격 실행
/// </summary>
/// <param name="syncData">입력 동기화 데이터</param>
public void Attack(NetworkHeroController.HeroInputSyncData syncData)
{
    if (_controllerState.CanAttack == false) return;
    _baseHeroAbility?.Attack(syncData);
}

/// <summary>
/// 공격 키 홀드
/// </summary>
/// <param name="syncData">입력 동기화 데이터</param>
public void AttackHold(NetworkHeroController.HeroInputSyncData syncData)
{
    if (_controllerState.CanAttack == false) return;
    _baseHeroAbility?.AttackHold(syncData);
}

/// <summary>
/// 공격 키 홀드 종료
/// </summary>
/// <param name="syncData">입력 동기화 데이터</param>
public void StopAttackHold(NetworkHeroController.HeroInputSyncData syncData)
{
    _baseHeroAbility?.AttackHoldFinished(syncData);
}

/// <summary>
/// 가드 실행

```

```

/// <summary>
/// </summary>
/// <param name="syncData">입력 동기화 데이터</param>
public void Guard(NetworkHeroController.HeroInputSyncData syncData)
{
    if (_controllerState.CanGuard == false) return;
    _baseHeroAbility?.Guard(syncData);
}

/// <summary>
/// 가드 종료
/// </summary>
/// <param name="syncData">입력 동기화 데이터</param>
public void StopGuard(NetworkHeroController.HeroInputSyncData syncData)
{
    _baseHeroAbility?.StopGuard(syncData);
}

/// <summary>
/// 궁극기 실행
/// </summary>
/// <param name="syncData">입력 동기화 데이터</param>
public void UltimateSkill(NetworkHeroController.HeroInputSyncData syncData)
{
    _baseHeroAbility?.UltimateSkill(syncData);
}

/// <summary>
/// 공격+가드 동시 입력 처리
/// </summary>
/// <param name="syncData">입력 동기화 데이터</param>
public void AttackGuardDualInput(NetworkHeroController.HeroInputSyncData syncData)
{
    _baseHeroAbility?.AttackGuardDualInput(syncData);
}

/// <summary>
/// 대시 (방향 벡터 기준)
/// </summary>
/// <param name="vertical">수직 입력값</param>
/// <param name="horizontal">수평 입력값</param>
public void Dash(float vertical, float horizontal)
{
    if (_controllerState.CanDash == false) return;
    var direction = new Vector2(horizontal, vertical);
    _baseHeroAbility?.Dash(direction);
}

```

기능 설명

이동 시스템

- 이동 가능상태를 체크하고 입력 방향으로 이동하며, 상황에 따라 이벤트를 발생
- 이동 처리는 BaseHeroMoveController에 위임

이벤트 시스템

- HeroController는 6가지 이벤트 타입을 관리하는 옵서버 패턴을 구현

이벤트 타입	인터페이스	용도
MoveControl	<code>IMoveControlEvent</code>	이동 입력 및 제어 상태 변화
Hit	<code>IHitEvent</code>	피격 수신
Skill	<code>ISkillEvent</code>	스킬 사용 및 상태 변화
Attack	<code>IAttackEvent</code>	공격 실행 및 완료
Stance	<code>IStanceEvent</code>	자세(StandUp/Down/Stun) 변경
CutScene	<code>ICutSceneEvent</code>	컷씬 시작/종료

- 각 이벤트 타입은 `Register()/UnRegister()` 메서드로 구독/해제할 수 있으며, `NotifyXXXEvents()` 메서드로 모든 구독자에게 이벤트 통지

상태 기반 행동 제어

- `IControllerState`를 통해 현재 캐릭터가 어떤 행동을 할 수 있는지 제어
- 전투 액션(Attack, Guard, Dash 등) 실행 전 해당 상태를 체크하여 부적절한 상태에서는 실행되지 않도록 제어.
- `CanMove`: 이동 가능 여부
- `CanAttack`: 공격 가능 여부
- `CanGuard`: 가드 가능 여부
- `CanDash`: 대시 가능 여부
- `CanControllable`: 전반적인 제어 가능 여부

의존성/상속 관계

- `MonoBehaviour`를 상속 받음
- 인터페이스 구현
 - `IHeroController` 인터페이스 구현
 - `IHeroDetector` 인터페이스 구현
 - `ISkillEventInvoker` 인터페이스 구현
 - `ICutSceneEventInvoker` 인터페이스 구현
- 관리되는 이벤트
 - `IMoveControlEvent`
 - `IHitEvent`
 - `ISkillEvent`
 - `IAttackEvent`
 - `IStanceEvent`


```

        heroEvent.OnAbilityChanged(abilityType);
        heroEvent.OnModeChanged(IHeroEvent.HeroMode.Standard);
    }

    // 이펙트 컨트롤러 설정 코루틴 시작
    if (_setupEffectController  $\neq$  null) StopCoroutine(_setupEffectController);
    _setupEffectController = StartCoroutine(CoSetupEffectController());
}

```

NetworkHeroObject에서 이벤트 리스너 등록/해제

```

//이벤트 리스너 등록
public void Register(ISkillEvent skillEvent)
{
    heroController.Register(skillEvent);
}

public void Register(IAttackEvent attackEvent)
{
    heroController.Register(attackEvent);
}

public void Register(ICutSceneEvent cutSceneEvent)
{
    heroController.Register(cutSceneEvent);
}

//이벤트 리스너 해제
public void UnRegister(ISkillEvent skillEvent)
{
    heroController.UnRegister(skillEvent);
}

public void UnRegister(IAttackEvent attackEvent)
{
    heroController.UnRegister(attackEvent);
}

public void UnRegister(ICutSceneEvent cutSceneEvent)
{
    heroController.UnRegister(cutSceneEvent);
}

// NetworkHeroObject가 디스폰시에도 이벤트 리스너 해제
protected override void OnDestroyed(NetworkRunner runner, bool hasState)
{
    // 설정 코루틴 중지
    if (_setupCoroutine  $\neq$  null)
    {
        StopCoroutine(_setupCoroutine);
        _setupCoroutine = null;
    }
}

```

```

// 모든 이벤트 등록 해제
UnRegister((IMatchEvent)heroAnimationController);
heroController.UnRegister(heroAnimationController.MoveControlEvent);
heroController.UnRegister(heroAnimationController.HitEvent);
heroController.UnRegister(heroAnimationController.StanceEvent);
heroController.UnRegister(_baseHeroAbility.HitEvent);
heroController.UnRegister(_baseHeroAbility.MoveControlEvent);
heroController.UnRegister(_baseHeroAbility.StanceEvent);
heroController.UnRegister((IAttackEvent)this);
heroController.UnRegister(_baseHeroAbility.SkillEvent);

// 로컬 플레이어인 경우 사망 이벤트 해제
if (IsRemote() == false)
{
    networkHeroStatus.OnDeath -= OnDeathEvent;
}

// 이벤트 및 참조 정리
networkHeroStatus.OnHpChanged -= heroAnimationController.OnHealthChange;
heroController.DetectCloseHeroEvent = null;
_heroEvents.Clear();

// 디스폰 알림
HeroObjectEvent.Instance.NotifyDeSpawned(this);
}

```

NetworkHeroController에서 입력 전달

```

private HeroController heroController;

[Rpc(RpcSources.InputAuthority, RpcTargets.All)]
private void RPC_UpdateHeroInputData(HeroInputData heroInputData)
{
    // 이동 입력 처리
    heroController.Move(heroInputData.Horizontal, heroInputData.Vertical, Runn

    // 개별 스킬/액션 입력 처리
    ProcessDash(heroInputData);
    ProcessAttack(heroInputData);
    ProcessAttackHold(heroInputData);
    ProcessFocus(heroInputData);
    ProcessGuard(heroInputData);
    ProcessUltimateSkill(heroInputData);
    ProcessAttackGuardDualInput(heroInputData);
}

private void ProcessAttack(HeroInputData inputData)
{
    if (inputData.IsAttack())
    {
        // 클라이언트 타임스탬프와 서버 시뮬레이션 타임으로 동기화 데이터 생성
        var syncData = new HeroInputSyncData(inputData.AttackTimeStamp, Runner
        heroController.Attack(syncData);
    }
}

```

```

    }
}

private void ProcessAttackHold(HeroInputData inputData)
{
    var syncData = new HeroInputSyncData(inputData.AttackTimeStamp, Runner.Sim
    if (inputData.IsAttackHold())
    {
        heroController.AttackHold(syncData);
    }
    else
    {
        heroController.StopAttackHold(syncData);
    }
}

private void ProcessDash(HeroInputData inputData)
{
    if (inputData.IsDash())
    {
        heroController.Dash(inputData.Horizontal, inputData.Vertical);
    }
}

private void ProcessGuard(HeroInputData inputData)
{
    var syncData = new HeroInputSyncData(inputData.AttackTimeStamp, Runner.Sim
    if (inputData.IsGuard())
    {
        heroController.Guard(syncData);
    }
    else
    {
        heroController.StopGuard(syncData);
    }
}

private void ProcessFocus(HeroInputData heroInputData)
{
    if (heroInputData.IsFocusMode())
    {
        // 현재 포커스 상태의 반대로 토글
        heroController.ToggleFocusMode(!heroStatus.GetFocusState());
    }
}

private void ProcessUltimateSkill(HeroInputData inputData)
{
    if (inputData.IsUltimateSkill())
    {
        var syncData = new HeroInputSyncData(inputData.AttackTimeStamp, Runner
        heroController.UltimateSkill(syncData);
    }
}

private void ProcessAttackGuardDualInput(HeroInputData inputData)

```

```

{
    if (inputData.IsAttackGuardDualInput())
    {
        var syncData = new HeroInputSyncData(inputData.AttackTimeStamp, Runner
        heroController.AttackGuardDualInput(syncData);
    }
}

```

NetworkHeroStatus에서 피격 및 자세 변경시 호출

```

[Rpc(RpcSources.All, RpcTargets.All)]
private void RPC_UpdateHitInfo(IHitEvent.HitInfo hitInfo)
{
    heroController.OnHitReceive(hitInfo);
    UpdateStance(hitInfo.HitterInfo);
    UpdateStiffness(hitInfo);
}

private void UpdateStance(IHitEvent.HitInfo.HitterState hitterState)
{
    if (CurrentStance == hitterState.HitterStance) return;
    ChangeStance(hitterState.HitterStance);
    heroController.OnStanceChanged(hitterState.HitterStance);
}

```

관련 클래스

- IHeroController
- BaseHeroMoveController
- HeroRigidbodyController
- IControllerState
- HeroControllerStateHandler
- NetworkHeroController
- BaseHeroAbility
- IHeroStatus
- IMoveControlEvent
- IHitEvent
- ISkillEvent
- IAttackEvent
- IStanceEvent
- ICutSceneEvent

IHeroInput

영웅 입력 인터페이스

개요

IHeroInput 인터페이스는 RFist 게임의 영웅 입력 계약을 정의합니다. 이동, 전투, 포커스 등 다양한 입력 메서드를 제공하며, **BaseHeroInput** 추상 클래스가 이 인터페이스를 구현합니다.

역할

- 영웅 캐릭터의 입력 메서드 계약 정의
- 이동/전투/포커스 입력값 제공
- 입력 시스템의 표준 인터페이스

선언

```
public interface IHeroInput
```

멤버

메서드

```
/// <summary>
/// 수평 입력값 가져오기 (좌/우)
/// </summary>
/// <returns>수평 입력값 (-1 ~ 1)</returns>
public float GetHorizontal();

/// <summary>
/// 수직 입력값 가져오기 (앞/뒤)
/// </summary>
/// <returns>수직 입력값 (-1 ~ 1)</returns>
public float GetVertical();

/// <summary>
/// 공격 입력 여부
/// </summary>
/// <returns>공격 입력 여부</returns>
public bool IsAttack();
```

```

/// <summary>
/// 공격 키 홀드 여부
/// </summary>
/// <returns>공격 키 홀드 여부</returns>
public bool IsAttackHold();

/// <summary>
/// 가드 입력 여부
/// </summary>
/// <returns>가드 입력 여부</returns>
public bool IsGuard();

/// <summary>
/// 대시 입력 여부
/// </summary>
/// <returns>대시 입력 여부</returns>
public bool IsDash();

/// <summary>
/// 궁극기 입력 여부
/// </summary>
/// <returns>궁극기 입력 여부</returns>
public bool IsUltimateAttack();

/// <summary>
/// 공격+가드 동시 입력 여부
/// </summary>
/// <returns>공격+가드 동시 입력 여부</returns>
public bool IsAttackGuardDualInput();

```

기능 설명

입력 카테고리

카테고리	메서드	설명
이동	GetHorizontal, GetVertical	좌/우, 앞/뒤 입력값 (-1 ~ 1)
전투	IsAttack, IsGuard, IsDash, IsUltimateAttack	공격/가드/대시/궁극기 입력 여부
특수	IsAttackGuardDualInput	공격+가드 동시 입력 여부

입력값 반환 타입

- float (-1 ~ 1): GetHorizontal, GetVertical - 축 입력값
- bool: IsAttack, IsGuard 등 - 버튼 입력 여부

의존성/상속 관계

구현 클래스

- `BaseHeroInput` 추상 클래스에서 구현
- `HeroKeyInput`에서 키보드 입력 구현
- `NetworkHeroInput`에서 입력소스 수집

사용 예시

`NetworkHeroInput`에서 입력 소스 수집시 사용

```
// BaseHeroInput.cs
public static IReadOnlyList<IHeroInput> HeroInputList => HeroInputs;

// NetworkHeroInput.cs
public override void OnInput(NetworkRunner runner, NetworkInput input)
{
    // 입력 권한이 없으면 처리하지 않음
    if (!Object.HasInputAuthority) return;

    // 새로운 입력 데이터 생성
    var heroInputData = new HeroInputData();

    // 등록된 모든 입력 소스에서 입력 수집
    foreach (var heroInput in BaseHeroInput.HeroInputList)
    {
        // 수평 이동 입력 수집 (이미 값이 있으면 건드리지 않음)
        if (heroInputData.Horizontal == 0)
        {
            heroInputData.Horizontal = heroInput.GetHorizontal();
        }

        // 수직 이동 입력 수집 (이미 값이 있으면 건드리지 않음)
        if (heroInputData.Vertical == 0)
        {
            heroInputData.Vertical = heroInput.GetVertical();
        }

        // 대시 입력 수집
        if (heroInput.IsDash())
        {
            heroInputData.Buttons |= HeroInputData.Dash;
        }

        // 공격 입력 수집 및 타임스탬프 기록
        if (heroInput.IsAttack())
        {
            heroInputData.Buttons |= HeroInputData.Attack;
            heroInputData.AttackTimeStamp = runner.SimulationTime;
        }

        // 공격 홀드(차지) 입력 수집 및 타임스탬프 기록
        if (heroInput.IsAttackHold())
        {
            heroInputData.Buttons |= HeroInputData.AttackHold;
        }
    }
}
```

```

        heroInputData.AttackTimeStamp = runner.SimulationTime;
    }
    else
    {
        // 홀드 해제 시에도 타임스탬프 갱신
        heroInputData.AttackTimeStamp = runner.SimulationTime;
    }

    // 가드 입력 수집 및 타임스탬프 기록
    if (heroInput.IsGuard())
    {
        heroInputData.Buttons |= HeroInputData.Guard;
        heroInputData.AttackTimeStamp = runner.SimulationTime;
    }

    // 포커스 모드 입력 수집
    if (heroInput.IsFocusMode())
    {
        heroInputData.Buttons |= HeroInputData.FocusMode;
    }

    // 포커스 변경 입력 수집
    if (heroInput.IsFocusChange())
    {
        heroInputData.Buttons |= HeroInputData.FocusChange;
    }

    // 궁극기 입력 수집 및 타임스탬프 기록
    if (heroInput.IsUltimateAttack())
    {
        heroInputData.Buttons |= HeroInputData.UltimateAttack;
        heroInputData.AttackTimeStamp = runner.SimulationTime;
    }

    // 공격+가드 동시 입력 수집 및 타임스탬프 기록
    if (heroInput.IsAttackGuardDualInput())
    {
        heroInputData.Buttons |= HeroInputData.AttackGuardDualInput;
        heroInputData.AttackTimeStamp = runner.SimulationTime;
    }
}

// 수집된 입력 데이터를 네트워크 입력으로 설정
input.Set(heroInputData);
}

```

관련 클래스

- [BaseHeroInput](#)
- [HeroKeyInput](#)
- [NetworkHeroController](#)

BaseHeroInput

영웅 입력의 기반 추상 클래스

개요

BaseHeroInput 클래스는 RFist 게임의 영웅 입력 시스템의 기반 추상 클래스입니다. 키보드, 게임패드, 모바일 등 다양한 입력 방식의 공통 인터페이스를 제공하며, 템플릿 메서드 패턴을 통해 구체적인 입력 로직은 하위 클래스에서 구현하도록 합니다.

역할

- 다양한 입력 방식(키보드, 게임패드, 모바일)의 공통 인터페이스 제공
- 이동, 전투, 가드, 대시 등의 입력 메서드 정의
- 입력 인스턴스의 중앙 관리 (static 리스트)

선언

```
public abstract class BaseHeroInput : MonoBehaviour, IHeroInput, IHeroEvent
```

멤버

속성

```
/// <summary>
/// 모든 영웅 입력 인스턴스 목록
/// </summary>
private static readonly List<BaseHeroInput> HeroInputs = new List<BaseHeroInput>()

/// <summary>
/// 영웅 입력 리스트 (읽기 전용)
/// </summary>
public static IReadOnlyList<IHeroInput> HeroInputList => HeroInputs;
```

메서드

```
/// <summary>
/// 활성화 시 추가 처리 (하위 클래스에서 재정의)
```

```
/// </summary>
protected virtual void OnEnabled()

/// <summary>
/// 비활성화 시 추가 처리 (하위 클래스에서 재정의)
/// </summary>
protected virtual void OnDisabled()

/// <summary>
/// 수평 입력값 가져오기 (좌/우)
/// </summary>
/// <returns>수평 입력값 (-1 ~ 1)</returns>
public abstract float GetHorizontal();

/// <summary>
/// 수직 입력값 가져오기 (앞/뒤)
/// </summary>
/// <returns>수직 입력값 (-1 ~ 1)</returns>
public abstract float GetVertical();

/// <summary>
/// 공격 입력 여부
/// </summary>
/// <returns>공격 입력 여부</returns>
public abstract bool IsAttack();

/// <summary>
/// 공격 키 홀드 여부
/// </summary>
/// <returns>공격 키 홀드 여부</returns>
public abstract bool IsAttackHold();

/// <summary>
/// 가드 입력 여부
/// </summary>
/// <returns>가드 입력 여부</returns>
public abstract bool IsGuard();

/// <summary>
/// 대시 입력 여부
/// </summary>
/// <returns>대시 입력 여부</returns>
public abstract bool IsDash();

/// <summary>
/// 궁극기 입력 여부
/// </summary>
/// <returns>궁극기 입력 여부</returns>
public abstract bool IsUltimateAttack();

/// <summary>
/// 공격+가드 동시 입력 여부
/// </summary>
/// <returns>공격+가드 동시 입력 여부</returns>
public abstract bool IsAttackGuardDualInput();
```

코드 스니펫

컴포넌트 활성화/비활성화시 정적 리스트 `HeroInputList` 에 인스턴스 등록/해제

```
/// <summary>
/// 컴포넌트 활성화 시 호출
/// </summary>
private void OnEnable()
{
    heroSystem.RegisterHeroEvent(this);
    HeroInputs.Add(this);
    OnEnabled();
    heroSystem.OnHeroSpawned += OnHeroSpawned;
}

/// <summary>
/// 컴포넌트 비활성화 시 호출
/// </summary>
private void OnDisable()
{
    heroSystem.UnregisterHeroEvent(this);
    HeroInputs.Remove(this);
    OnDisabled();
    heroSystem.OnHeroSpawned -= OnHeroSpawned;
}
```

기능 설명

입력 카테고리

카테고리	메서드	설명
이동	<code>GetHorizontal</code> , <code>GetVertical</code>	좌/우, 앞/뒤 입력값 (-1 ~ 1)
전투	<code>IsAttack</code> , <code>IsGuard</code> , <code>IsDash</code> , <code>IsUltimateAttack</code>	공격/가드/대시/궁극기 입력 여부
특수	<code>IsAttackGuardDualInput</code>	공격+가드 동시 입력 여부

정적 인스턴스 관리

- 모든 `BaseHeroInput` 인스턴스는 정적리스트 `HeroInputs` 에 자동 등록되어 중앙에서 관리
- `HeroInputList`: 읽기 전용으로 외부에서 접근 가능
- 활성화 시 자동 등록, 비활성화 시 자동 제거

의존성/상속 관계

- `MonoBehaviour`를 상속받음
- 구현 클래스
 - `HeroKeyInput` 클래스에서 키보드 입력 구현
- `HeroSystem` 클래스에서 영웅 관련 이벤트 구독
- `IHeroInput` 인터페이스 구현
- `IHeroEvent` 인터페이스 구현
- `NetworkHeroInput` 클래스에서 정적 리스트의 데이터 사용

사용 예시

`NetworkHeroInput`에서 정적 리스트 `HeroInputList`를 통해 모든 인풋 데이터 로드

```
public override void OnInput(NetworkRunner runner, NetworkInput input)
{
    // 입력 권한이 없으면 처리하지 않음
    if (!Object.HasInputAuthority) return;

    // 새로운 입력 데이터 생성
    var heroInputData = new HeroInputData();

    // 등록된 모든 입력 소스에서 입력 수집
    foreach (var heroInput in BaseHeroInput.HeroInputList)
    {
        // 수평 이동 입력 수집 (이미 값이 있으면 건드리지 않음)
        if (heroInputData.Horizontal == 0)
        {
            heroInputData.Horizontal = heroInput.GetHorizontal();
        }

        // 수직 이동 입력 수집 (이미 값이 있으면 건드리지 않음)
        if (heroInputData.Vertical == 0)
        {
            heroInputData.Vertical = heroInput.GetVertical();
        }

        // 대시 입력 수집
        if (heroInput.IsDash())
        {
            heroInputData.Buttons |= HeroInputData.Dash;
        }

        // 공격 입력 수집 및 타임스탬프 기록
        if (heroInput.IsAttack())
        {
            heroInputData.Buttons |= HeroInputData.Attack;
            heroInputData.AttackTimeStamp = runner.SimulationTime;
        }

        // 공격 홀드(차지) 입력 수집 및 타임스탬프 기록
        if (heroInput.IsAttackHold())
        {

```

```

        heroInputData.Buttons |= HeroInputData.AttackHold;
        heroInputData.AttackTimeStamp = runner.SimulationTime;
    }
    else
    {
        // 홀드 해제 시에도 타임스탬프 갱신
        heroInputData.AttackTimeStamp = runner.SimulationTime;
    }

    // 가드 입력 수집 및 타임스탬프 기록
    if (heroInput.IsGuard())
    {
        heroInputData.Buttons |= HeroInputData.Guard;
        heroInputData.AttackTimeStamp = runner.SimulationTime;
    }

    // 포커스 모드 입력 수집
    if (heroInput.IsFocusMode())
    {
        heroInputData.Buttons |= HeroInputData.FocusMode;
    }

    // 포커스 변경 입력 수집
    if (heroInput.IsFocusChange())
    {
        heroInputData.Buttons |= HeroInputData.FocusChange;
    }

    // 궁극기 입력 수집 및 타임스탬프 기록
    if (heroInput.IsUltimateAttack())
    {
        heroInputData.Buttons |= HeroInputData.UltimateAttack;
        heroInputData.AttackTimeStamp = runner.SimulationTime;
    }

    // 공격+가드 동시 입력 수집 및 타임스탬프 기록
    if (heroInput.IsAttackGuardDualInput())
    {
        heroInputData.Buttons |= HeroInputData.AttackGuardDualInput;
        heroInputData.AttackTimeStamp = runner.SimulationTime;
    }
}

// 수집된 입력 데이터를 네트워크 입력으로 설정
input.Set(heroInputData);
}

```

관련 클래스

- [IHeroInput](#)
- [HeroKeyInput](#)
- [NetworkHeroInput](#)

HeroKeyInput

키보드 입력을 처리하는 입력 컴포넌트

개요

HeroKeyInput 클래스는 RFist 게임의 키보드 입력을 처리하는 입력 컴포넌트입니다. [Input System](#) 을 사용하여 키보드 및 마우스 입력을 영웅 동작으로 변환하며, **BaseHeroInput** 추상 클래스를 상속받아 구현됩니다.

역할

- [Input System](#) 을 활용한 키보드/마우스 입력 처리
- 이동, 전투, 포커스 등의 입력 상태 관리
- 단발성 입력과 지속성 입력(홀드) 구분

선언

```
public class HeroKeyInput : BaseHeroInput
```

멤버

속성

```
/// <summary>
/// Input System 액션 에셋
/// </summary>
private RFistActions _inputActions;

/// <summary>
/// 공격 입력 플래그
/// </summary>
private bool _isAttack;

/// <summary>
/// 공격 홀드 입력 플래그
/// </summary>
private bool _isAttackHold;

/// <summary>
```

```

/// 가드 입력 플래그
/// </summary>
private bool _isGuard;

/// <summary>
/// 대시 입력 플래그
/// </summary>
private bool _isDash;

/// <summary>
/// 포커스 모드 입력 플래그
/// </summary>
private bool _isFocusMode;

/// <summary>
/// 포커스 변경 입력 플래그
/// </summary>
private bool _isFocusChange;

/// <summary>
/// 궁극기 입력 플래그
/// </summary>
private bool _isUltimateAttack;

/// <summary>
/// 공격+가드 동시 입력 플래그
/// </summary>
private bool _isAttackGuardDualInput;

/// <summary>
/// 홀드 입력에 필요한 최소 시간 (초)
/// </summary>
private const float HoldRequiredTime = 0.3f;

```

메서드

```

/// <summary>
/// 초기화 - Input System 액션 생성
/// </summary>
private void Awake()

/// <summary>
/// 활성화 시 Input System 이벤트 구독
/// </summary>
protected override void OnEnabled()

/// <summary>
/// 비활성화 시 Input System 이벤트 구독 해제
/// </summary>
protected override void OnDisabled()

/// <summary>
/// 수평 입력값 가져오기
/// </summary>

```

```

/// </summary>
/// <returns>수평 입력값 (-1 ~ 1)</returns>
public override float GetHorizontal()

/// <summary>
/// 수직 입력값 가져오기
/// </summary>
/// <returns>수직 입력값 (-1 ~ 1)</returns>
public override float GetVertical()

/// <summary>
/// 공격 입력 시 호출
/// </summary>
/// <param name="obj">InputAction 콜백 컨텍스트</param>
private void OnAttackPerformed(InputAction.CallbackContext obj)

/// <summary>
/// 공격 입력 해제 시 호출
/// </summary>
/// <param name="obj">InputAction 콜백 컨텍스트</param>
private void OnAttackReleased(InputAction.CallbackContext obj)

/// <summary>
/// 가드 입력 시 호출
/// </summary>
/// <param name="obj">InputAction 콜백 컨텍스트</param>
private void OnGuardPerformed(InputAction.CallbackContext obj)

/// <summary>
/// 가드 입력 해제 시 호출
/// </summary>
/// <param name="obj">InputAction 콜백 컨텍스트</param>
private void OnGuardReleased(InputAction.CallbackContext obj)

/// <summary>
/// 대시 입력 시 호출
/// </summary>
/// <param name="obj">InputAction 콜백 컨텍스트</param>
private void OnDashPerformed(InputAction.CallbackContext obj)

/// <summary>
/// 궁극기 입력 시 호출
/// </summary>
/// <param name="obj">InputAction 콜백 컨텍스트</param>
private void OnUltimateAttack(InputAction.CallbackContext obj)

/// <summary>
/// 공격+가드 동시 입력 시 호출
/// </summary>
/// <param name="obj">InputAction 콜백 컨텍스트</param>
private void OnAttackGuardDualInput(InputAction.CallbackContext obj)

/// <summary>
/// 공격 입력 여부 (한 프레임만 true 반환 후 자동 리셋)
/// </summary>
/// <returns>공격 입력 여부</returns>

```

```

public override bool IsAttack()

    /// <summary>
    /// 공격 키 홀드 여부
    /// </summary>
    /// <returns>공격 키 홀드 여부</returns>
public override bool IsAttackHold()

    /// <summary>
    /// 가드 입력 여부
    /// </summary>
    /// <returns>가드 입력 여부</returns>
public override bool IsGuard()

    /// <summary>
    /// 대시 입력 여부 (한 프레임만 true 반환 후 자동 리셋)
    /// </summary>
    /// <returns>대시 입력 여부</returns>
public override bool IsDash()

    /// <summary>
    /// 포커스 모드 입력 여부 (한 프레임만 true 반환 후 자동 리셋)
    /// </summary>
    /// <returns>포커스 모드 입력 여부</returns>
public override bool IsFocusMode()

    /// <summary>
    /// 포커스 변경 입력 여부 (한 프레임만 true 반환 후 자동 리셋)
    /// </summary>
    /// <returns>포커스 변경 입력 여부</returns>
public override bool IsFocusChange()

    /// <summary>
    /// 궁극기 입력 여부 (한 프레임만 true 반환 후 자동 리셋)
    /// </summary>
    /// <returns>궁극기 입력 여부</returns>
public override bool IsUltimateAttack()

    /// <summary>
    /// 공격+가드 동시 입력 여부 (한 프레임만 true 반환 후 자동 리셋)
    /// </summary>
    /// <returns>공격+가드 동시 입력 여부</returns>
public override bool IsAttackGuardDualInput()

```

코드 스니펫

공격 입력 처리 (홀드 구분)

```

    /// <summary>
    /// 공격 입력 시 호출
    /// </summary>
    private void OnAttackPerformed(InputAction.CallbackContext obj)
    {

```

```

    // 홀드 인터랙션인지 확인
    if (obj.interaction is HoldInteraction)
    {
        _isAttackHold = true;
    }
    else
    {
        _isAttack = true;
    }
}

```

단발성 입력 리셋 패턴

```

/// <summary>
/// 공격 입력 여부 (한 프레임만 true 반환 후 자동 리셋)
/// </summary>
public override bool IsAttack()
{
    try
    {
        return _isAttack;
    }
    finally
    {
        _isAttack = false;
    }
}

```

이동 입력값 읽기

```

/// <summary>
/// 수평 입력값 가져오기 (A/D 또는 화살표 좌/우)
/// </summary>
public override float GetHorizontal()
{
    return _inputActions?.Hero.Move.ReadValue<Vector2>().x ?? 0f;
}

```

기능 설명

입력 동작 분류

동작	입력 타입	리셋 방식	Input Action
이동	축 입력 (연속)	없음	Move (Vector2)
공격	버튼 (단발)	자동 리셋	Attack (Tap)
공격 홀드	버튼 (지속)	수동 리셋	Attack (Hold)

이름	입력 방식	리셋 방식	호출 방법
가드	버튼 (지속)	수동 리셋	<code>Guard</code> (Press)
대시	버튼 (단발)	자동 리셋	<code>Dash</code> (Press)
궁극기	버튼 (단발)	자동 리셋	<code>UltimateAttack</code> (Press)
공격+가드	버튼 (단발)	자동 리셋	<code>AttackGuardDualInput</code> (Press)

단발성 vs 지속성 입력

단발성 입력 (`IsAttack`, `IsDash` 등):

- 한 프레임만 `true` 반환
- try-finally 패턴으로 자동 리셋
- 연속 입력 방지에 사용

지속성 입력 (`IsGuard`, `IsAttackHold` 등):

- 버튼이 눌러있는 동안 계속 `true`
- 수동으로 `false`로 리셋
- 홀드 동작에 사용

의존성/상속 관계

- `MonoBehaviour`를 상속받음
- `BaseHeroInput`를 상속 받음
- `IHeroInput` 인터페이스 간접 구현(`BaseHeroInput`을 통해 호출됨)
- `RFistActions` 클래스 생성 및 구독(`Input System` [🔗](#) 에셋)

사용 예시

해당 클래스 호출은 상위클래스 `BaseHeroInput`를 통해 호출됨

관련 클래스

- `BaseHeroInput`
- `IHeroInput`
- `HeroController`
- `NetworkHeroController`
- `HeroSystem`

IHeroController

영웅 캐릭터의 이동 및 제어 기능을 정의하는 인터페이스

개요

IHeroController 인터페이스는 RFist 게임의 영웅 캐릭터 이동 및 제어 기능을 정의합니다. DoTween 기반의 물리법칙 없는 이동, 넉백, 대시, 타겟팅 등의 핵심 기능을 제공하며, **HeroController** 클래스가 이 인터페이스를 구현합니다.

역할

- 영웅 캐릭터의 이동 기능 계약 정의
- DoTween 기반 절대/상대 이동 인터페이스 제공
- 넉백 및 대시 기능 표준화
- 자동 타겟팅 및 방향 전환 기능 정의
- 동작 중인 Tween 기반 동작 취소 기능 정의

선언

```
public interface IHeroController
```

멤버

메서드

```
/// <summary>
/// DoTween을 이용해 캐릭터를 물리법칙 없이 지정된 방향으로 절대 이동
/// </summary>
/// <param name="distance">이동 거리. DoMove(forward * distance, time)에서 사용</param>
/// <param name="time">이동 시간(초). DoMove(forward * distance, time)에서 사용</param>
/// <param name="direction">이동할 방향 벡터</param>
public void MoveToAbsoluteDirection(float distance, float time, Vector3 direction)

/// <summary>
/// DoTween을 이용해 캐릭터를 물리법칙 없이 상대 방향으로 이동
/// </summary>
/// <param name="distance">이동 거리</param>
/// <param name="time">이동 시간(초)</param>
/// <param name="direction">상대 방향 벡터</param>
```



```

/// <param name="direction">경로 방향 벡터</param>
public void MoveToRelativeDirection(float distance, float time, Vector3 direction);

/// <summary>
/// DoTween을 사용하여 넉백 처리
/// </summary>
/// <param name="attackerPos">공격자 위치. 넉백은 플레이어 기준 공격자의 반대 방향으로 발생
/// <param name="force">적용할 넉백 힘</param>
public void KnockBack(Vector3 attackerPos, float force);

/// <summary>
/// 대시 이동 실행
/// </summary>
/// <param name="distance">대시 거리</param>
/// <param name="time">대시 시간(초)</param>
/// <param name="direction">대시 방향</param>
public void Dash(float distance, float time, Vector3 direction);

/// <summary>
/// 현재 진행 중인 이동 동작 취소
/// </summary>
public void OnCancel();

/// <summary>
/// Rigidbody의 현재 회전값 가져오기
/// </summary>
/// <returns>현재 회전값(Quaternion)</returns>
public Quaternion GetRigidbodyRotation();

/// <summary>
/// 주변에서 가장 가까운 타겟을 찾아 바라보기
/// </summary>
/// <param name="autoTargetingDistance">자동 타겟팅 최대 거리</param>
/// <param name="autoTargetingAngle">자동 타겟팅 각도 범위</param>
public void FindAndLookTarget(float autoTargetingDistance, int autoTargetingAngle);

/// <summary>
/// 주변에서 가장 가까운 타겟 찾기
/// </summary>
/// <param name="autoTargetingDistance">타겟 감지 거리</param>
/// <param name="autoTargetingAngle">타겟 감지 각도 범위</param>
/// <param name="targetTransform">찾은 타겟의 Transform (출력 파라미터)</param>
/// <returns>타겟 찾기 성공 여부</returns>
public bool FindTarget(float autoTargetingDistance, int autoTargetingAngle, out Transform targetTransform);

/// <summary>
/// 특정 Transform을 바라볼도록 회전
/// </summary>
/// <param name="targetPosition">바라볼 대상의 Transform</param>
public void LookAt(Transform targetPosition);

/// <summary>
/// 특정 방향을 바라볼도록 회전
/// </summary>
/// <param name="targetDirection">바라볼 방향 벡터</param>
public void LookAt(Vector3 targetDirection);

```

기능 설명

이동 기능 정의

- 절대 이동 (**MoveToAbsoluteDirection**): 월드 좌표계 기준으로 지정된 방향으로 이동
- 상대 이동 (**MoveToRelativeDirection**): 캐릭터의 현재 방향 기준으로 상대적인 방향으로 이동
- 넉백(**KnockBack**): 공격자 기준 반대 방향으로 이동

회전 기능 정의

- 타겟 서칭(**FindTarget**): 조건(거리, 각도)에 맞는 타겟을 찾아 반환
- 타겟 방향으로 회전(**FindAndLookTarget**): 타겟을 찾아 즉시 바라보기
- 특정 방향으로 회전(**LookAt**): 특정 위치나 방향을 바라볼 수 있도록 회전

의존성/상속 관계

- 구현 클래스
 - HeroController** **IHeroController**의 구현체로, 모든 메서드를 구현
- NetworkHeroController**에서 **HeroController**를 통해 간접적으로 사용
- BaseHeroAbility**를 상속받는 클래스들이 피격, 넉백과 관련된 제어가 필요할 때 사용
- BaseHeroSkill**를 상속받는 클래스들이 스킬과 관련된 제어가 필요할 때 사용

사용 예시

BaseHeroAbility를 상속한 캐릭터 어빌리티 클래스에서 넉백 이동에 사용

```
// MaiAbility.cs
private IHeroController _heroController;

protected override void OnHit(IHitEvent.HitInfo hitInfo)
{
    if (hitInfo.HitterInfo.InvincibleState)
    {
        PlayTrailEffect();
        return;
    }

    if (hitInfo.HitterInfo.GuardState)
    {
        OnGuard(hitInfo);
        SetupStiffnessRoutine();
    }
    else
    {
        var hitEffect = GetHitEffectType(hitInfo);
        var effect = FindEffect(hitEffect);
        if (effect is not null)
```

```

        {
            PlayEffect(effect, hitInfo);
            PlaySoundEffect(effect);
        }

        var spPoint = _heroStatus.GetSp().Current;
        _heroStatus.ChangeSpecialPoint(spPoint + hitInfo.Damage);
        _heroStatus.ApplyDamage(hitInfo.Damage);
        SetupStiffnessRoutine();
        _skillManager.Hit(hitInfo);
    }

    if (_heroObjectInfo.IsRemote()) return;
    _heroController.KnockBack(hitInfo.AttackerPos, hitInfo.KnockBackPower);
}

```

BaseSkillManager를 상속한 캐릭터 스킬매니저 클래스에서 공격 대상 타겟팅시 사용

```

//MaiSkillManager.cs
protected IHeroController HeroController;

private void OnAttackStart(BaseHeroSkill.SkillStartResult skillStartResult)
{
    if (MoveInputDir == Vector2.zero)
    {
        if (_currentSkill != null && _currentSkill.AutoTargeting && HeroObjectInfo.IsRemote())
        {
            HeroController.FindAndLookTarget(_currentSkill.AutoTargetingDistance, skillStartResult.TargetPos);
        }
    }
    else
    {
        HeroController.LookAt(new Vector3(MoveInputDir.x, 0, MoveInputDir.y));
    }
    SkillEventInvoker.NotifySkillEvents(skillEvent => skillEvent?.OnSkill(skillStartResult));
    HeroStatus.SetAttackState(true);
}

```

'BaseHeroSkill'를 상속한 캐릭터 스킬 클래스에서 다양한 용도로 사용

```

1. 공격스킬 사용 시 스킬의 이동 동작 수행
// BaseHeroAttackSkill.cs
protected IHeroController HeroController;
/// <summary>
/// 스킬 이동을 수행합니다.
/// </summary>
private void OnSkillMovement()
{
    _canMove = false;
    if (_characterMovement.direction == CharacterMovement.Direction.Forward)
    {
        MoveController.MoveForward(MoveForwardDistance);
    }
    else if (_characterMovement.direction == CharacterMovement.Direction.Backward)
    {
        MoveController.MoveBackward(MoveBackwardDistance);
    }
}

```

```

        HeroController.MoveToRelativeDirection(
            _characterMovement.moveDistance,
            _characterMovement.moveToDistanceTime,
            Vector3.forward);
    }
}

2. 피격, 캔슬등 스킬 취소시 이동 동작 취소
// BaseHeroAttackSkill.cs
protected IHeroController HeroController;

/// <summary>
/// 피격, 혹은 외부적 요인으로 스킬 캔슬시 호출
/// </summary>
protected override void SkillCancellationByExternal()
{
    HeroController.OnCancel();
    var result = new SkillFinishResult(this, SkillFinishResult.Fail.CancellationByExternal, OnFinishSkill?.Invoke(result));
}

/// <summary>
/// 스킬캔슬 등을 통한 스킬 캔슬시 호출
/// </summary>
protected override void SkillCancellationBySelf()
{
    HeroController.OnCancel();
    var result = new SkillFinishResult(this, SkillFinishResult.Fail.SelfCancellation, OnFinishSkill?.Invoke(result));
}

3. 대시(혹은 다른 이동기) 스킬 사용시 이동 동작 호출시 사용
// DashSkill.cs
protected IHeroController HeroController;

private void ExecuteDash(Vector3 direction)
{
    var data = skillData.FirstOrDefault(data => data.SkillId == GetSkillId());
    if (data == null) return;
    var movement = data.GetCharacterMovement();

    HeroController.Dash(movement.moveDistance, movement.moveToDistanceTime, direction);

    PlayDashEffect(data);
}

```

관련 클래스

- [HeroController](#)
- [BaseHeroMoveController](#)
- [HeroRigidbodyController](#)
- [NetworkHeroController](#)
- [BaseHeroAbility](#)

- BaseHeroAbility
- BaseHeroSkill
- BaseSkillManager
- BaseHeroAttackSkill

BaseHeroMoveController

영웅 이동 제어의 기반 추상 클래스

개요

`BaseHeroMoveController` 클래스는 RFist 게임의 영웅 이동 제어를 위한 기반 추상 클래스입니다. Rigidbody 기반(`HeroRigidbodyController`), KCC(`HeroSimpleKccController`) 등 다양한 이동 구현체의 공통 인터페이스를 제공하며, 템플릿 메서드 패턴을 통해 구체적인 이동 로직은 하위 클래스에서 구현하도록 합니다.

역할

- 다양한 이동 구현체(`HeroRigidbodyController`, `HeroSimpleKccController` 등)의 공통 인터페이스 제공
- 이동, 회전, 순간 이동, 넉백 등의 기본 동작 정의
- 템플릿 메서드 패턴을 통한 구현체 확장 지원

선언

```
[DisallowMultipleComponent]
public abstract class BaseHeroMoveController : MonoBehaviour
```

멤버

속성

```
/// <summary>
/// 컨트롤러 상태 인터페이스
/// </summary>
protected IControllerState ControllerState;

/// <summary>
/// 영웅 오브젝트 정보 인터페이스
/// </summary>
protected IHeroObjectInfo HeroObjectInfo;

/// <summary>
/// 영웅 기본 능력치
/// </summary>
```

```
/// </summary>  
protected HeroBaseStat HeroBaseStat;
```

메서드

```
/// <summary>  
/// 일반 이동 처리  
/// </summary>  
/// <param name="deltaTime">델타 타임</param>  
/// <param name="turnDirection">회전 방향 (기본값: default)</param>  
public void Move(float horizontal, float vertical, float deltaTime, Vector2 turnDirection)  
protected abstract void OnMove(Vector2 moveDirection, float deltaTime, Vector2 turnDirection);  
  
/// <summary>  
/// 절대 방향 이동 처리  
/// </summary>  
/// <param name="distance">이동 거리</param>  
/// <param name="time">이동 시간</param>  
/// <param name="direction">이동 방향 (월드 좌표계)</param>  
public void MoveToAbsoluteDirection(float distance, float time, Vector3 direction)  
protected abstract void OnMoveToAbsoluteDirection(float distance, float time, Vector3 direction);  
  
/// <summary>  
/// 상대 방향 이동 처리  
/// </summary>  
/// <param name="distance">이동 거리</param>  
/// <param name="time">이동 시간</param>  
/// <param name="direction">이동 방향 (로컬 좌표계)</param>  
public void MoveToRelativeDirection(float distance, float time, Vector3 direction)  
protected abstract void OnMoveToRelativeDirection(float distance, float time, Vector3 direction);  
  
/// <summary>  
/// 넉백 처리  
/// </summary>  
/// <param name="attackerPos">공격자 위치</param>  
/// <param name="force">넉백 힘</param>  
public void KnockBack(Vector3 attackerPos, float force)  
protected abstract void OnKnockBack(Vector3 attackerPos, float force);  
  
/// <summary>  
/// 특정 트랜스폼을 바라볼도록 회전  
/// </summary>  
/// <param name="targetPosition">바라볼 대상 트랜스폼</param>  
public void LookAt(Transform skillResultHitTransform)  
protected abstract void OnLookAt(Transform targetPosition);  
  
/// <summary>  
/// 특정 방향을 바라볼도록 회전  
/// </summary>  
/// <param name="targetDirection">바라볼 방향 벡터</param>  
public void LookAt(Vector3 targetDirection)  
protected abstract void OnLookAt(Vector3 targetDirection);  
  
/// <summary>
```

```

/// 현재 회전값 가져오기
/// </summary>
/// <returns>현재 회전값(Quaternion)</returns>
public Quaternion GetRigidbodyRotation()
protected abstract Quaternion GetRotation();

/// <summary>
/// 순간 이동 처리
/// </summary>
/// <param name="position">이동할 위치</param>
/// <param name="rotation">설정할 회전값</param>
public void SetPositionAndRotation(Vector3 position, Quaternion rotation)
protected abstract void OnTeleport(Vector3 position, Quaternion rotation);

/// <summary>
/// 트윈 이동 취소 (구현체에서 재정의)
/// </summary>
public void Cancel()
protected abstract void OnCancelTween();

/// <summary>
/// 이동 컨트롤러 초기 설정
/// </summary>
/// <param name="controllerState">컨트롤러 상태 인터페이스</param>
/// <param name="heroObjectInfo">영웅 오브젝트 정보</param>
/// <param name="heroBaseStat">영웅 기본 능력치</param>
public void Setup(IControllerState controllerState, IHeroObjectInfo heroObjectInfo, HeroBaseStat heroBaseStat)
protected abstract void _Setup();

/// <summary>
/// 컨트롤러 활성화/비활성화 설정
/// </summary>
/// <param name="isActive">활성화 여부</param>
protected abstract void _SetActive(bool isActive);
public void SetActive(bool isActive)

```

코드 스니펫

초기 설정 흐름

```

public void Setup(IControllerState controllerState, IHeroObjectInfo heroObjectInfo, HeroBaseStat heroBaseStat)
{
    // 의존성 주입
    ControllerState = controllerState;
    HeroObjectInfo = heroObjectInfo;
    HeroBaseStat = heroBaseStat;

    // 구현체별 추가 설정 호출
    _Setup();
}

```


기능 설명

이동 유형

이동 유형	메서드	설명
일반 이동	<code>Move</code>	입력값 기반의 기본 이동
절대 이동	<code>MoveToAbsoluteDirection</code>	월드 좌표계 기준 이동
상대 이동	<code>MoveToRelativeDirection</code>	로컬 좌표계 기준 이동
넉백	<code>KnockBack</code>	물리 기반 넉백
순간 이동	<code>SetPositionAndRotation</code>	위치/회전 즉시 변경

의존성/상속 관계

상속/구현

- `MonoBehaviour`를 상속 받음
- 구현 클래스
 - `HeroRigidbodyController` 클래스에서 이동 구현
- `IControllerState` 인터페이스를 통해 컨트롤러 상태 정보 확인
- `HeroBaseStat` 클래스에서 영웅 기본 능력치 정보 확인
- `HeroController` 클래스에서 이동 제어
- `BaseHeroAbility` 클래스에서 스킬 실행 중 이동 제어

사용 예시

`HeroController`에서 이동 명령 호출

```
private BaseHeroMoveController heroMoveController;

public void Move(float horizontal, float vertical, float deltaTime)
{
    if (_heroStatus.IsNull()) return;
    _moveDirection = new Vector2(horizontal, vertical);

    // 이동 제어 이벤트 통지
    NotifyMoveControlEvents(moveEvent => moveEvent?.OnMoveControlEvent(_moveDirection));
    if (_controllerState.CanMove == false) return;

    if (!_isFocusMode)
    {
        // 일반 이동 모드
        if (_lastMoveDirection != _moveDirection)
        {
            // 방향이 변경된 경우
            NotifyMoveControlEvents(moveEvent => moveEvent?.OnMoveEvent(_moveDirection));
            heroMoveController.Move(horizontal, vertical, deltaTime, _moveDirection);
        }
    }
}
```

```

        _lastMoveDirection = _moveDirection;
    }
    else
    {
        // 동일한 방향 유지
        heroMoveController.Move(horizontal, vertical, deltaTime, Vector2.zero);
    }
}
else
{
    ...
}
}

```

DualMatchPresenter에서 라운드 초기화 및 영웅 리셋시 초기 위치로 영웅 텔레포트

```

// DualMatchPresenter
private HeroSystem heroSystem;

private void SetupLocalPlayer()
{
    var spawnPoint = spawnPointHandler.GetSpawnPointPositionAndRotation();
    var networkHeroObject = heroSystem.GetNetworkHeroObject();
    networkHeroObject.SetPositionAndRotation(spawnPoint.position, spawnPoint.rotation);
    networkHeroObject.ResetHero(_currentRound);
    networkHeroObject.EnableModelLights(false);
}

// NetworkHeroObject
private HeroController heroController;

public void SetPositionAndRotation(Vector3 position, Quaternion rotation)
{
    heroController.SetPositionAndRotation(position, rotation);
}

// HeroController
private BaseHeroMoveController heroMoveController;

public void SetPositionAndRotation(Vector3 position, Quaternion rotation)
{
    heroMoveController.SetPositionAndRotation(position, rotation);
}

// BaseHeroMoveController
public void SetPositionAndRotation(Vector3 position, Quaternion rotation)
{
    OnTeleport(position, rotation);
}

```

BaseHeroAttackSkill)에서 스킬 효과에 따라 스킬사용시 캐릭터 이동

```

// BaseHeroAttackSkill
protected IHeroController HeroController;

private void OnSkillMovement()
{
    _canMove = false;
    if (_characterMovement.direction == CharacterMovement.Direction.Forward)
    {
        HeroController.MoveToRelativeDirection(
            _characterMovement.moveDistance,
            _characterMovement.moveToDistanceTime,
            Vector3.forward);
    }
}

// HeroController(IHeroController 구현체)
private BaseHeroMoveController heroMoveController;

public void MoveToRelativeDirection(float distance, float time, Vector3 direction)
{
    heroMoveController.MoveToRelativeDirection(distance, time, direction);
}

// BaseHeroMoveController
public void MoveToRelativeDirection(float distance, float time, Vector3 direction)
{
    OnMoveToRelativeDirection(distance, time, direction);
}

```

캐릭터 피격 후 각 캐릭터 상태에 따라 넉백 적용

```

// RektorAbility
private IHeroController _heroController;

protected override void OnHit(IHitEvent.HitInfo hitInfo)
{
    ...

    if (hitInfo.HitterInfo.GuardState)
    {
        OnGuard(hitInfo);
        SetupStiffnessRoutine();
    }
    else
    {
        var hitEffect = GetHitEffectType(hitInfo);
        var effect = FindEffect(hitEffect);
        if (effect is not null)
        {
            PlayEffect(effect, hitInfo);
            PlaySoundEffect(effect);
        }
    }
}

```

```

        var spPoint = _heroStatus.GetSp().Current;
        _heroStatus.ChangeSpecialPoint(spPoint + hitInfo.Damage);
        _heroStatus.ApplyDamage(hitInfo.Damage);
        SetupStiffnessRoutine();

        if (!_isSuperArmor) return;

        _skillManager.Hit(hitInfo);
    }

    if (_heroObjectInfo.IsRemote()) return;
    _heroController.KnockBack(hitInfo.AttackerPos, hitInfo.KnockBackPower);
}

// HeroController(IHeroController 구현체)
private BaseHeroMoveController heroMoveController;

public void KnockBack(Vector3 attackerPos, float force)
{
    heroMoveController.KnockBack(attackerPos, force);
}

// BaseHeroMoveController
public void KnockBack(Vector3 attackerPos, float force)
{
    OnKnockBack(attackerPos, force);
}

```

관련 클래스

- [HeroController](#)
- [HeroRigidbodyController](#)
- [IControllerState](#)
- [HeroBaseStat](#)
- [IHeroController](#)

HeroRigidbodyController

Rigidbody 기반의 영웅 이동 컨트롤러 구현체

개요

HeroRigidbodyController 클래스는 RFist 게임의 Rigidbody 기반 영웅 이동 컨트롤러 구현체입니다. **BaseHeroMoveController**를 상속받아 물리 기반 이동, **DoTween** 기반 트윈 이동, 카메라 기준 이동, 넉백 등의 기능을 구현합니다.

역할

- Rigidbody를 사용한 물리 기반 이동 구현
- **DoTween** 기반 트윈 이동 (대시, 넉백 등) 구현
- 카메라 기준 상대 이동 및 회전 처리
- 벽/아바타 충돌 감지 및 처리

Tween 데이터 생성 및 처리



선언

```
public class HeroRigidbodyController : BaseHeroMoveController
```

멤버

중첩 구조체

```
/// <summary>
/// DoTween 이동에 필요한 데이터 구조체
/// </summary>
private struct HeroTweenMoveData
{
    public Vector3 Destination;
    public float Duration;
    public Vector3 RayDir;
    public float RayDist;
    public int RayDetectLayer;
    public HeroTweenMoveData(Vector3 destination, float duration, Vector3 rayD
}
}
```

속성

```
/// <summary>
/// Rigidbody 컴포넌트 참조
/// </summary>
[SerializeField]
private Rigidbody rigidBody;

/// <summary>
/// LookAt 회전 완료 임계값 (각도)
/// </summary>
private const float LookAtThreshold = 0.01f;

/// <summary>
/// LookAt 회전 속도
/// </summary>
private const float LookAtSpeed = 20f;

/// <summary>
/// 기본 넉백 지속 시간
/// </summary>
private const float DefaultKnockBackDuration = 0.2f;

/// <summary>
/// 기본 레이캐스트 거리
/// </summary>
private const float DefaultRaycastDistance = 0.5f;

/// <summary>
/// 목표 회전 방향
/// </summary>
```

```
private Quaternion _targetDirection;

/// <summary>
/// 타겟 트랜스폼
/// </summary>
private Transform _targetTransform;

/// <summary>
/// 현재 DoTween 인스턴스
/// </summary>
private Tween _currentTween;

/// <summary>
/// 현재 이동 방향
/// </summary>
private Vector2 _direction;

/// <summary>
/// 착지 여부
/// </summary>
private bool _isGrounded;

/// <summary>
/// 점프 중 여부
/// </summary>
private bool _isJumping;

/// <summary>
/// 리모트(네트워크 원격) 여부
/// </summary>
private bool _isRemote = true;

/// <summary>
/// 컨트롤러 활성화 상태
/// </summary>
private bool _controllerActive = false;

/// <summary>
/// 아바타 레이어 마스크
/// </summary>
private int _avatarLayer;

/// <summary>
/// 벽 레이어 마스크
/// </summary>
private int _wallLayer;

/// <summary>
/// 바닥 레이어 마스크
/// </summary>
private int _groundLayer;

/// <summary>
/// 메인 카메라 트랜스폼
/// </summary>
private Transform cameraTransform;
```

메서드

```
/// <summary>
/// 초기화 - 레이어 마스크 설정 및 카메라 참조
/// </summary>
private void Awake()

/// <summary>
/// 물리 업데이트 - 중력 처리 및 회전 보간
/// </summary>
private void FixedUpdate()

/// <summary>
/// 일반 이동 구현
/// </summary>
protected override void OnMove(Vector2 moveDirection, float deltaTime, Vector2

/// <summary>
/// 카메라 기준 이동 처리
/// </summary>
private void MoveUsingCamera(Vector2 direction, float speed, float deltaTime)

/// <summary>
/// 직접 이동 처리
/// </summary>
private void Move(Vector2 direction, float speed, float deltaTime)

/// <summary>
/// 순간 이동 구현
/// </summary>
protected override void OnTeleport(Vector3 position, Quaternion rotation)

/// <summary>
/// 특정 방향을 바라볼도록 회전
/// </summary>
protected override void OnLookAt(Vector3 targetDirection)

/// <summary>
/// 특정 트랜스폼을 바라볼도록 회전
/// </summary>
protected override void OnLookAt(Transform targetTransform)

/// <summary>
/// 목표 회전으로 보간하여 회전
/// </summary>
private void LookAt(Quaternion targetRotation)

/// <summary>
/// 절대 방향 이동 구현 (DoTween 기반)
/// </summary>
protected override void OnMoveToAbsoluteDirection(float distance, float time,
```



```

/// <summary>
/// 상대 방향 이동 구현 (DoTween 기반)
/// </summary>
protected override void OnMoveToRelativeDirection(float distance, float time, Vector3 direction)

/// <summary>
/// 목표 위치 조정 (벽 충돌 시 거리 조정)
/// </summary>
private Vector3 AdjustTargetPos(float distance, Vector3 direction, Vector3 targetPos)

/// <summary>
/// 낙백 구현 (DoTween 기반)
/// </summary>
protected override void OnKnockBack(Vector3 attackerPos, float force)

/// <summary>
/// 트윈 이동 취소
/// </summary>
protected override void OnCancelTween()

/// <summary>
/// 현재 회전값 가져오기
/// </summary>
protected override Quaternion GetRotation()

/// <summary>
/// 초기 설정
/// </summary>
protected override void _Setup()

/// <summary>
/// 활성화/비활성화 설정
/// </summary>
protected override void _SetActive(bool isActive)

/// <summary>
/// 해당 방향으로 이동 가능한지 체크
/// </summary>
private bool IsMoveableDirection(Vector3 rayDir, float rayDist, int targetLayer)

/// <summary>
/// 방향 이동 트윈 생성
/// </summary>
private Tween CreateMoveDirectionTween(HeroTweenMoveData moveData)

```

코드 스니펫

FixedUpdate에서 회전 및 중력처리

```

private void FixedUpdate()
{
    // 리모트 또는 비활성화 상태면 처리하지 않음
    if (isRemote || controllerActive == false) return;
}

```

```

if (_isRemote || _controllerState == false) return;

// 착지 상태 체크
_isGounded = Physics.CheckSphere(rigidBody.position, 0.25f, _groundLayer);

// 이동 불가 또는 입력 없으면 수평 속도 0으로
if (ControllerState.CanMove == false ||
    _direction == Vector2.zero)
{
    rigidBody.linearVelocity = new Vector3(0, Physics.gravity.y, 0);
}
else
{
    // 공중에 있고 점프 중이 아니면 중력 적용
    if (_isGounded == false && _isJumping == false)
    {
        rigidBody.linearVelocity = new Vector3(
            rigidBody.linearVelocity.x, Physics.gravity.y, rigidBody.linearVelocity.z);
    }
}

// 회전 처리
if (_targetTransform != null)
{
    OnLookAt(_targetTransform);
}
else
{
    LookAt(_targetDirection);
}
}

```

이동 처리

```

protected override void OnMove(Vector2 moveDirection, float deltaTime, Vector2 targetDirection)
{
    // 리모트면 처리하지 않음
    if (_isRemote) return;

    _direction = moveDirection;

    // 카메라 기준 이동
    if (moveUsingCamera)
    {
        MoveUsingCamera(moveDirection, HeroBaseStat.defaultMoveSpeed, deltaTime);
    }

    // 방향 전환 필요 시
    if (turnDirection != default)
    {
        var (camF, camR) = GetCameraBasisXZ();
        var turnWorld = (camF * turnDirection.y + camR * turnDirection.x);
        if (turnWorld.sqrMagnitude > 0.0001f)
            _targetDirection = Quaternion.LookRotation(turnWorld);
    }
}

```

```

    }
    else
    {
        // 직접 이동
        Move(moveDirection, HeroBaseStat.defaultMoveSpeed, deltaTime);
        if (turnDirection == default) return;
        _targetDirection = Quaternion.LookRotation(new Vector3(turnDirection.x,
    }
}

private void MoveUsingCamera(Vector2 direction, float speed, float deltaTime)
{
    // 입력이 없거나 키네마틱이면 리턴
    if (direction.sqrMagnitude < 0.01f || rigidBody.isKinematic) return;

    // 카메라 기준 방향 벡터 계산
    var (camForward, camRight) = GetCameraBasisXZ();

    Vector3 moveWorld = (camForward * direction.y + camRight * direction.x);
    moveWorld.y = 0f;

    if (moveWorld.sqrMagnitude > 1f) moveWorld.Normalize();

    // 속도 적용
    var yVelocity = _isGrounded ? 0 : Physics.gravity.y;
    var targetLinearVelocity = moveWorld * speed;
    rigidBody.linearVelocity = new Vector3(targetLinearVelocity.x, yVelocity,

    // 목표 회전 설정
    _targetDirection = Quaternion.LookRotation(moveWorld);
    _targetTransform = null;
}

private void Move(Vector2 direction, float speed, float deltaTime)
{
    // 입력이 없거나 키네마틱이면 리턴
    if(direction.sqrMagnitude < 0.01f || rigidBody.isKinematic) return;

    // 속도 적용
    rigidBody.linearVelocity = new Vector3(direction.x * speed, 0, direction.y
    _targetTransform = null;
}

```

회전 및 보간회전

```

protected override void OnLookAt(Vector3 targetDirection)
{
    var (camForward, camRight) = GetCameraBasisXZ();

    Vector3 moveWorld = (camForward * targetDirection.z + camRight * targetDir
    moveWorld.y = 0f;

    if (moveWorld.sqrMagnitude > 1f) moveWorld.Normalize();
}

```

```

        _targetDirection = Quaternion.LookRotation(moveWorld);
        rigidBody.MoveRotation(_targetDirection);
    }

    protected override void OnLookAt(Transform targetTransform)
    {
        if (!targetTransform) return;
        var directionToTarget = (targetTransform.position - transform.position);
        var targetRotation = Quaternion.LookRotation(directionToTarget);

        _targetDirection = targetRotation;
        rigidBody.MoveRotation(targetRotation);
    }

    private void LookAt(Quaternion targetRotation)
    {
        // 임계값 이내이면 리턴
        if (Quaternion.Angle(rigidBody.rotation, targetRotation) < LookAtThreshold) return;

        // 보간하여 회전
        var rotation = Quaternion.Lerp(rigidBody.rotation, targetRotation, LookAtSpeed * Time.deltaTime);
        rigidBody.MoveRotation(rotation);
    }

```

절대/상대방향 트윈 이동

```

    protected override void OnMoveToAbsoluteDirection(float distance, float time, Vector3 direction)
    {
        // 리모트면 처리하지 않음
        if (!_isRemote) return;

        // 이동 가능 여부 체크
        if (!IsMoveableDirection(direction, DefaultRaycastDistance, _wallLayer | _avatarLayer)) return;

        // 기존 트윈 취소
        _currentTween?.Kill();

        // 목표 위치 조정 (벽 충돌 고려)
        var targetPos = AdjustTargetPos(distance, direction, rigidBody.position + direction * distance);

        // 이동 데이터 생성 및 트윈 실행
        var moveData = new HeroTweenMoveData(
            destination: targetPos,
            duration: time,
            rayDir: direction,
            rayDist: DefaultRaycastDistance,
            rayDetectLayer: _wallLayer | _avatarLayer);

        _currentTween = CreateMoveDirectionTween(moveData);
    }

    protected override void OnMoveToRelativeDirection(float distance, float time, Vector3 direction)
    {
        // 리모트면 처리하지 않음
        if (!_isRemote) return;

        // 이동 가능 여부 체크
        if (!IsMoveableDirection(direction, DefaultRaycastDistance, _wallLayer | _avatarLayer)) return;

        // 기존 트윈 취소
        _currentTween?.Kill();

        // 목표 위치 조정 (벽 충돌 고려)
        var targetPos = AdjustTargetPos(distance, direction, rigidBody.position + direction * distance);

        // 이동 데이터 생성 및 트윈 실행
        var moveData = new HeroTweenMoveData(
            destination: targetPos,
            duration: time,
            rayDir: direction,
            rayDist: DefaultRaycastDistance,
            rayDetectLayer: _wallLayer | _avatarLayer);

        _currentTween = CreateMoveDirectionTween(moveData);
    }

```

```

{
    // 리모트면 처리하지 않음
    if (!_isRemote) return;

    // 로컬 방향을 월드 방향으로 변환
    var dir = _targetDirection * direction;

    // 이동 가능 여부 체크
    if (IsMoveableDirection(dir, DefaultRaycastDistance, _wallLayer | _avatarLayer))
    {
        // 기존 트윈 취소
        _currentTween?.Kill();

        // 목표 위치 조정
        var targetPos = AdjustTargetPos(distance, dir, rigidBody.position + dir * distance);

        // 이동 데이터 생성 및 트윈 실행
        var moveData = new HeroTweenMoveData(
            destination: targetPos,
            duration: time,
            rayDir: dir,
            rayDist: DefaultRaycastDistance,
            rayDetectLayer: _wallLayer | _avatarLayer);

        _currentTween = CreateMoveDirectionTween(moveData);
    }
}

private bool IsMoveableDirection(Vector3 rayDir, float rayDist, int targetLayer)
{
    // 레이캐스트로 충돌 체크
    if (Physics.Raycast(rigidBody.position + rigidBody.centerOfMass, rayDir, out RaycastHit hit, rayDist, targetLayer))
    {
        return false;
    }
    return true;
}

```

넉백 구현

```

protected override void OnKnockBack(Vector3 attackerPos, float force)
{
    // 리모트면 처리하지 않음
    if (!_isRemote) return;

    // 넉백 방향 계산 (공격자 반대 방향)
    var knockBackDir = -(attackerPos - rigidBody.position).normalized;

    // 이동 가능 여부 체크
    if (IsMoveableDirection(knockBackDir, DefaultRaycastDistance, _wallLayer | _avatarLayer))
    {
        // 목표 위치 계산
        var destination = rigidBody.position + knockBackDir * force;

        // 이동 데이터 생성 및 트윈 실행
    }
}

```

```

// 이동 데이터 생성 및 트윈 생성
var moveData = new HeroTweenMoveData(
    destination: destination,
    duration: DefaultKnockBackDuration,
    rayDir: knockBackDir,
    rayDist: DefaultRaycastDistance,
    rayDetectLayer: _wallLayer | _avatarLayer);

_currentTween = CreateMoveDirectionTween(moveData);
}

```

DoTween 이동 트윈 생성

```

private Tween CreateMoveDirectionTween(HeroTweenMoveData moveData)
{
    Vector3 startPos = rigidBody.position;

    return DOVirtual.Float(0, 1, moveData.Duration, (progress) =>
    {
        // 이동 중 충돌 체크
        if (IsMoveableDirection(moveData.RayDir, moveData.RayDist, moveData.RayDetectLayer))
        {
            OnCancelTween();
            return;
        }

        // 수평 위치 보간
        Vector3 targetHorizontalPos = Vector3.Lerp(
            new Vector3(startPos.x, 0, startPos.z),
            new Vector3(moveData.Destination.x, 0, moveData.Destination.z), progress);

        // 새 위치 적용
        Vector3 newPos = new Vector3(targetHorizontalPos.x, rigidBody.position.y, targetHorizontalPos.z);
        rigidBody.MovePosition(newPos);

        // 중력 적용
        rigidBody.linearVelocity = new Vector3(0, _isGrounded ? 0 : Physics.gravity.y, 0);

    }).SetUpdate(UpdateType.Fixed)
    .OnComplete(() =>
    {
        _currentTween = null;
    });
}

```

벽 충돌 감지 및 위치 조정

```

private Vector3 AdjustTargetPos(float distance, Vector3 direction, Vector3 targetPos)
{
    // 벽과 충돌하면 거리 조정
    if (Physics.Raycast(rigidBody.position, direction, out RaycastHit hit, distance))
    {
        // 벽의 법선 벡터
        Vector3 normal = hit.normal;
        // 충돌한 벽의 거리
        float hitDistance = hit.distance;
        // 조정할 거리
        float adjustDistance = distance - hitDistance;
        // 조정된 위치
        Vector3 adjustedPos = targetPos + direction * adjustDistance;
        return adjustedPos;
    }
    return targetPos;
}

```

```

        var adjustedDistance = hit.distance - 0.2f;
        if (adjustedDistance > 0)
        {
            targetPos = rigidBody.position + direction * adjustedDistance;
        }
    }
    return targetPos;
}

```

기능 설명

이동 시스템

- `HeroRigidbodyController`는 두 가지 이동 방식을 제공
 - 1 물리 기반 이동: `MoveUsingCamera()`, `Move()` - `Rigidbody.velocity`를 직접 설정
 - 2 트윈 기반 이동: `CreateMoveDirectionTween()` - `DoTween`을 사용한 보간 이동

충돌 감지

- 이동 전 `IsMoveableDirection()`으로 충돌 체크
- 레이캐스트로 벽/아바타 레이어 체크
- 트윈 이동 중에도 지속적으로 체크하여 즉시 중단 가능

회전 처리

- `_targetDirection`을 향해 `FixedUpdate`에서 보간 회전
- `LookAtThreshold` 이내이면 회전 중단

넉백 시스템

- `OnKnockBack`은 다음 단계로 실행
 - 1 공격자 반대 방향 계산
 - 2 이동 가능 여부 체크
 - 3 `DoTween` 트윈 생성 및 실행

의존성/상속 관계

상속/구현

- `BaseHeroMoveController` 클래스를 상속 받음
- `HeroBaseStat` 클래스에서 영웅 기본 능력치 정보 확인
- `IControllerState` 인터페이스를 통해 컨트롤러 상태 정보 확인
- `IHeroObjectInfo` 인터페이스를 통해 리모트 여부 확인
- `DoTween` [🔗](#) 패키지를 통해 트윈 이동 구현

사용 예시

부모 클래스 [BaseHeroMoveController](#)를 통해 기능 호출 받음

관련 클래스

- [BaseHeroMoveController](#)
- [HeroController](#)
- [HeroBaseStat](#)
- [IControllerState](#)
- [DoTween](#) ↗

IMoveControlEvents

이동 제어 이벤트 인터페이스

개요

`IMoveControlEvents` 인터페이스는 RFist 게임의 이동 입력 이벤트를 정의합니다. 이동 입력과 실제 이동 이벤트를 구분하여 처리하며, 포커스 모드와 제어 가능 여부를 함께 전달합니다.

역할

- 이동 입력 이벤트 계약 정의
- 실제 이동 발생 이벤트 계약 정의
- 이동 방향, 포커스 모드, 제어 가능 여부 전달

선언

```
public interface IMoveControlEvents
```

멤버

메서드

```
/// <summary>
/// 이동 입력 이벤트
/// 입력 디바이스로부터 이동 방향과 포커스 모드, 제어 가능 여부를 전달받습니다.
/// </summary>
/// <param name="direction">이동 방향 (수평/수직)</param>
/// <param name="isFocusMode">포커스 모드 여부</param>
/// <param name="controllable">제어 가능 여부</param>
public void OnMoveControlEvents(Vector2 direction, bool isFocusMode, bool controllable);

/// <summary>
/// 실제 이동 이벤트
/// 실제 이동이 발생했을 때 호출됩니다.
/// </summary>
/// <param name="direction">이동 방향 (수평/수직)</param>
/// <param name="isFocusMode">포커스 모드 여부</param>
public void OnMoveEvent(Vector2 direction, bool isFocusMode);
```

기능 설명

이벤트 구분

- `IMoveControlEvent` 는 두 가지 이벤트를 구분 제공

메서드	호출 시점	용도
<code>OnMoveControlEvent</code>	입력 디바이스에서 이동 입력이 감지될 때	입력 처리, 상태 체크
<code>OnMoveEvent</code>	실제 이동이 실행될 때	이동 애니메이션, 이펙트

이벤트 파라미터

direction (Vector2):

- `x`: 수평 이동 (-1: 좌, 1: 우, 0: 정지)
- `y`: 수직 이동 (-1: 하, 1: 상, 0: 정지)

controllable (bool) - OnMoveControlEvent 전용:

- `true`: 이동 제어 가능
- `false`: 이동 제어 불가 (스턴, 컷씬 등)

의존성/상속 관계

- 구현 클래스
 - `BaseHeroAbility`
 - `HeroAnimationController`
- `HeroController` 클래스에서 이벤트 등록 및 관리

사용 예시

`HeroController`에서 `OnMoveControlEvent` 및 `OnMoveEvent` 이벤트 발신

```
public void Move(float horizontal, float vertical, float deltaTime)
{
    if (_heroStatus.IsNull()) return;
    _moveDirection = new Vector2(horizontal, vertical);

    // 이동 제어 이벤트 통지
    NotifyMoveControlEvents(moveEvent => moveEvent?.OnMoveControlEvent(_moveDirection));
    if (_controllerState.CanMove == false) return;

    if (!_isFocusMode)
    {
        // 일반 이동 모드
        if (_lastMoveDirection != _moveDirection)
        {
            // 방향이 변경된 경우
        }
    }
}
```

```

        // 방향이 변경된 경우
        NotifyMoveControlEvents(moveEvent ⇒ moveEvent?.OnMoveEvent(_moveD
        heroMoveController.Move(horizontal, vertical, deltaTime, _moveDire
        _lastMoveDirection = _moveDirection;
    }
    else
    {
        // 동일한 방향 유지
        heroMoveController.Move(horizontal, vertical, deltaTime, Vector2.z
    }
}
else
{
    // 포커스 모드 이동 (타겟을 바라보며 이동)
    if (_focusTarget ≠ null)
    {
        heroMoveController.FocusMove(horizontal, vertical, deltaTime, _foc
    }
    NotifyMoveControlEvents(moveEvent ⇒ moveEvent?.OnMoveEvent(_moveDirec
}
}
}

```

관련 클래스

- [HeroController](#)
- [BaseHeroAbility](#)
- [HeroAnimationController](#)

IControllerState

영웅 컨트롤러의 상태를 정의하는 인터페이스

개요

`IControllerState` 인터페이스는 RFist 게임의 영웅 컨트롤러 상태를 정의합니다. 이동, 대시, 공격, 가드 등의 행동 가능 여부를 제공하며, `HeroControllerStateHandler` 클래스가 이 인터페이스를 구현합니다.

역할

- 영웅 캐릭터의 행동 가능 상태 계약 정의
- 이동/대시/공격/가드/제어 가능 여부 제공
- 상태 기반 행동 제어의 표준 인터페이스

선언

```
public interface IControllerState
```

멤버

메서드

```
/// <summary>
/// 컨트롤러 상태 초기 설정
/// </summary>
/// <param name="ability">영웅 능력치 컴포넌트</param>
public void Setup(BaseHeroAbility ability);
```

속성

```
/// <summary>
/// 이동 가능 여부
/// </summary>
public bool CanMove { get; }

/// <summary>
```

```

/// 대시 가능 여부
/// </summary>
public bool CanDash { get; }

/// <summary>
/// 공격 가능 여부
/// </summary>
public bool CanAttack { get; }

/// <summary>
/// 가드 가능 여부
/// </summary>
public bool CanGuard { get; }

/// <summary>
/// 전반적인 제어 가능 여부
/// </summary>
public bool CanControllable { get; }

```

기능 설명

상태 속성

- `IControllerState`는 5가지 행동 가능 상태 정의

속성	설명	일반적인 제약 조건
<code>CanMove</code>	이동 가능 여부	피격 중, 스킬 사용 중, 다운, 컷썸, 가드 중 불가
<code>CanDash</code>	대시 가능 여부	피격 중, 컷썸, 대시 쿨다운 중 불가
<code>CanAttack</code>	공격 가능 여부	피격 중, 컷썸 중 불가
<code>CanGuard</code>	가드 가능 여부	스킬 사용 중, 피격 중, 컷썸, 가드 중 불가
<code>CanControllable</code>	전반적 제어 가능 여부	피격 중, 컷썸 중 불가

상태 변화

- 상태는 다음과 같은 상황에서 변경
- **스킬 사용**: 스킬 타입에 따라 `CanMove`, `CanGuard` 등이 제한됨
- **피격**: `CanMove`, `CanAttack`, `CanDash` 등이 일시적으로 불가능해짐
- **자세 변경**: 다운/스턴 시 대부분의 행동 불가
- **컷썸**: 모든 행동 제한

의존성/상속 관계

- 구현 클래스
 - `HeroControllerStateHandler`에서 컨트롤러 상태 구현
- `HeroController`에서 컨트롤러 상태 확인
- `BaseHeroMoveController`에서 이동 가능 여부 확인

- [BaseHeroMoveController](#)에서 이동 동작 처리
- [BaseHeroAbility](#)를 통해 구현체에서 [HeroBaseStat](#) 참조 획득

사용 예시

[HeroController](#)에서 컨트롤러 상태를 확인하여 입력 동작 처리

```
private IControllerState _controllerState;

// 1. 이동동작
/// <summary>
/// 이동 처리
/// </summary>
/// <param name="horizontal">수평 입력값</param>
/// <param name="vertical">수직 입력값</param>
/// <param name="deltaTime">델타 타임</param>
public void Move(float horizontal, float vertical, float deltaTime)
{
    if (_heroStatus.IsNull()) return;
    _moveDirection = new Vector2(horizontal, vertical);

    // 이동 제어 이벤트 통지
    NotifyMoveControlEvents(moveEvent => moveEvent?.OnMoveControlEvent(_moveDirection));
    if (_controllerState.CanMove == false) return;

    ...
}

// 2. 대시처리
/// <summary>
/// 대시 (방향 벡터 기준)
/// </summary>
/// <param name="vertical">수직 입력값</param>
/// <param name="horizontal">수평 입력값</param>
public void Dash(float vertical, float horizontal)
{
    if (_controllerState.CanDash == false) return;
    var direction = new Vector2(horizontal, vertical);
    _baseHeroAbility?.Dash(direction);
}

// 3. 공격/공격 홀드 처리
/// <summary>
/// 공격 실행
/// </summary>
/// <param name="syncData">입력 동기화 데이터</param>
public void Attack(NetworkHeroController.HeroInputSyncData syncData)
{
    if (_controllerState.CanAttack == false) return;
    _baseHeroAbility?.Attack(syncData);
}

/// <summary>
/// 공격 키 홀드
```

```

/// </summary>
/// <param name="syncData">입력 동기화 데이터</param>
public void AttackHold(NetworkHeroController.HeroInputSyncData syncData)
{
    if (_controllerState.CanAttack == false) return;
    _baseHeroAbility?.AttackHold(syncData);
}

// 4. 가드 처리
/// <summary>
/// 가드 실행
/// </summary>
/// <param name="syncData">입력 동기화 데이터</param>
public void Guard(NetworkHeroController.HeroInputSyncData syncData)
{
    if (_controllerState.CanGuard == false) return;
    _baseHeroAbility?.Guard(syncData);
}

```

HeroRigidbodyController에서 물리 업데이트시 상태 확인

```

Protected IControllerState ControllerState;

/// <summary>
/// 물리 업데이트 - 중력 처리 및 회전 보간
/// </summary>
private void FixedUpdate()
{
    // 리모트 또는 비활성화 상태면 처리하지 않음
    if (_isRemote || _controllerActive == false) return;

    // 착지 상태 체크
    _isGrounded = Physics.CheckSphere(rigidBody.position, 0.25f, _groundLayer);

    // 이동 불가 또는 입력 없으면 수평 속도 0으로
    if (ControllerState.CanMove == false ||
        _direction == Vector2.zero)
    {
        rigidBody.linearVelocity = new Vector3(0, Physics.gravity.y, 0);
    }
    else
    {
        // 공중에 있고 점프 중이 아니면 중력 적용
        if (_isGrounded == false && _isJumping == false)
        {
            rigidBody.linearVelocity = new Vector3(
                rigidBody.linearVelocity.x, Physics.gravity.y, rigidBody.linearVelocity.z);
        }
    }

    // 회전 처리
    if (_targetTransform != null)
    {

```

```
        OnLookAt(_targetTransform);
    }
    else
    {
        LookAt(_targetDirection);
    }
}
```

관련 클래스

- [HeroControllerStateHandler](#)
- [HeroController](#)
- [BaseHeroMoveController](#)
- [BaseHeroAbility](#)

HeroControllerStateHandler

영웅 컨트롤러의 상태를 관리하는 핸들러 클래스

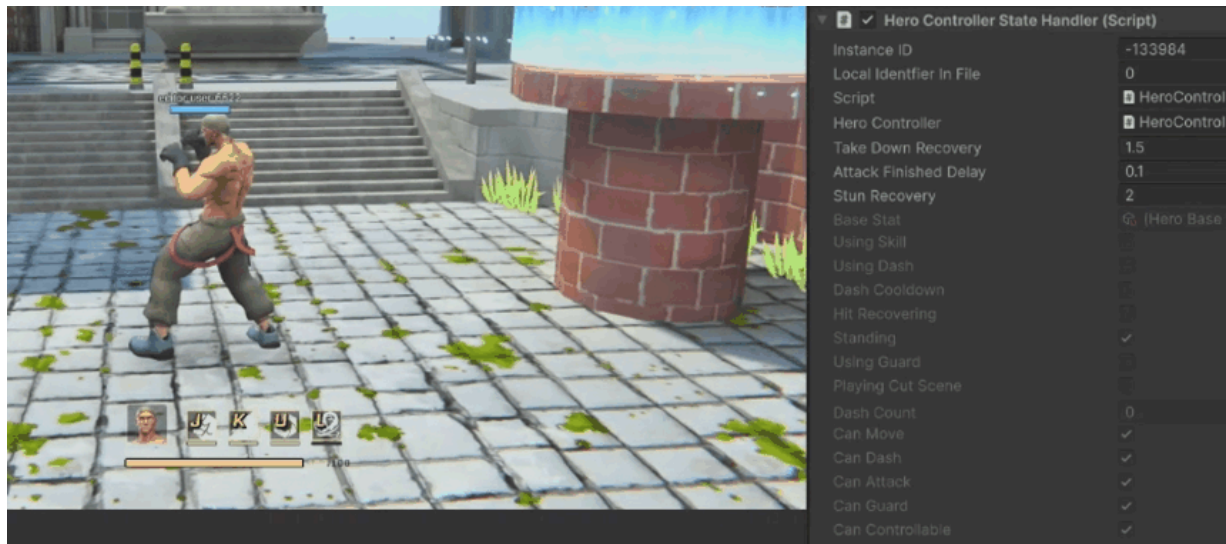
개요

HeroControllerStateHandler 클래스는 RFist 게임의 영웅 컨트롤러 상태를 관리하는 핸들러입니다. 스킬 사용, 피격, 자세 변경(일어나기/다운/스턴), 컷씬 등의 이벤트를 수신하여 행동 가능 상태(CanMove, CanAttack 등)를 동적으로 업데이트합니다. **IControllerState** 인터페이스를 구현합니다.

역할

- 스킬 시작/종료에 따른 상태 관리 (이동/공격/가드 가능 여부)
- 피격 시 컨트롤 회복 시간 관리 및 상태 제한
- 대시 횟수 및 쿨다운 관리
- 자세 변경(일어나기/다운/스턴)에 따른 상태 전환
- 컷씬 재생 중 컨트롤 제한

컨트롤러 상태 관리



선언

```
public class HeroControllerStateHandler : MonoBehaviour, ISkillEvent, IHitEvent
```

멤버

속성

```
/// <summary>
/// 영웅 컨트롤러 참조
/// </summary>
[SerializeField]
private HeroController heroController;

/// <summary>
/// 다운 상태 회복 시간
/// </summary>
[SerializeField]
private float takeDownRecovery = 1.5f;

/// <summary>
/// 공격 종료 후 지연 시간
/// </summary>
[SerializeField]
private float attackFinishedDelay = 0.1f;

/// <summary>
/// 스톤 회복 시간
/// </summary>
[SerializeField]
private float stunRecovery = 2f;

/// <summary>
/// 스킬 이벤트 인터페이스 (this 캐스팅)
/// </summary>
private ISkillEvent SkillEvent => this;

/// <summary>
/// 피격 이벤트 인터페이스 (this 캐스팅)
/// </summary>
private IHitEvent HitEvent => this;

/// <summary>
/// 자세 변경 이벤트 인터페이스 (this 캐스팅)
/// </summary>
private IStanceEvent StanceEvent => this;

/// <summary>
/// 컷씬 이벤트 인터페이스 (this 캐스팅)
/// </summary>
private ICutSceneEvent CutSceneEvent => this;

/// <summary>
/// 피격 회복 코루틴 참조
/// </summary>
private Coroutine _hitRecoveryCoroutine;

/// <summary>
/// 대시 쿨다운 코루틴 참조
/// </summary>
```

```
/// </summary>
private Coroutine _dashCooldownCoroutine;

/// <summary>
/// 영웅 기본 능력치
/// </summary>
private HeroBaseStat _baseStat;

/// <summary>
/// 스킬 사용 중 여부
/// </summary>
private bool _usingSkill;

/// <summary>
/// 대시 사용 중 여부
/// </summary>
private bool _usingDash;

/// <summary>
/// 대시 쿨다운 중 여부
/// </summary>
private bool _dashCooldown;

/// <summary>
/// 피격 회복 중 여부
/// </summary>
private bool _hitRecovering;

/// <summary>
/// 서있는 상태 여부
/// </summary>
private bool _standing;

/// <summary>
/// 가드 사용 중 여부
/// </summary>
private bool _usingGuard;

/// <summary>
/// 컷씬 재생 중 여부
/// </summary>
private bool _playingCutScene;

/// <summary>
/// 현재 대시 횟수
/// </summary>
private int _dashCount;

/// <summary>
/// 이동 가능 여부
/// </summary>
public bool CanMove { get; private set; }

/// <summary>
/// 대시 가능 여부
/// </summary>
```

```

public bool CanDash { get; private set; }

/// <summary>
/// 공격 가능 여부
/// </summary>
public bool CanAttack { get; private set; }

/// <summary>
/// 가드 가능 여부
/// </summary>
public bool CanGuard { get; private set; }

/// <summary>
/// 전반적인 제어 가능 여부
/// </summary>
public bool CanControllable { get; private set; }

```

메서드

```

/// <summary>
/// 영웅 능력치 설정
/// </summary>
/// <param name="ability">영웅 능력치 컴포넌트</param>
public void Setup(BaseHeroAbility ability)

/// <summary>
/// 상태 초기화
/// </summary>
public void ResetState()

/// <summary>
/// 상태 업데이트 - 모든 상태 플래그를 기반으로 IControllerState 속성 갱신
/// </summary>
private void UpdateState()

/// <summary>
/// 대시 횟수 체크
/// </summary>
/// <returns>대시 가능 여부</returns>
private bool CheckDashCount()

/// <summary>
/// 스킬 시작 시 호출
/// </summary>
/// <param name="skillStartResult">스킬 시작 결과</param>
public void OnSkill(BaseHeroSkill.SkillStartResult skillStartResult)

/// <summary>
/// 스킬 종료 시 호출
/// </summary>
/// <param name="skillFinishResult">스킬 종료 결과</param>
public void OnSkillFinished(BaseHeroSkill.SkillFinishResult skillFinishResult)

```

```

/// <summary>
/// 스킬 시작 처리
/// </summary>
private void OnSkillStart()

/// <summary>
/// 스킬 완료 처리
/// </summary>
private void OnSkillComplete()

/// <summary>
/// 대시 시작 처리
/// </summary>
/// <param name="duration">대시 지속 시간</param>
private void OnDash(float duration)

/// <summary>
/// 가드 시작 처리
/// </summary>
private void OnGuard()

/// <summary>
/// 가드 완료 처리
/// </summary>
private void OnGuardComplete()

/// <summary>
/// 대시 완료 처리
/// </summary>
private void OnDashCompleted()

/// <summary>
/// 일어나기 완료 처리
/// </summary>
private void OnStandUpComplete()

/// <summary>
/// 피격 수신 시 호출
/// </summary>
/// <param name="hitInfo">피격 정보</param>
public void OnHitReceive(IHitEvent.HitInfo hitInfo)

/// <summary>
/// 피격 회복 대기 코루틴
/// </summary>
/// <param name="hitRecoveryTime">회복 시간</param>
private IEnumerator CoWaitHitRecovery(float hitRecoveryTime)

/// <summary>
/// 대시 쿨다운 대기 코루틴
/// </summary>
/// <param name="dashCooldownTime">쿨다운 시간</param>
private IEnumerator CoWaitDashCooldown(float dashCooldownTime)

/// <summary>
/// 특수 이동 시퀀스(컷썬) 시작 시 호출

```

```

/// <summary>
/// </summary>
public void OnSpecialMoveSequenceStart()

/// <summary>
/// 특수 이동 시퀀스(컷씬) 종료 시 호출
/// </summary>
public void OnSpecialMoveSequenceEnd()

/// <summary>
/// 다운 상태로 전환
/// </summary>
public void TakeDown()

/// <summary>
/// 일어나기 처리
/// </summary>
public void WakeUp()

/// <summary>
/// 스톤 상태로 전환
/// </summary>
public void Stun()

```

코드 스니펫

상태 업데이트 로직

```

/// <summary>
/// 상태 업데이트 - 모든 상태 플래그를 기반으로 IControllerState 속성 갱신
/// </summary>
private void UpdateState()
{
    // 이동 가능: 피격 회복 중 아님, 스킬 사용 중 아님, 서있음, 컷씬 아님, 가드 중 아님
    CanMove = _hitRecovering = false
    && _usingSkill = false
    && _standing
    && _playingCutScene = false
    && _usingGuard = false;

    // 대시 가능: 피격 회복 중 아님, 컷씬 아님, 대시 횟수 체크, 쿨다운 아님
    CanDash = _hitRecovering = false
    && _playingCutScene = false
    && CheckDashCount()
    && _dashCooldown = false;

    // 공격 가능: 피격 회복 중 아님, 컷씬 아님
    CanAttack = _hitRecovering = false &&
    _playingCutScene = false;

    // 가드 가능: 스킬 사용 중 아님, 피격 회복 중 아님, 컷씬 아님, 가드 중 아님
    CanGuard = _usingSkill = false
    && _hitRecovering = false

```

```

        && _playingCutScene = false;
        && _usingGuard = false;

        // 전반적 제어 가능: 피격 회복 중 아님, 컷씬 아님
        CanControllable = _hitRecovering = false;
        && _playingCutScene = false;
    }

```

스킬 타입별 상태 처리

```

/// <summary>
/// 스킬 시작 시 호출
/// </summary>
public void OnSkill(BaseHeroSkill.SkillStartResult skillStartResult)
{
    switch (skillStartResult.SkillType)
    {
        case BaseHeroSkill.SkillType.Attack:
        case BaseHeroSkill.SkillType.AttackHold:
        case BaseHeroSkill.SkillType.Utility:
        case BaseHeroSkill.SkillType.StanceChange:
            OnSkillStart();
            break;
        case BaseHeroSkill.SkillType.Dash:
        case BaseHeroSkill.SkillType.Roll:
            OnDash(skillStartResult.Duration);
            break;
        case BaseHeroSkill.SkillType.Guard:
            OnGuard();
            break;
    }
}

/// <summary>
/// 스킬 시작 처리
/// </summary>
private void OnSkillStart()
{
    _usingSkill = true;
    UpdateState();
}

/// <summary>
/// 대시 시작 처리
/// </summary>
/// <param name="duration">대시 지속 시간</param>
private void OnDash(float duration)
{
    _usingSkill = true;
    _usingDash = true;
    _dashCount++;

    // 최대 대시 횟수 도달 시 쿨다운 시작
    if (_dashCount > _baseStat.maxDashCount)

```

```

    if (_dashCount >= _baseStat.maxDashCount)
    {
        _dashCooldown = true;
        _dashCooldownCoroutine = StartCoroutine(CoWaitDashCooldown(_baseStat.d
    }

    UpdateState();
}

/// <summary>
/// 가드 시작 처리
/// </summary>
private void OnGuard()
{
    _usingGuard = true;
    UpdateState();
}

/// <summary>
/// 스킬 종료 시 호출
/// </summary>
/// <param name="skillFinishResult">스킬 종료 결과</param>
public void OnSkillFinished(BaseHeroSkill.SkillFinishResult skillFinishResult)
{
    switch (skillFinishResult.Skill.skillType)
    {
        case BaseHeroSkill.SkillType.Attack :
        case BaseHeroSkill.SkillType.AttackHold:
        case BaseHeroSkill.SkillType.Utility:
            OnSkillComplete();
            break;
        case BaseHeroSkill.SkillType.WakeUpAttack:
        case BaseHeroSkill.SkillType.StanceChange:
            OnStandUpComplete();
            break;
        case BaseHeroSkill.SkillType.Dash:
        case BaseHeroSkill.SkillType.Roll:
            OnDashCompleted();
            break;
        case BaseHeroSkill.SkillType.Guard:
            OnGuardComplete();
            break;
    }
}

/// <summary>
/// 스킬 완료 처리
/// </summary>
private void OnSkillComplete()
{
    _usingSkill = false;
    UpdateState();
}

/// <summary>
/// 다시 완료 처리

```



```

/// </summary>
private void OnDashCompleted()
{
    _usingSkill = false;
    _standing = true;
    _usingDash = false;

    // 대시 완료 후 쿨다운 시작
    if (_dashCooldown == false)
    {
        _dashCooldown = true;
        _dashCooldownCoroutine = StartCoroutine(CoWaitDashCooldown(_baseStat.d

    }

    UpdateState();
}

/// <summary>
/// 가드 완료 처리
/// </summary>
private void OnGuardComplete()
{
    _usingGuard = false;
    UpdateState();
}

/// <summary>
/// 일어나기 완료 처리
/// </summary>
private void OnStandUpComplete()
{
    _usingSkill = false;
    _standing = true;
    _usingDash = false;
    UpdateState();
}

```

대시 쿨다운 관리

```

/// <summary>
/// 대시 시작 처리
/// </summary>
private void OnDash(float duration)
{
    _usingSkill = true;
    _usingDash = true;
    _dashCount++;

    // 최대 대시 횟수 도달 시 쿨다운 시작
    if (_dashCount ≥ _baseStat.maxDashCount)
    {
        _dashCooldown = true;
        _dashCooldownCoroutine = StartCoroutine(CoWaitDashCooldown(_baseStat.d
    }

```

```

    }

    UpdateState();
}

/// <summary>
/// 대시 쿨다운 대기 코루틴
/// </summary>
private IEnumerator CoWaitDashCooldown(float dashCooldownTime)
{
    yield return new WaitForSeconds(dashCooldownTime);
    _dashCount = 0;
    _dashCooldown = false;
    _dashCooldownCoroutine = null;

    UpdateState();
}

```

피격 회복 처리

```

/// <summary>
/// 피격 수신 시 호출
/// </summary>
public void OnHitReceive(IHitEvent.HitInfo hitInfo)
{
    // 피격자가 StandUp상태가 아닐시 중지
    if (hitInfo.HitterInfo.HitterStance != IHeroStatus.Stance.StandUp) return;

    // 기존 회복 코루틴 중지
    if (_hitRecoveryCoroutine != null) StopCoroutine(_hitRecoveryCoroutine);

    // 피격 회복 상태로 전환
    _hitRecovering = true;
    _hitRecoveryCoroutine = StartCoroutine(CoWaitHitRecovery(hitInfo.RecoveryTime));

    UpdateState();
}

/// <summary>
/// 다운 상태로 전환
/// </summary>
public void TakeDown()
{
    // 기존 회복 코루틴 중지
    if (_hitRecoveryCoroutine != null)
    {
        StopCoroutine(_hitRecoveryCoroutine);
    }

    // 다운 상태로 전환
    _standing = false;
    _usingSkill = false;

    // 회복 코루틴 시작
}

```

```

        // 회복 코루틴 시작
        _hitRecovering = true;
        _hitRecoveryCoroutine = StartCoroutine(CoWaitHitRecovery(takeDownRecovery)
        UpdateState());
    }

    /// <summary>
    /// 스톤 상태로 전환
    /// </summary>
    public void Stun()
    {
        // 기존 회복 코루틴 중지
        if (_hitRecoveryCoroutine != null)
        {
            StopCoroutine(_hitRecoveryCoroutine);
        }

        // 스톤 상태로 전환
        _hitRecovering = true;
        _hitRecoveryCoroutine = StartCoroutine(CoWaitHitRecovery(stunRecovery));
        UpdateState();
    }

    /// <summary>
    /// 피격 회복 대기 코루틴
    /// </summary>
    private IEnumerator CoWaitHitRecovery(float hitRecoveryTime)
    {
        yield return new WaitForSeconds(hitRecoveryTime);
        _hitRecovering = false;
        _hitRecoveryCoroutine = null;
        UpdateState();
    }
}

```

기능 설명

상태 관리 시스템

- HeroControllerStateHandler는 다양한 상태 플래그를 조합하여 5가지 행동 가능 상태 결정

상태	조건
CanMove	!_hitRecovering && !_usingSkill && _standing && !_playingCutScene && !_usingGuard
CanDash	!_hitRecovering && !_playingCutScene && CheckDashCount() && !_dashCooldown
CanAttack	!_hitRecovering && !_playingCutScene
CanGuard	!_usingSkill && !_hitRecovering && !_playingCutScene && !_usingGuard
CanControllable	!_hitRecovering && !_playingCutScene

스킬 타입별 처리

- 스킬 타입에 따라 다른 상태 처리
 - Attack/AttackHold/Utility/StanceChange**: 스킬 사용 중 상태로 전환
 - Dash/Roll**: 대시 횟수 증가 및 쿨다운 관리
 - Guard**: 가드 상태로 전환
 - WakeUpAttack**: 일어나기 완료 처리

대시 쿨다운 시스템

- `maxDashCount` 만큼 연속 대시 가능
- 최대 횟수 도달 시 `dashCoolTime` 동안 쿨다운
- 쿨다운 중에는 대시 불가

피격/스턴/다운 상태

- 피격 시 `RecoveryTime` 동안 회복 상태
- 스턴 시 `stunRecovery` 시간 동안 행동 불가
- 다운 시 `takeDownRecovery` 시간 동안 행동 불가

의존성/상속 관계

- `MonoBehaviour` 를 상속 받음
- 인터페이스 구현
 - `ISkillEvent` 인터페이스 구현
 - `IHitEvent` 인터페이스 구현
 - `IControllerState` 인터페이스 구현
 - `IStanceEvent` 인터페이스 구현
 - `ICutSceneEvent` 인터페이스 구현
- `HeroController` 에서 상태를 통해 입력 조건 확인
- `HeroBaseStat` 스탯 정보를 통해 대시 쿨타임, 대시 횟수 제어
- `BaseHeroSkill` 스킬 타입 정보를 통해 컨트롤러 상태 제어

사용 예시

`NetworkHeroObject` 에서 게임/라운드 리셋시 컨트롤러 상태 리셋

```
/// <summary>
/// 영웅 리셋을 모든 클라이언트에 동기화하는 RPC
/// </summary>
/// <param name="round">현재 라운드 번호</param>
[Rpc(RpcSources.InputAuthority, RpcTargets.All)]
private void RPC_ResetHero(int round)
{
    // 상태 초기화
    networkHeroStatus.Setup(_baseHeroAbility.GetHeroStat(), round);
}
```

```
heroControllerStateHandler.ResetState();
heroController.ResetFocusMode();
heroAnimationController.StartMatch();

// 첫 라운드인 경우 특수 게이지 초기화
if (IsFirstRound(round)) networkHeroStatus.ChangeSpecialPoint(0);
}
```

관련 클래스

- [HeroController](#)
- [BaseHeroAbility](#)
- [HeroBaseStat](#)
- [BaseHeroSkill](#)
- [ISkillEvent](#)
- [IHitEvent](#)
- [IStanceEvent](#)
- [ICutSceneEvent](#)
- [IControllerState](#)

ICutSceneEvent

컷씬(특수 스킬) 이벤트 인터페이스

개요

ICutSceneEvent 인터페이스는 RFist 게임의 컷씬(특수 스킬) 이벤트를 정의합니다. 특수 스킬 컷씬의 시작과 종료를 알리는 이벤트를 정의하며, **HeroControllerStateHandler**, **HitPanel** 클래스가 이 인터페이스를 구현합니다.

역할

- 특수 스킬 컷씬 시작/종료 이벤트 정의

선언

```
public interface ICutSceneEvent
```

멤버

메서드

```
/// <summary>
/// 특수 스킬 컷씬 시작 시 호출
/// </summary>
public void OnSpecialMoveSequenceStart();

/// <summary>
/// 특수 스킬 컷씬 종료 시 호출
/// </summary>
public void OnSpecialMoveSequenceEnd();
```

기능 설명

컷씬 이벤트

메서드	호출 시점	역할
OnSpecialMoveSequenceStart	특수 스킬(궁극기 등) 사용 시	컷씬 시작, 컨트롤 비활성화

OnSpecialMoveSequenceEnd

컷씬 애니메이션 종료 시

컨트롤 복원, 상태 초기화

이벤트 흐름

- 1 컷씬 발생
- 2 `TimelineManager` 등에서 `CutSceneController`를 통해 `ICutSceneEventInvoker` 이벤트 발신자 사용
- 3 이벤트 발신자가 모든 `ICutSceneEvent` 구독자에게 알림(컷씬 재생 및 종료)

의존성/상속 관계

- `HeroControllerStateHandler`에서 통해 컨트롤러 상태 변경
- `HitPanel`에서 히트정보 표시 제어
- `CutSceneController`에서 컷씬 시작/종료 시 이벤트 발신
- `ICutSceneEventInvoker` 인터페이스를 통해 이벤트 발신
- `HeroController`에서 이벤트 수신자 등록 및 관리

사용 예시

`HeroController`에서 이벤트 리스너 등록 및 관리

```
private readonly HashSet<ICutSceneEvent> _cutSceneEvents = new HashSet<ICutSceneEvent>();

public void Register(ICutSceneEvent cutSceneEvent)
{
    _cutSceneEvents.Add(cutSceneEvent);
}

public void UnRegister(ICutSceneEvent cutSceneEvent)
{
    _cutSceneEvents.Remove(cutSceneEvent);
}
```

`HeroControllerStateHandler`에서 컨트롤러 활성화/비활성화 제어

```
public void OnSpecialMoveSequenceStart()
{
    _playingCutScene = true;
    UpdateState(); // 상태 업데이트로 컨트롤 비활성화
}

public void OnSpecialMoveSequenceEnd()
{
    _playingCutScene = false;
    UpdateState(); // 상태 업데이트로 컨트롤 복원
}

private void UpdateState()
```

```

private void UpdateState()
{
    // 이동 가능: 피격 회복 중 아님, 스킬 사용 중 아님, 서있음, 컷씬 아님, 가드 중 아님
    CanMove = _hitRecovering == false
        && _usingSkill == false
        && _standing
        && _playingCutScene == false
        && _usingGuard == false;

    // 대시 가능: 피격 회복 중 아님, 컷씬 아님, 대시 횟수 체크, 쿨다운 아님
    CanDash = _hitRecovering == false
        && _playingCutScene == false
        && CheckDashCount()
        && _dashCooldown == false;

    // 공격 가능: 피격 회복 중 아님, 컷씬 아님
    CanAttack = _hitRecovering == false &&
        _playingCutScene == false;

    // 가드 가능: 스킬 사용 중 아님, 피격 회복 중 아님, 컷씬 아님, 가드 중 아님
    CanGuard = _usingSkill == false
        && _hitRecovering == false
        && _playingCutScene == false
        && _usingGuard == false;

    // 전반적 제어 가능: 피격 회복 중 아님, 컷씬 아님
    CanControllable = _hitRecovering == false
        && _playingCutScene == false;
}

```

관련 클래스

- [HeroControllerStateHandler](#)
- [CutSceneController](#)
- [ICutSceneEventInvoker](#)
- [HeroController](#)

ICutSceneEventInvoker

컷씬 이벤트 발신자 인터페이스

개요

`ICutSceneEventInvoker` 인터페이스는 RFist 게임의 컷씬 이벤트 발신자 역할을 정의합니다.

`ICutSceneEvent`를 구현하는 모든 구독자에게 이벤트를 발신하는 기능을 제공합니다.

역할

- `ICutSceneEvent` 구독자 관리
- 컷씬 이벤트 일괄 발신
- 발신자-구독자 간 느슨한 결합 제공

선언

```
public interface ICutSceneEventInvoker
```

멤버

메서드

```
/// <summary>
/// 컷씬 이벤트 발신
/// ICutSceneEvent를 구현하는 모든 구독자에게 지정된 액션을 실행합니다.
/// </summary>
/// <param name="action">구독자에게 실행할 액션 (예: OnSpecialMoveSequenceStart)</param>
public void NotifyCutSceneEvents(Action<ICutSceneEvent> action);
```

기능 설명

관찰자 패턴 구조

- `ICutSceneEventInvoker`는 관찰자 패턴의 발신자 역할
- `NotifyCutSceneEvents(Action<ICutSceneEvent> action)`를 통해 이벤트 발신
- `ICutSceneEvent` 구독자에게 알림

이벤트 흐름

- 1 컷씬 발생
- 2 `TimelineManager` 등에서 `CutSceneController`를 통해 `ICutSceneEventInvoker` 이벤트 발신자 사용
- 3 이벤트 발신자가 모든 `ICutSceneEvent` 구독자에게 알림(컷씬 재생 및 종료)

Action 활용

- `NotifyCutSceneEvents(Action<ICutSceneEvent> action)`는 제네릭 Action 델리게이트를 사용하여 유연하게 이벤트 발신

```
// 컷씬 시작 시 모든 구독자에게 OnSpecialMoveSequenceStart 호출
invoker.NotifyCutSceneEvents(action => action?.OnSpecialMoveSequenceStart());

// 컷씬 종료 시 모든 구독자에게 OnSpecialMoveSequenceEnd 호출
invoker.NotifyCutSceneEvents(action => action?.OnSpecialMoveSequenceEnd());
```

의존성/상속 관계

- `ICutSceneEvent` 인터페이스 발신 대상
- `CutSceneController`: 컷씬 시작/종료 시 이벤트 발신
- `HeroController`에서 해당 인터페이스 구현

사용 예시

`HeroController` 해당 인터페이스를 구현하여 `ICutSceneEvent`로 이벤트 발신

```
public void NotifyCutSceneEvents(Action<ICutSceneEvent> action)
{
    foreach (var cutSceneEvent in _cutSceneEvents)
    {
        action?.Invoke(cutSceneEvent);
    }
}
```

`CutSceneController`에서 해당 인터페이스를 통해 이벤트 발신

```
public void StartCutScene()
{
    if (IsInitialized() == false)
    {
        Debug.LogError($"[CutSceneController] Not Initialized");
        return;
    }

    // 구현자 구현
```

```

        // 공격자가 로컬인지 확인
        _attackIsLocal = !_attacker.IsRemote();

        // 모든 컷씬 이벤트 리스너에게 시작 통지
        foreach (var cutSceneEventInvoker in _cutSceneEventInvokers)
        {
            cutSceneEventInvoker.NotifyCutSceneEvents(action ⇒ action?.OnSpecialM

        }
    }

    public void OnEndCutScene()
    {
        _attacker = null;
        _hitter = null;

        // 모든 컷씬 이벤트 리스너에게 종료 통지
        foreach (var cutSceneEventInvoker in _cutSceneEventInvokers)
        {
            cutSceneEventInvoker.NotifyCutSceneEvents(action ⇒ action?.OnSpecialM

        }
        _cutSceneEventInvokers.Clear();
    }

```

관련 클래스

- [ICutSceneEvent](#)
- [CutSceneController](#)
- [HeroController](#)

CutSceneController

특수 스킬(컷씬) 시퀀스를 제어하는 컨트롤러

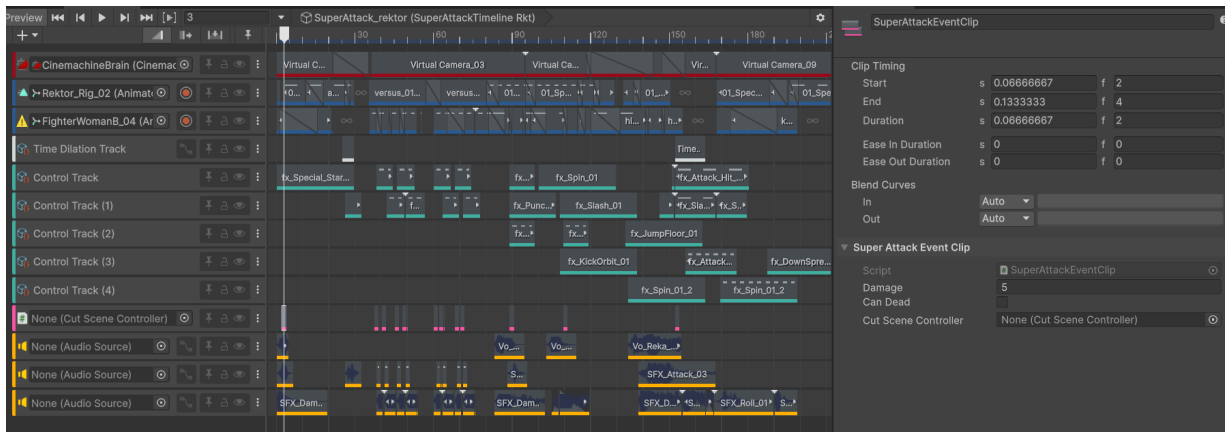
개요

CutSceneController 클래스는 RFist 게임의 **Timeline** 기반 특수 스킬(궁극기)에서 발생하는 이벤트 및 데미지를 처리하는 컨트롤러입니다. **TimelineManager**가 생성한 타임라인(PlayableDirector)에 바인딩되어, 타임라인 재생 중 특정 시점에 데미지 적용 및 컷씬 이벤트를 처리합니다.

역할

- TimelineManager**에서 발생하는 컷씬 이벤트 처리 (시작/종료)
- 컷씬 중 피격자에게 데미지 적용
- 로컬/원격 플레이어 구분에 따른 공격 이벤트 발생

Timeline 패널에서 **CutSceneController** 호출



선언

```
public class CutSceneController : MonoBehaviour
```

멤버

속성

```
/// <summary>  
/// 공격자 네트워크 영웅 오브젝트
```

```

    /// </summary>
    private NetworkHeroObject _attacker;

    /// <summary>
    /// 피격자 네트워크 영웅 오브젝트
    /// </summary>
    private NetworkHeroObject _hitter;

    /// <summary>
    /// 공격자가 로컬 플레이어인지 여부
    /// </summary>
    private bool _attackIsLocal;

    /// <summary>
    /// 컷씬 이벤트 발신자 목록
    /// </summary>
    private readonly HashSet<ICutSceneEventInvoker> _cutSceneEventInvokers = new H

```

메서드

```

    /// <summary>
    /// 컷씬 컨트롤러 초기화
    /// </summary>
    /// <param name="attacker">공격자 네트워크 영웅 오브젝트</param>
    /// <param name="hitter">피격자 네트워크 영웅 오브젝트</param>
    public void Initialize(NetworkHeroObject attacker, NetworkHeroObject hitter)

    /// <summary>
    /// 초기화 여부 확인
    /// </summary>
    /// <returns>초기화 완료 여부</returns>
    private bool IsInitialized()

    /// <summary>
    /// 컷씬 시작
    /// </summary>
    public void StartCutScene()

    /// <summary>
    /// 컷씬 종료 처리
    /// </summary>
    public void OnEndCutScene()

    /// <summary>
    /// 피격자에게 데미지 적용
    /// </summary>
    /// <param name="damage">데미지량</param>
    /// <param name="canDead">사망 가능 여부</param>
    public void ApplyDamageToHitter(int damage, bool canDead)

```

코드 스니펫

컷씬 시작

```
public void StartCutScene()
{
    if (IsInitialized() == false)
    {
        Debug.LogError($"[CutSceneController] Not Initialized");
        return;
    }

    // 공격자가 로컬인지 확인
    _attackIsLocal = !_attacker.IsRemote();

    // 모든 컷씬 이벤트 리스너에게 시작 통지
    foreach (var cutSceneEventInvoker in _cutSceneEventInvokers)
    {
        cutSceneEventInvoker.NotifyCutSceneEvents(action => action?.OnSpecialM
    }
}
```

데미지 적용

```
/// <summary>
/// 피격자에게 데미지 적용
/// </summary>
public void ApplyDamageToHitter(int damage, bool canDead)
{
    // 로컬 공격자인 경우 공격 이벤트 발생
    if (_attackIsLocal)
    {
        var hitInfo = new IHitEvent.HitInfo();
        _attacker.GetBaseHeroAbility().ExternalAttackEvent(hitInfo);
    }

    // 피격자 데미지 계산
    var currentHp = _hitter.GetHeroStatus().GetHp().Current;
    var hp = currentHp - damage;

    // 사망 처리 (최소 1 유지 또는 0으로 설정)
    if (hp <= 0)
    {
        hp = canDead ? 0 : 1;
    }

    // 피격자 체력 변경
    _hitter.GetBaseHeroAbility().ChangeHp(hp);
}
```

기능 설명

컷씬 생명주기

- 1 초기화 (Initialize): 공격자와 피격자 설정, 이벤트 발신자 등록
- 2 시작 (StartCutScene): 로컬 여부 확인, 이벤트 통지
- 3 데미지 처리 (ApplyDamageToHitter): 체력 계산 및 사망 판정
- 4 종료 (OnEndCutScene): 참조 해제, 이벤트 통지, 정리

이벤트 통지 시스템

- 컷씬 시작/종료 시 `ICutSceneEventInvoker`를 통해 이벤트를 통지
- `OnSpecialMoveSequenceStart()`: 컷씬 시작 시 호출
- `OnSpecialMoveSequenceEnd()`: 컷씬 종료 시 호출
- 해당 이벤트는 `HeroControllerStateHandler` 및 `HitPanel`에서 수신

데미지 처리

- 공격자가 로컬 플레이어일시: 공격 이벤트 발생
- 피격자: 공격자의 호출에 따라 체력 계산 및 변경
- 사망 판정: `canDead` 파라미터에 따라 최소 체력 1 유지 또는 0으로 설정
- 네트워크 동기화: 공격자의 로컬 여부에 따라 이벤트 처리 분기

의존성/상속 관계

- `MonoBehaviour`를 상속 받음
- `NetworkHeroObject` 공격자/피격자 오브젝트
- `ICutSceneEventInvoker` 이벤트 발신 인터페이스
- `IHitEvent`를 통해 공격 이벤트 발생
- `BaseHeroAbility`의 메서드를 통해 피격자 체력 변경

사용 예시

`SuperAttackEventBehaviour`에서 `Timeline`의 프레임 처리시 데미지 처리 메서드 호출

```
public override void ProcessFrame(Playable playable, FrameData info, object playerData)
{
    if (!_triggered || info.effectiveWeight <= 0f)
        return;

    // playerData에서 set
    if (cutSceneController == null && playerData is CutSceneController csc)
    {
        cutSceneController = csc;
        //Debug.Log("Fallback - CutSceneController Binding success");
    }

    if (cutSceneController == null)
    {
```

```
        {
            Debug.LogWarning(" cutSceneController null");
            return;
        }

        Debug.Log($" Special Attack : {damage}");
        cutSceneController.ApplyDamageToHitter(damage, canDead);
        _triggered = true;
    }
}
```

관련 클래스

- [NetworkHeroObject](#)
- [ICutSceneEventInvoker](#)
- [ICutSceneEvent](#)
- [HeroControllerStateHandler](#)
- [BaseHeroAbility](#)
- [BaseHeroSkill](#)